

Module-3

Local Search and Adversarial Search

~~Local Search algorithms – Hill-climbing search, Simulated annealing, Genetic Algorithm,~~ Adversarial Search: Game Trees and Minimax Evaluation, Elementary two-players games: tic-tac-toe, Minimax with Alpha-Beta Pruning.

Adversarial Search

Adversarial search in artificial intelligence is a decision-making technique for multi-agent competitive environments where agents have opposing goals.

Adversarial search

- Adversarial search is search when there is an **"enemy" or "opponent"** changing the state of the problem every step in a direction you do not want.
- Examples: Chess, business, trading, war. You change state, but **then you don't control the next state**. Opponent will change the next state in a way.

Key Concepts in Adversarial Search

- **Competitive Environment**
- **Conflicting Goals**
- **Game Theory**
- **Game Tree**
- **Minimax Algorithm**
- **Alpha-Beta Pruning**

How Adversarial Search Works?

- **Modeling the Game**
- **Anticipating Opponent Moves**
- **Evaluating Outcomes**
- **Selecting a Strategy**

Applications

- **Game Playing AI**
- **Business and Trading**

Game Playing

Introduction

- Game playing is one of the oldest sub-field of AI.
- Game playing involves **abstract and pure** form of competition that seems to require intelligence.
- It is easy to represent **the state and actions**.
- To implement game playing is required very little world knowledge.

Why Study Game Playing?

- Games allow us to experiment with easier versions of **real-world situations**
- **Hostile agents** act against our goals
- Games have a finite set of moves
- Games are fairly easy to represent
- Good idea to decide about what to think
- Perfection is unrealistic, must settle for good
- One of the earliest areas of AI
 - Claude Shannon and Alan Turing wrote chess programs in 1950s
- The opponent introduces **uncertainty**
- The environment may contain uncertainty (backgammon)
- Search space too hard to consider exhaustively
 - **Chess has about 10^{40} legal positions**
 - Efficient and effective search strategies even more critical
- Games are fun to target!

Assumptions

- Static or dynamic?
- Fully or partially observable?
- Discrete or continuous?
- Deterministic or stochastic?
- Episodic or sequential?
- Single agent or multiple agent?

Zero-Sum Games

- Focus primarily on “adversarial games”
- Two-player, zero-sum games

As Player 1 gains strength →

← Player 2 loses strength

and vice versa

The sum of the two strengths is always 0.

- The most common used AI technique in game is search.
- Game playing research has contributed ideas on how **to make the best use of time to reach good decisions.**
- Game playing is a search problem defined by:
 - » **Initial state of the game**
 - » **Operators defining legal moves**
 - » **Successor function**
 - » **Terminal test defining end of game states**
 - » **Goal test**
 - » **Path cost/ utility/ payoff function**

Search Applied to Adversarial Games

- Initial state
 - Current board position (description of current game state)
- Operators
 - Legal moves a player can make
- Terminal nodes
 - Leaf nodes in the tree
 - Indicate the game is over
- Utility function
 - Payoff function
 - Value of the outcome of a game
 - Example: tic tac toe, utility is -1, 0, or 1

Using Search

- Search could be used to find a perfect sequence of moves except the following problems arise:
 - There exists an adversary who is trying to minimize your chance of winning every other move
 - You cannot control his/her move
 - Search trees can be very large, but you have finite time to move
 - Chess has 10^{40} nodes in search space
 - With single-agent search, can afford to wait
 - Some two-player games have time limits
 - Solution?
 - Search to n levels in the tree (n ply)
 - Evaluate the nodes at the nth level
 - Head for the best looking node

Characteristics of game playing

- **There is always an “Unpredictable” opponent**
 - Due to their uncertainty.
 - He/she also wants to win.
 - Solution for each problem is a strategy, which specifies a move for every possible opponent reply.
- **Time Limits**
 - Games are often played under strict time constraints and therefore must be very effectively handled.
 - There are special games where the two players have exactly opposite goals.
 - It can be divided into two types.
 - **Perfect Information games** (where both players have access to the same information)
 - **Ex: Chess**
 - **Imperfect Information games** (different information can be accessed)
 - **Ex:tic-tac-toe, Battleship, blind, Bridge**

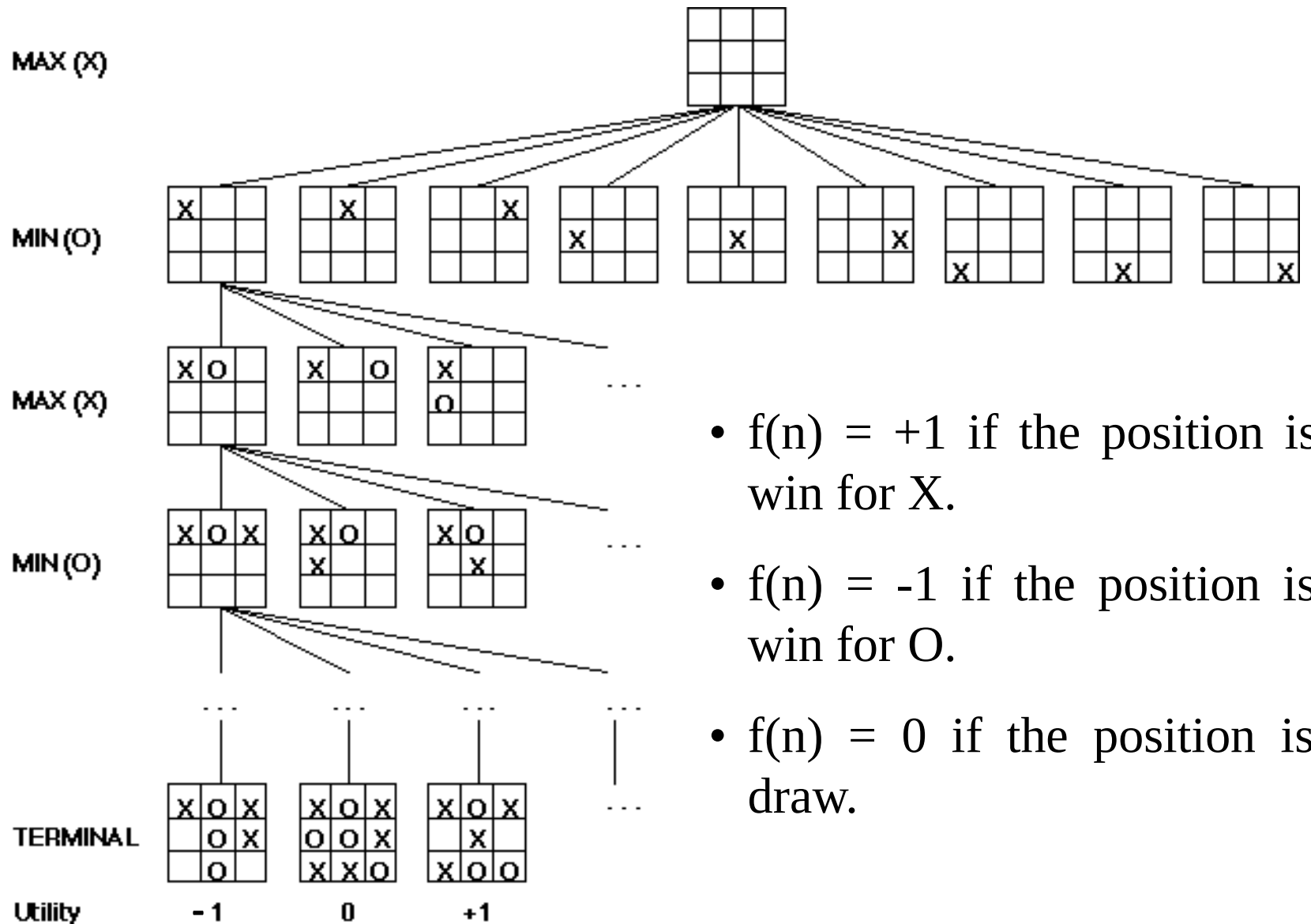
Optimal Decisions in Games

- MIN and MAX are two game players
- First move performed by MAX then turn is given to MIN and so on until the game is over
- A loser need to pay penalties and game points a credits are gifted to the winning player, at end of the game
- A game as search problem and broadly defined by following components
 1. **The Initial State:** It is start state position on the board, which leads to further move.
 2. **A successor function:** It is collection of (move, state) pairs, each specifies a legal move and output state. Here actions are considered.
 3. **Terminal Test:** It is final state which declares that the game is over or ended or terminal states.

Game Trees

- **Game Tree:** Game tree is graphical representation of the initial state and legal moves for each side (two players – one by one) for a specific game
- MAX has nine possible move from initial state(Tic tac toe).
- MAX with X and MIN with O symbol. The leaf nodes are entitled by utility value
- The MAX is assumed with high values and MIN is with low values. The best move is always determined by MAX by using search tree
- **Optimal Strategies**
 - A sequence of moves which achieves Win state/ Terminal state/ Goal state in search problem
- **Optimal Strategy in Game**
 - It is techniques which always leads to superior that any other strategy as opponent is playing in perfect manner

Game tree for Tic-Tac-Toe: 2-player, deterministic



- $f(n) = +1$ if the position is a win for X.
- $f(n) = -1$ if the position is a win for O.
- $f(n) = 0$ if the position is a draw.

Minimax Rule

- Goal of game tree search: to determine **one move** for Max player that **maximizes** the **guaranteed payoff** for a given game tree for MAX

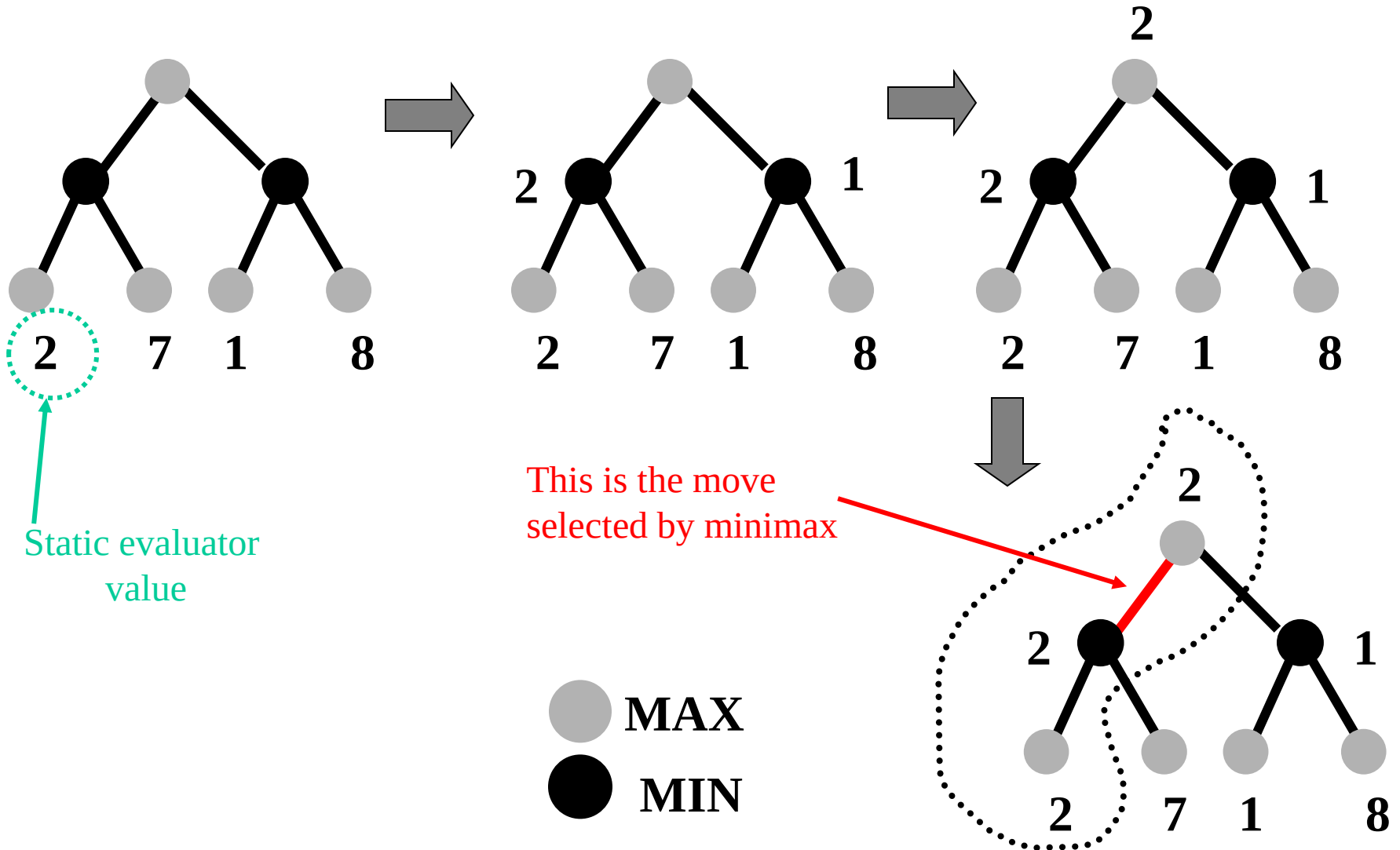
Regardless of the moves the MIN will take

- The value of each node (Max and MIN) is determined by (back up from) the values of its children
- MAX plays the worst case scenario:
 - Always assume MIN to take moves to maximize his pay-off (i.e., to minimize the pay-off of MAX)
- For a MAX node, the backed up value is the **maximum** of the values associated with its children
- For a MIN node, the backed up value is the **minimum** of the values associated with its children

MINIMAX Procedure

- Starting from the **leaves of the tree** (with final scores with respect to one player, MAX), and go backwards towards the root.
- At each step, one player (MAX) takes the action that leads to the highest score, while the other player (MIN) takes the action that leads to the lowest score.
- All nodes in the tree will be scored, and the path from root to the actual result is the one on which all nodes have the same score.
- We are MAX - trying to maximise our score / move to best state. Opponent is MIN - tries to minimise our score / move to worst state for us.

Minimax Search



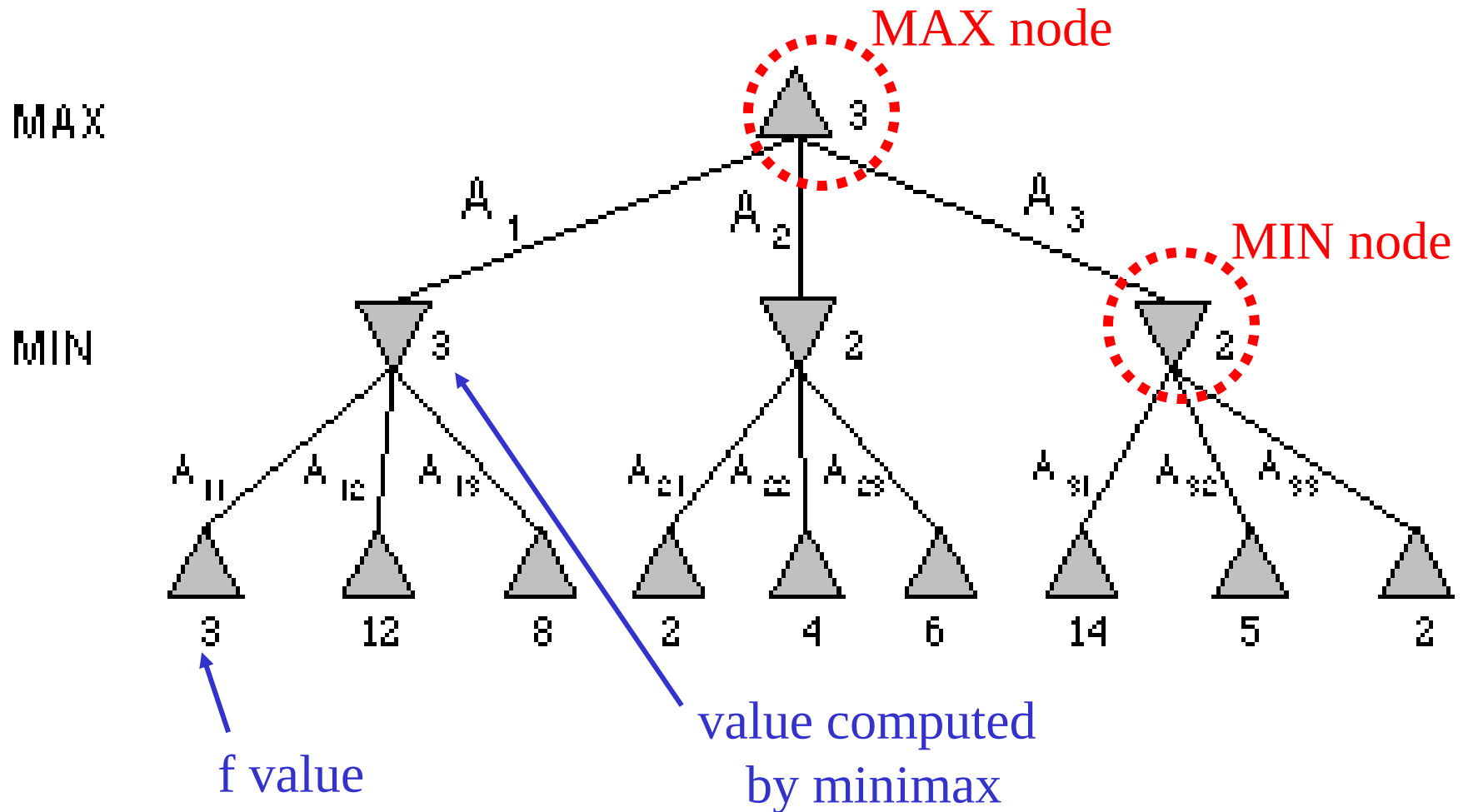
Comments on Minimax search

- The search is **depth-first** with the given depth (ply) as the limit
 - Time complexity: $O(b^d)$
 - Linear space complexity
- Performance depends on
 - Quality of evaluation functions (domain knowledge)
 - Depth of the search (computer power and search algorithm)
- Different from ordinary state space search
 - Not to search for a solution but for one move only
 - No cost is associated with each arc
 - MAX does not know how MIN is going to counter each of his moves
- Minimax rule is a basis for other game tree search algorithms

Minimax

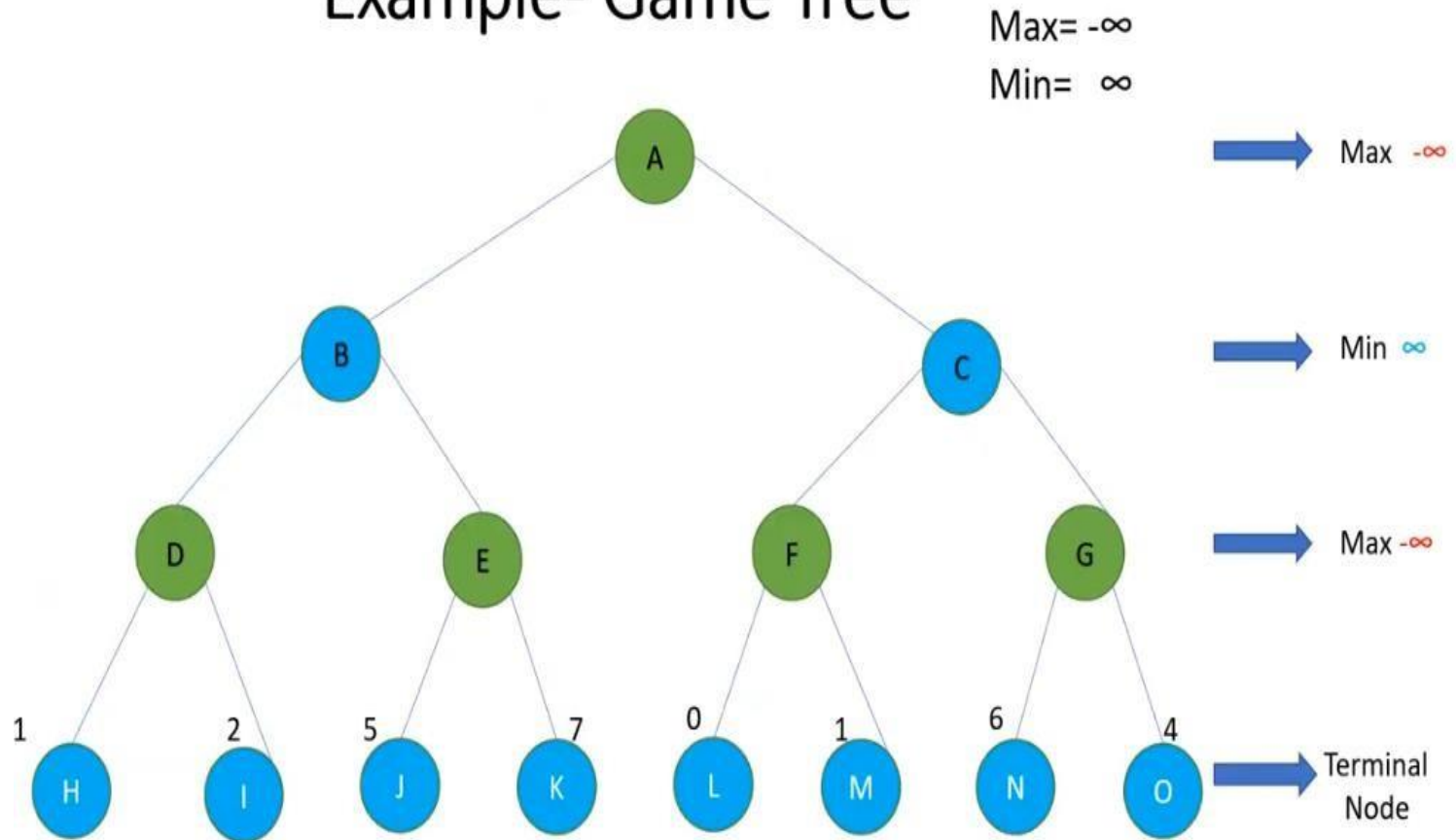
- Perfect play for deterministic games
- Idea: choose move to position with highest **minimax value**
= best achievable payoff against best play
- E.g., 2-ply game:

Minimax Tree



Minimax

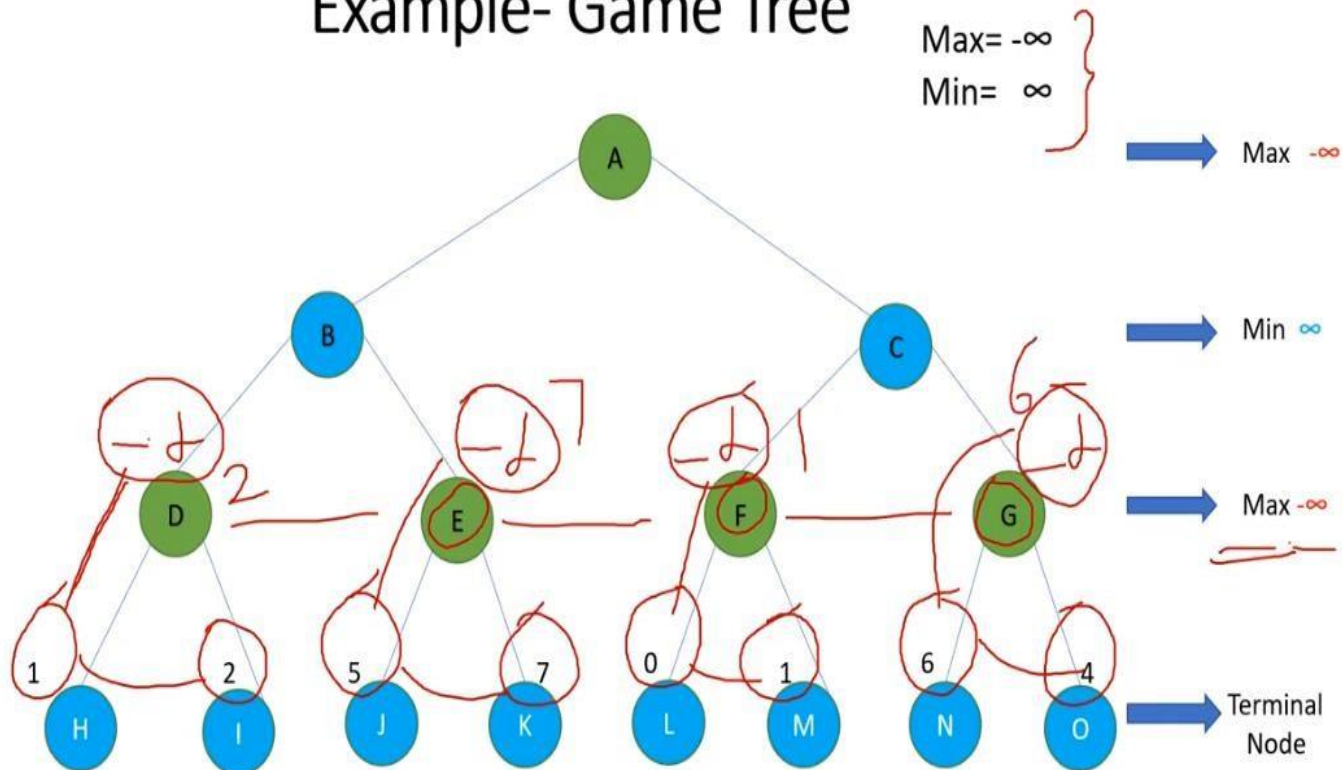
Example- Game Tree



Minimax

- Node D $\rightarrow \max(1, -\infty) \Rightarrow \max(1, 2) = 2$
- Node E $\rightarrow \max(5, -\infty) \Rightarrow \max(5, 7) = 7$
- Node F $\rightarrow \max(0, -\infty) \Rightarrow \max(0, 1) = 1$
- Node G $\rightarrow \max(6, -\infty) = \max(6, 4) = 6$

Example- Game Tree



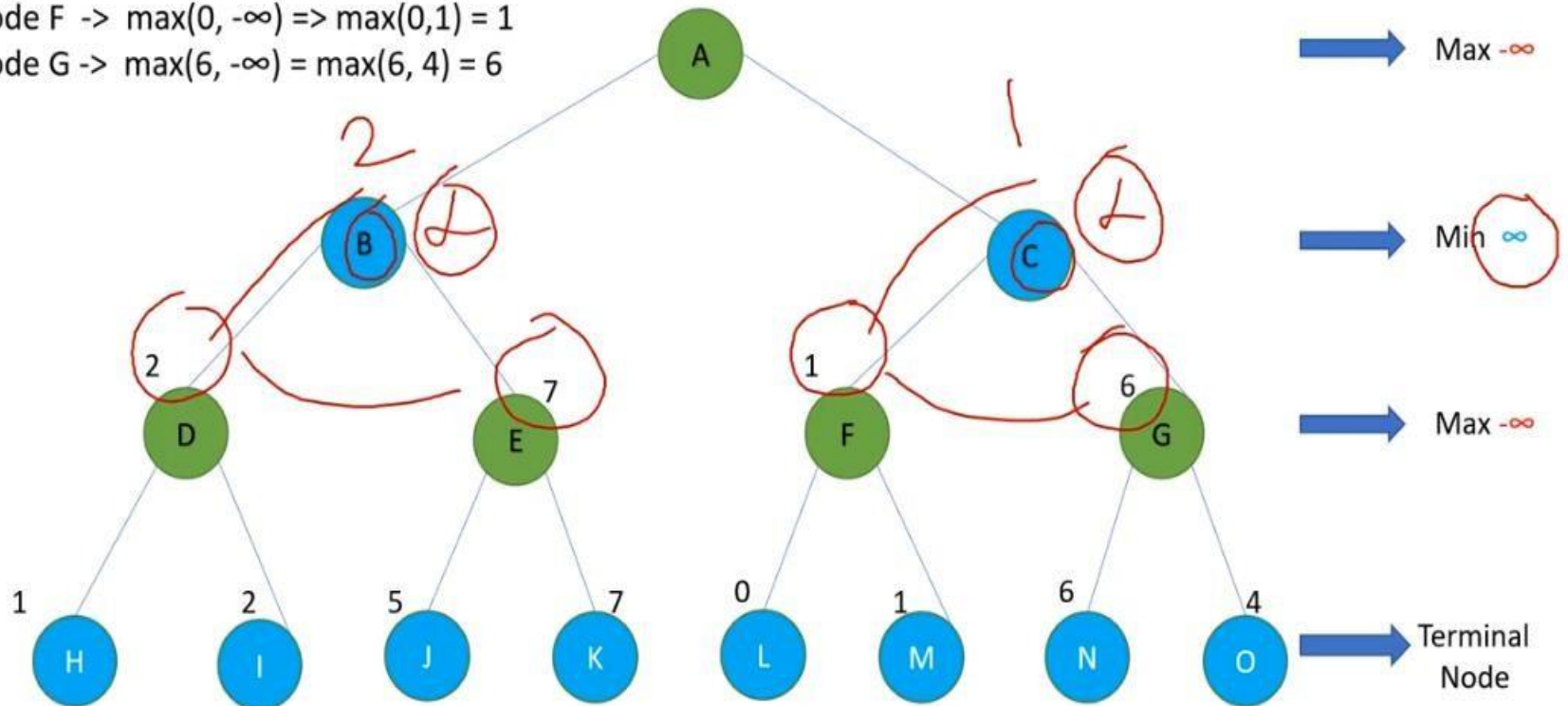
Minimax

Example

- Node D $\rightarrow \max(1, -\infty) \Rightarrow \max(1, 2) = 2$
- Node E $\rightarrow \max(5, -\infty) \Rightarrow \max(5, 7) = 7$
- Node F $\rightarrow \max(0, -\infty) \Rightarrow \max(0, 1) = 1$
- Node G $\rightarrow \max(6, -\infty) = \max(6, 4) = 6$

Max = $-\infty$

Min = ∞



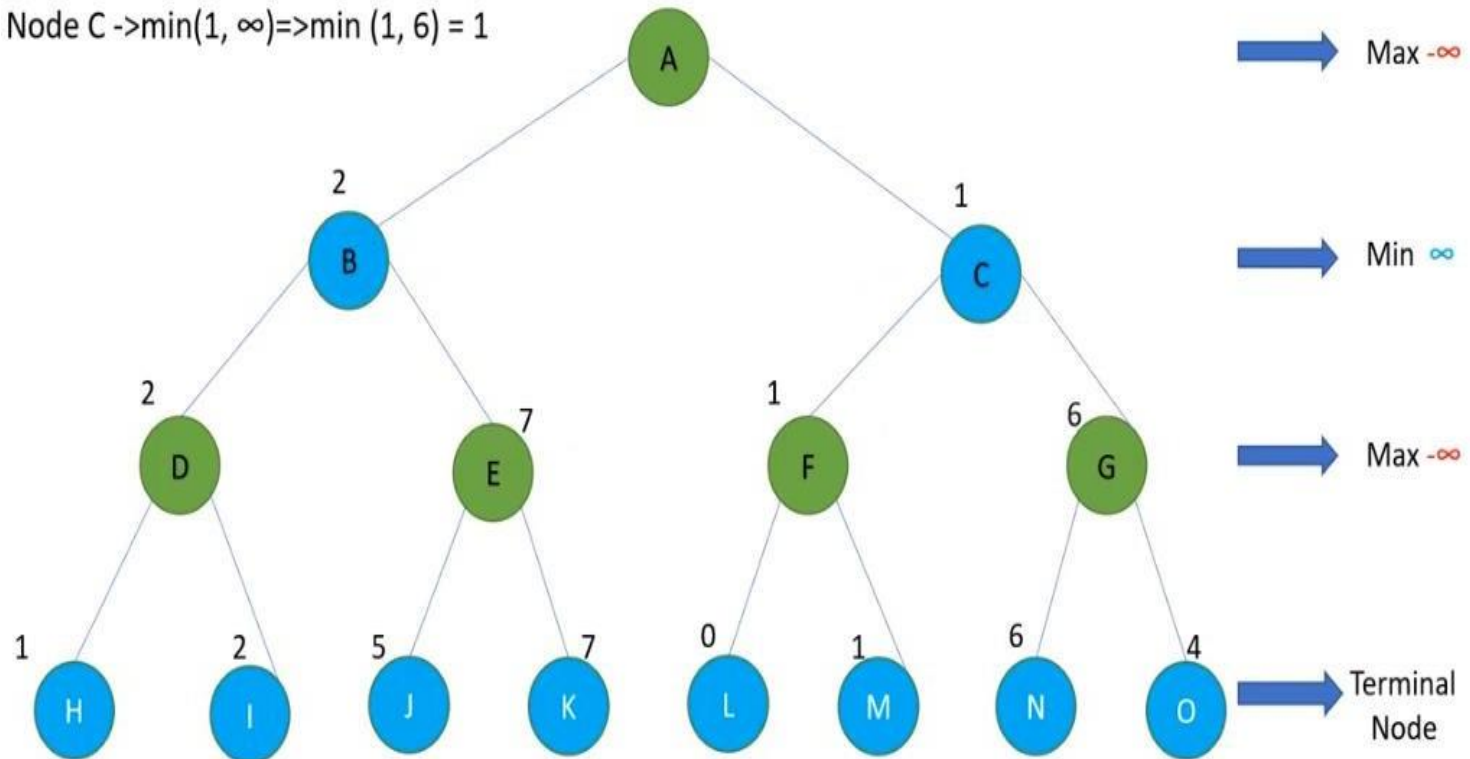
Minimax

Example

Node B $\rightarrow \min(2, \infty) \Rightarrow \min(2, 7) = 2$
Node C $\rightarrow \min(1, \infty) \Rightarrow \min(1, 6) = 1$

Max = $-\infty$

Min = ∞



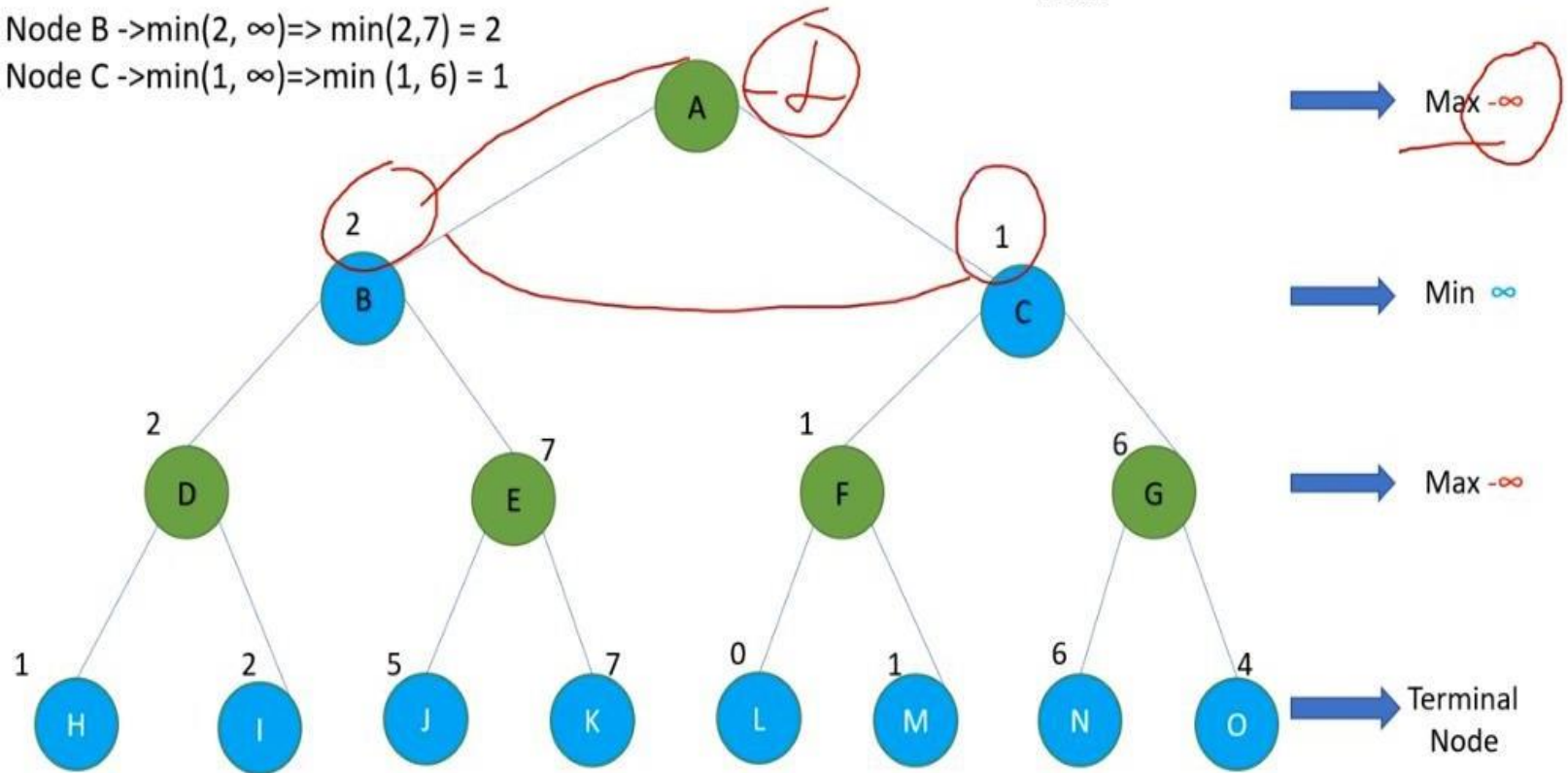
Minimax

Example

Node B $\rightarrow \min(2, \infty) \Rightarrow \min(2, 7) = 2$
Node C $\rightarrow \min(1, \infty) \Rightarrow \min(1, 6) = 1$

Max = $-\infty$

Min = ∞



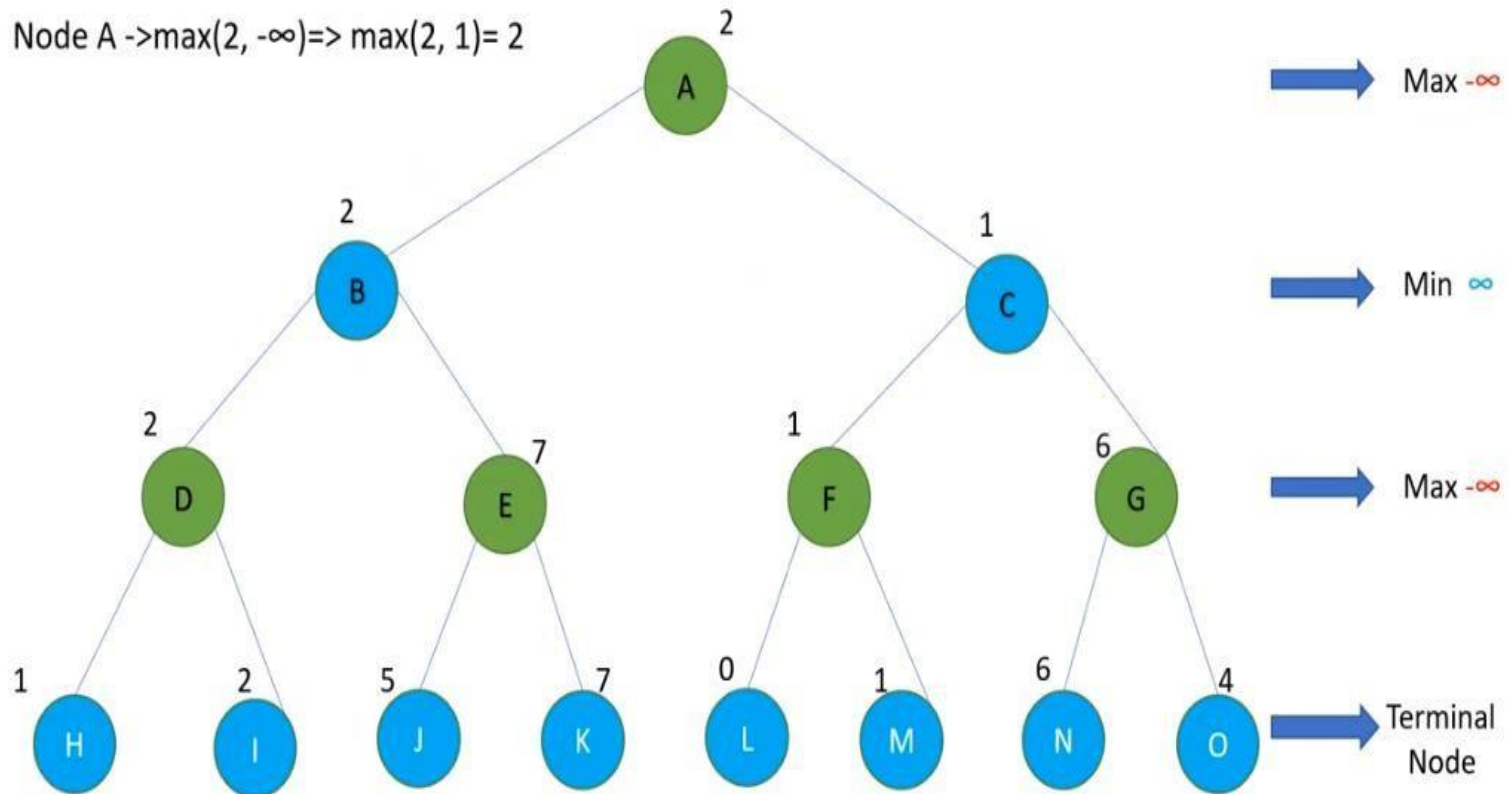
Minimax

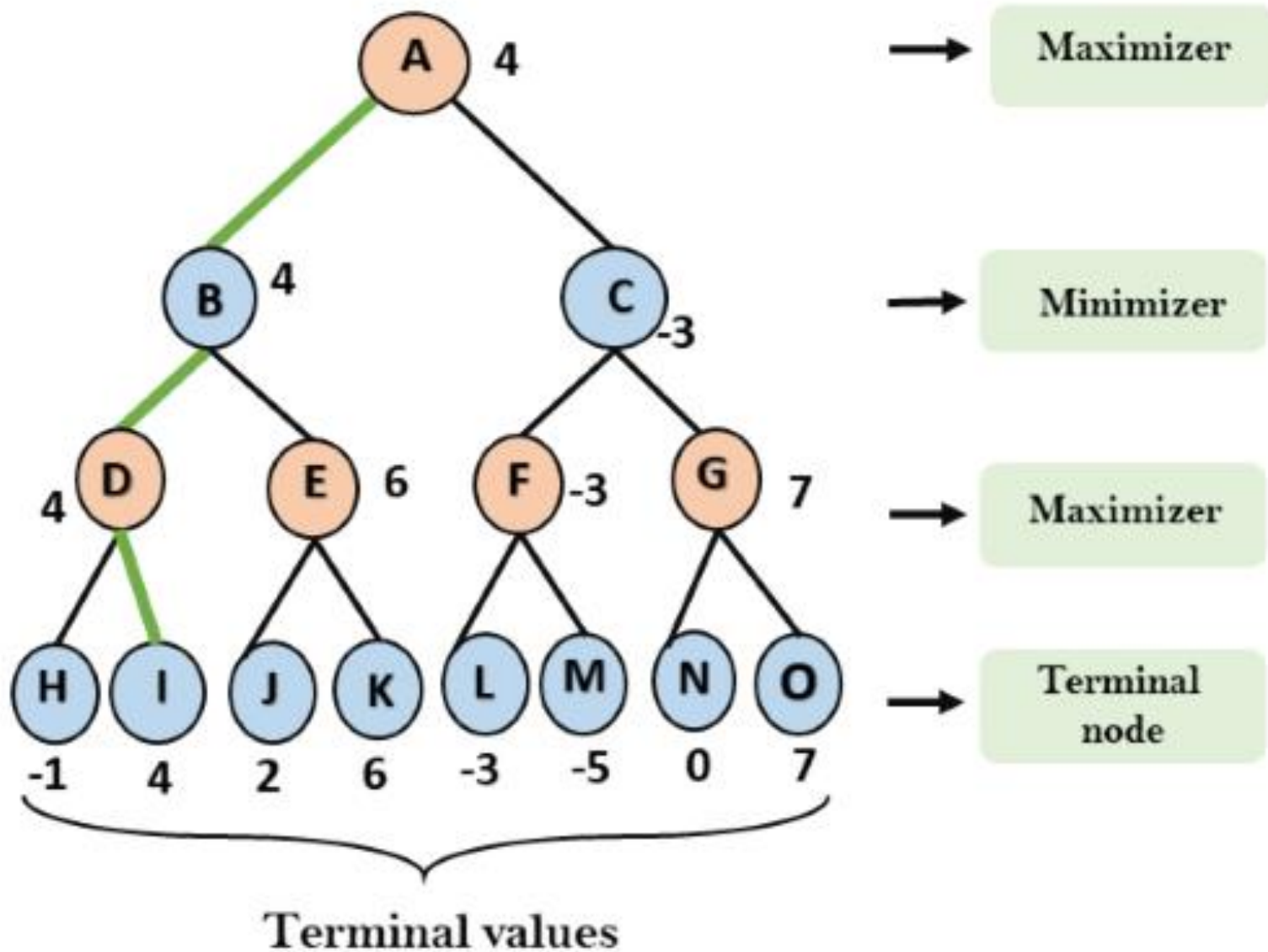
Example

Max = $-\infty$

Min = ∞

Node A $\rightarrow \max(2, -\infty) \Rightarrow \max(2, 1) = 2$





Minimax algorithm

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow \infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

return *v*

Properties of minimax

- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (**depth-first exploration**)

Alpha-Beta pruning



Why do we need **Alpha-beta pruning with MiniMax algorithm?**

- Alpha-beta pruning is a crucial optimization technique used in **conjunction with the minimax algorithm**, rather than a replacement for it.
- The core reason for its necessity lies in **the computational efficiency it provides, especially in games with large search spaces.**

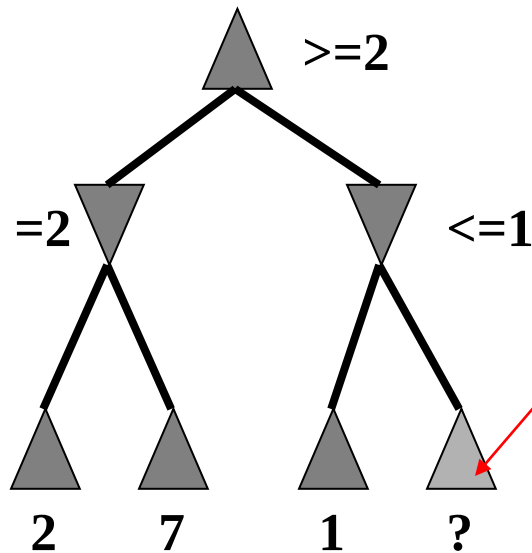
➤ **Reduces Computation Time**

➤ **Allows Deeper Search**

➤ **Maintains Optimality**

Alpha-Beta pruning

- We can **improve on the performance** of the minimax algorithm through alpha-beta pruning.
- Basic idea: *“If you have an idea that is surely bad, don't take the time to see how truly awful it is.”* -- Pat Winston



- We don't need to compute the value at this node.
- No matter what it is, it can't effect the value of the root node.

Alpha-Beta pruning

- Traverse the search tree in **depth-first order**
- At each **Max** node n , **alpha(n)** = maximum value found so far
 - **Start with -infinity and only increase**
 - Increases if a child of n returns a value greater than the current alpha
 - Serve as a tentative lower bound of the final pay-off
- At each **Min** node n , **beta(n)** = minimum value found so far
 - **Start with infinity and only decrease**
 - Decreases if a child of n returns a value less than the current beta
 - Serve as a tentative upper bound of the final pay-off

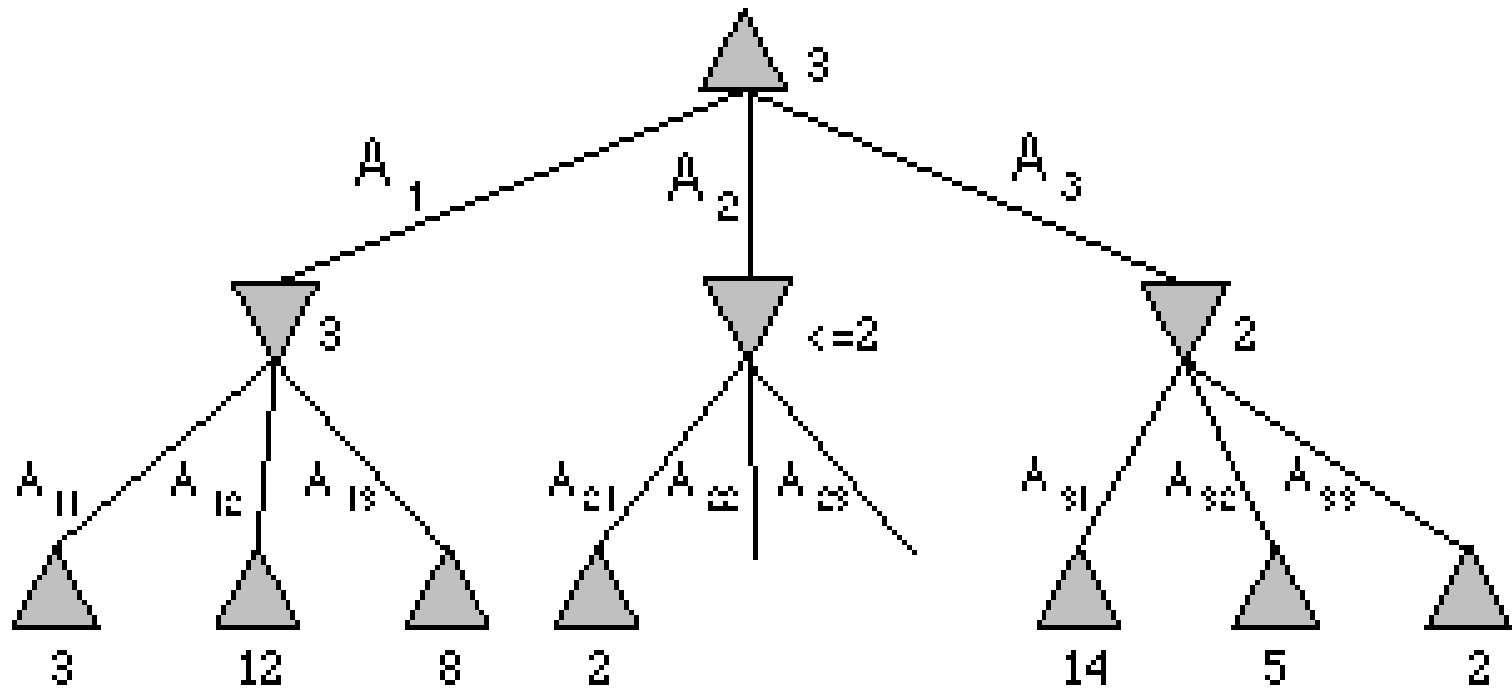
Alpha-Beta pruning

- **Alpha cutoff:** Given a Max node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if **$\alpha(n) \geq \beta(n)$**
(alpha increases and passes beta from below)
- **Beta cutoff.:** Given a Min node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if **$\beta(n) \leq \alpha(n)$**
(beta decreases and passes alpha from above)
- Carry alpha and beta values down during search
Pruning occurs whenever $\alpha \geq \beta$

Alpha-Beta pruning

MAX

MIN



Alpha-Beta pruning

- Two functions **recursively call each other**

```
function MAX-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    alpha := max{alpha, MIN-value(n', alpha, beta)};
    if alpha >= beta then return beta  /* pruning */
  end{do}
  return alpha
```

```
function MIN-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    beta := min{beta, MAX-value(n', alpha, beta)};
    if beta <= alpha then return alpha  /* pruning */
  end{do}
  return beta
```

initiation :
$\alpha := -\infty$
$\beta := \infty$

Effectiveness of Alpha-beta pruning

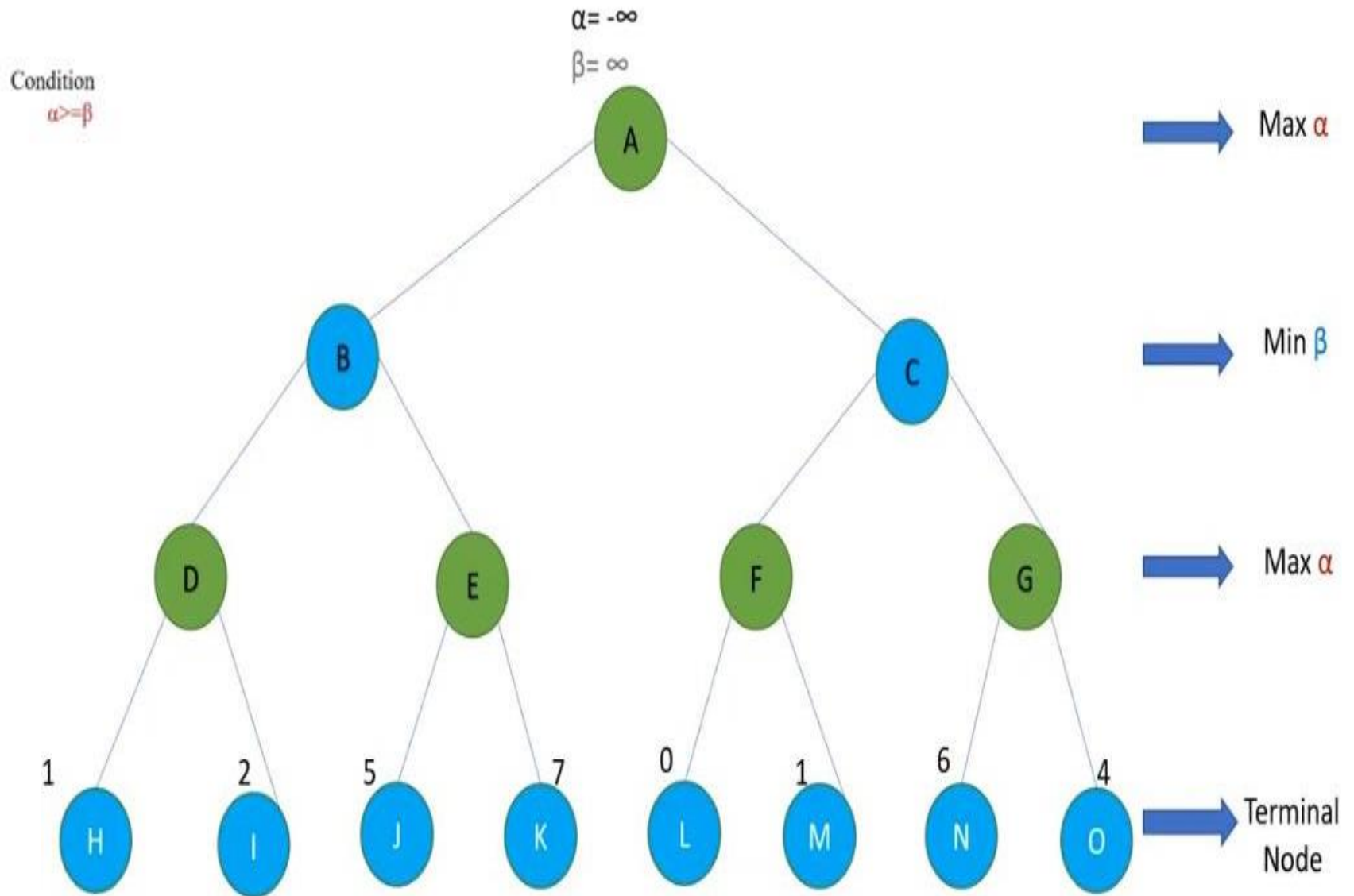
- Alpha-Beta is **guaranteed to compute the same value** for the root node as computed by Minimax.
- **Worst case: NO pruning, examining $O(b^d)$ leaf nodes**, where each node has b children and a d -ply search is performed
- **Best case: examine only $O(b^{(d/2)})$ leaf nodes.**
 - You can **search twice as deep as Minimax!** Or the branch factor is $b^{(1/2)}$ rather than b .
- **Best case** is when each player's best move is the leftmost alternative, i.e. at **MAX nodes** the child with the largest value generated first, and at **MIN nodes** the child with the smallest value generated first.
- In **Deep Blue**, they found empirically that Alpha-Beta pruning meant that **the average branching factor** at each node was about 6 instead of about 35-40

Alpha-Beta pruning

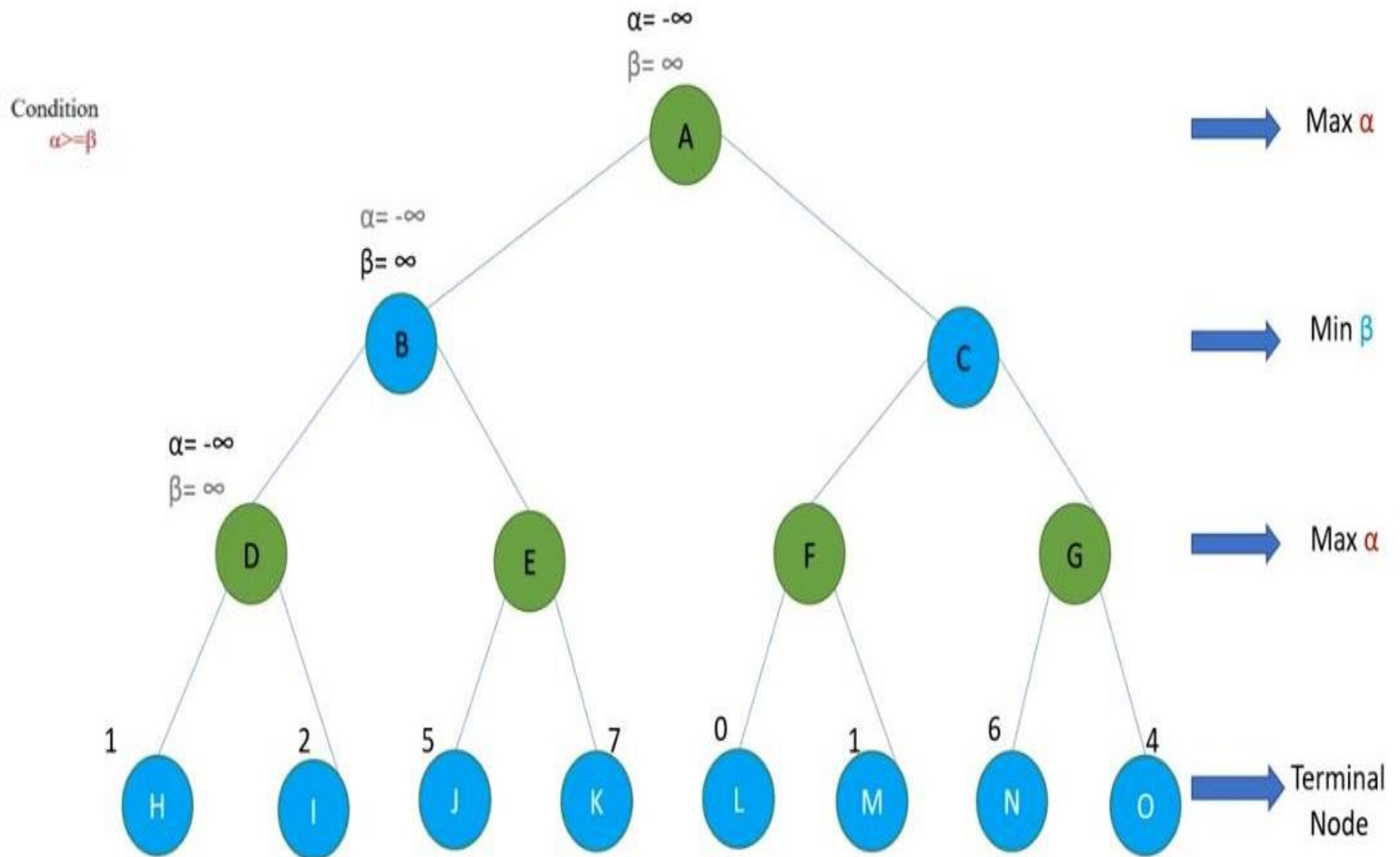
- The basic idea of alpha-beta cutoffs is “**It is possible to compute the correct minimax decision without looking at every node in the search tree**”
- Allow the search process **to ignore the portion of the search tree that makes no difference to the final choice**
- General principle of α - β pruning is
 - Consider a **node n somewhere** in the tree, such that a player has a chance to move to this node.
 - If the player has a better chance m either at the parent node of n then n will never be reached in actual play.

- **Alpha:** The highest value that the maximizer can guarantee by making some move at the current node OR at some node earlier on the path.
- **Beta:** The lowest value that the minimizer can guarantee by making some move at the current node OR at some node earlier on the path to this node.

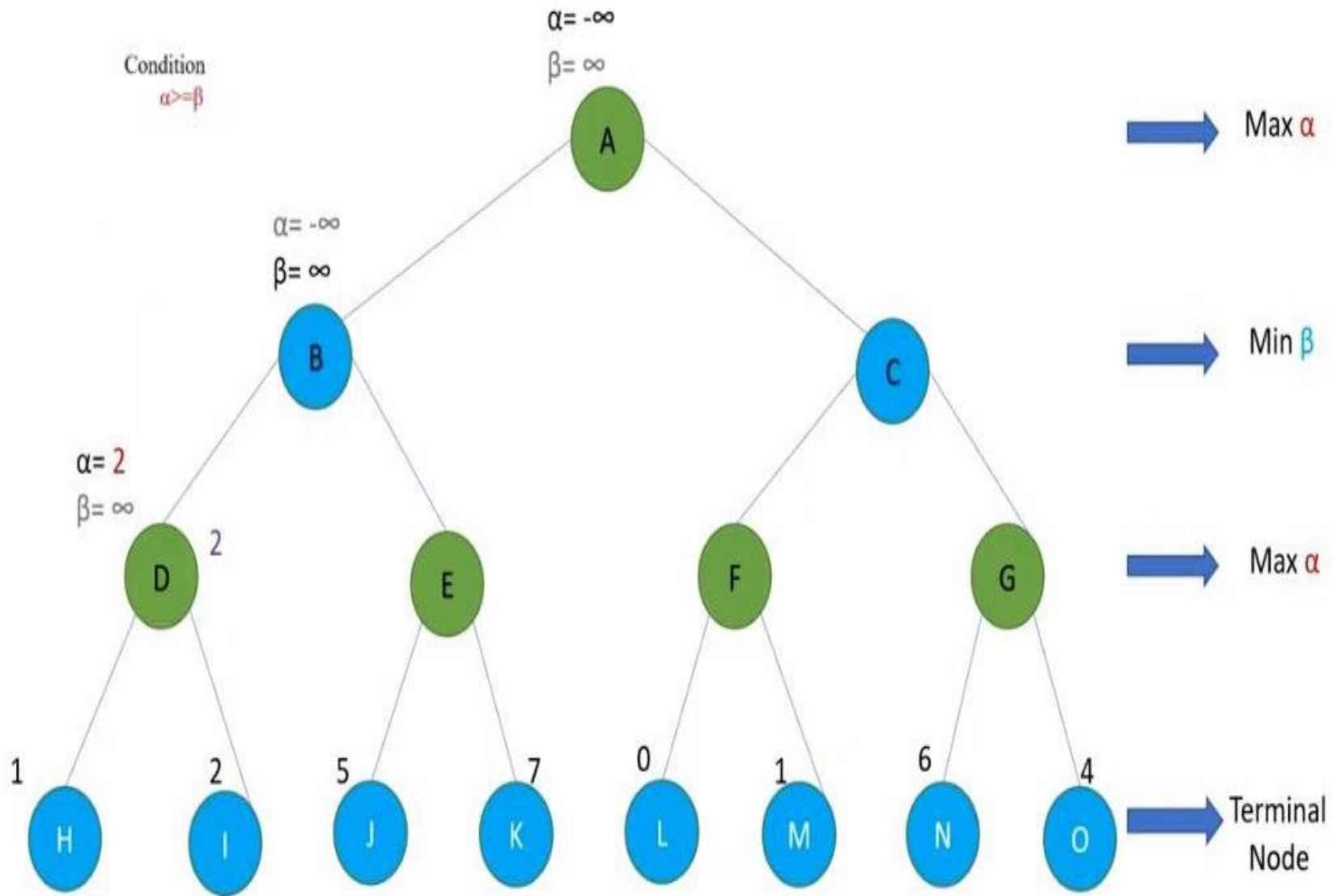
Alpha-Beta pruning: Example



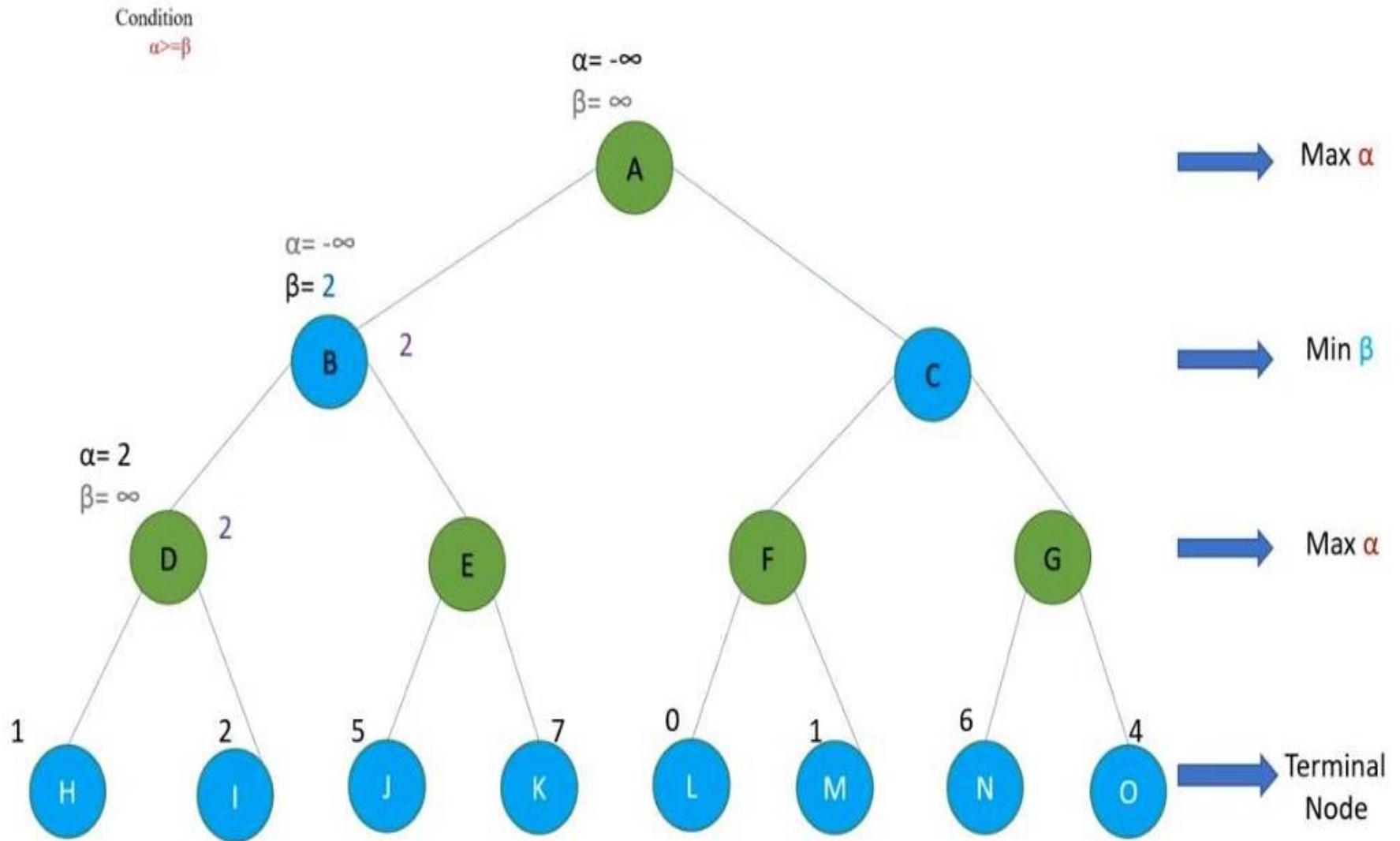
Alpha-Beta pruning: Example



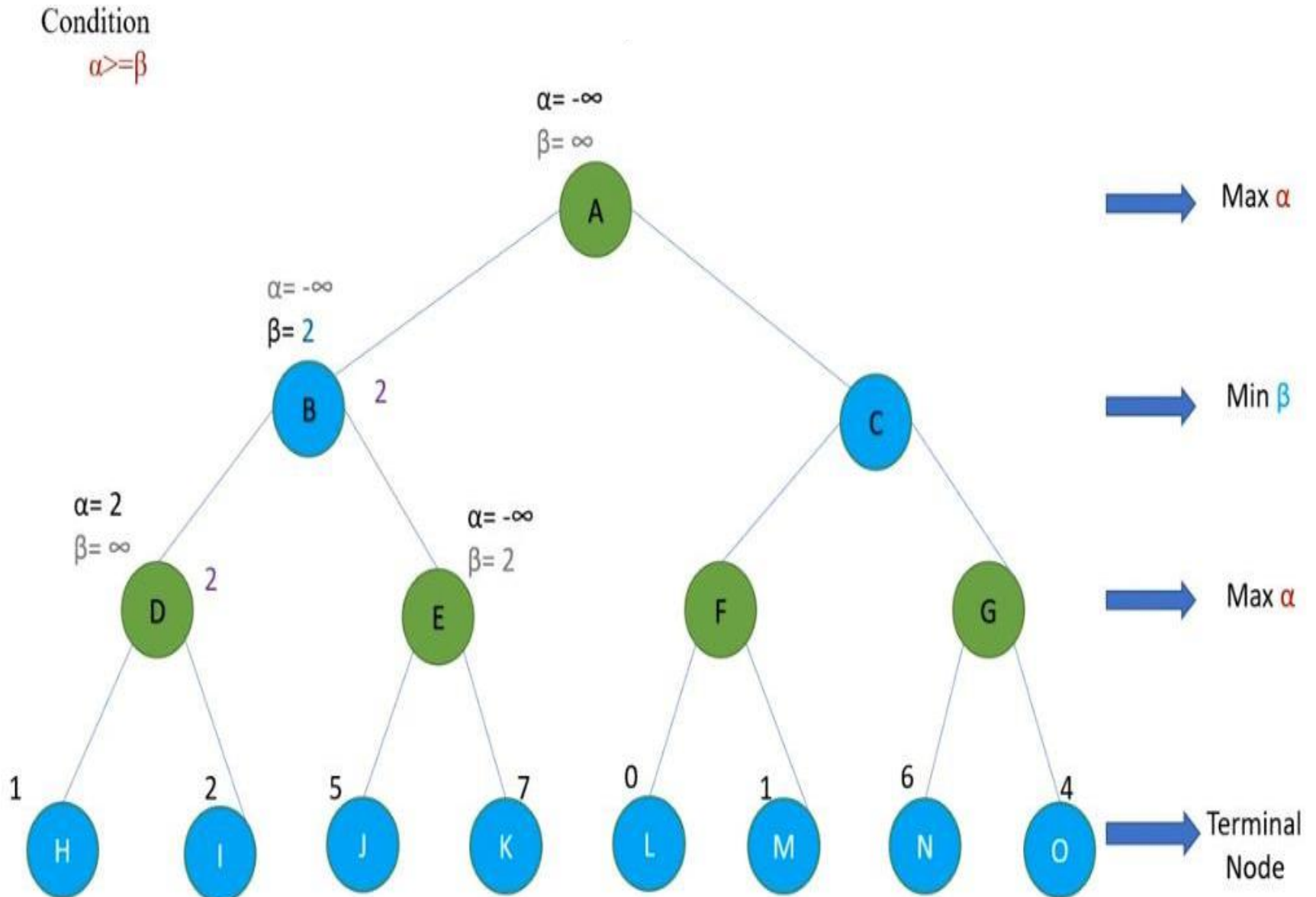
Alpha-Beta pruning: Example



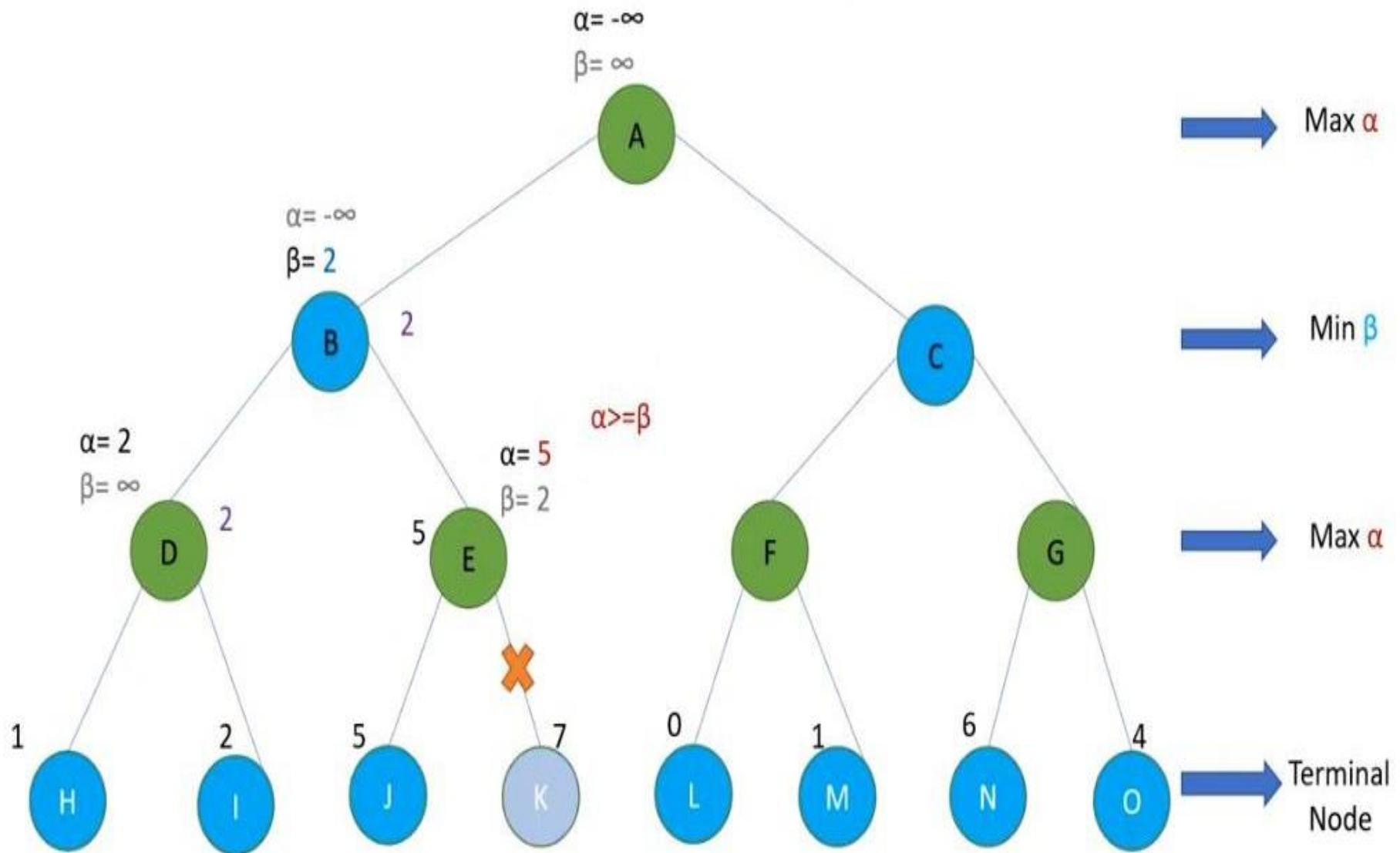
Alpha-Beta pruning: Example



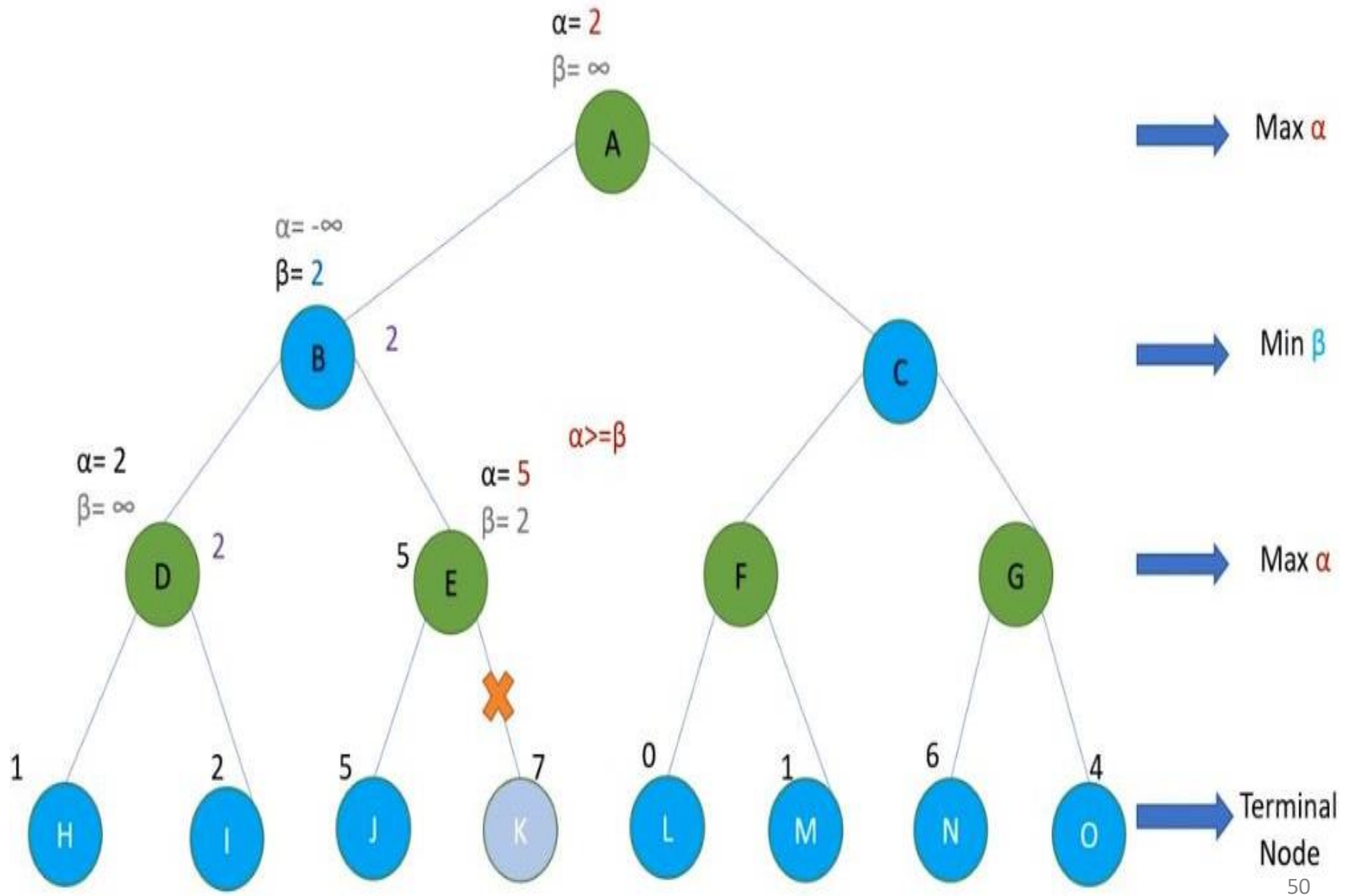
Alpha-Beta pruning: Example



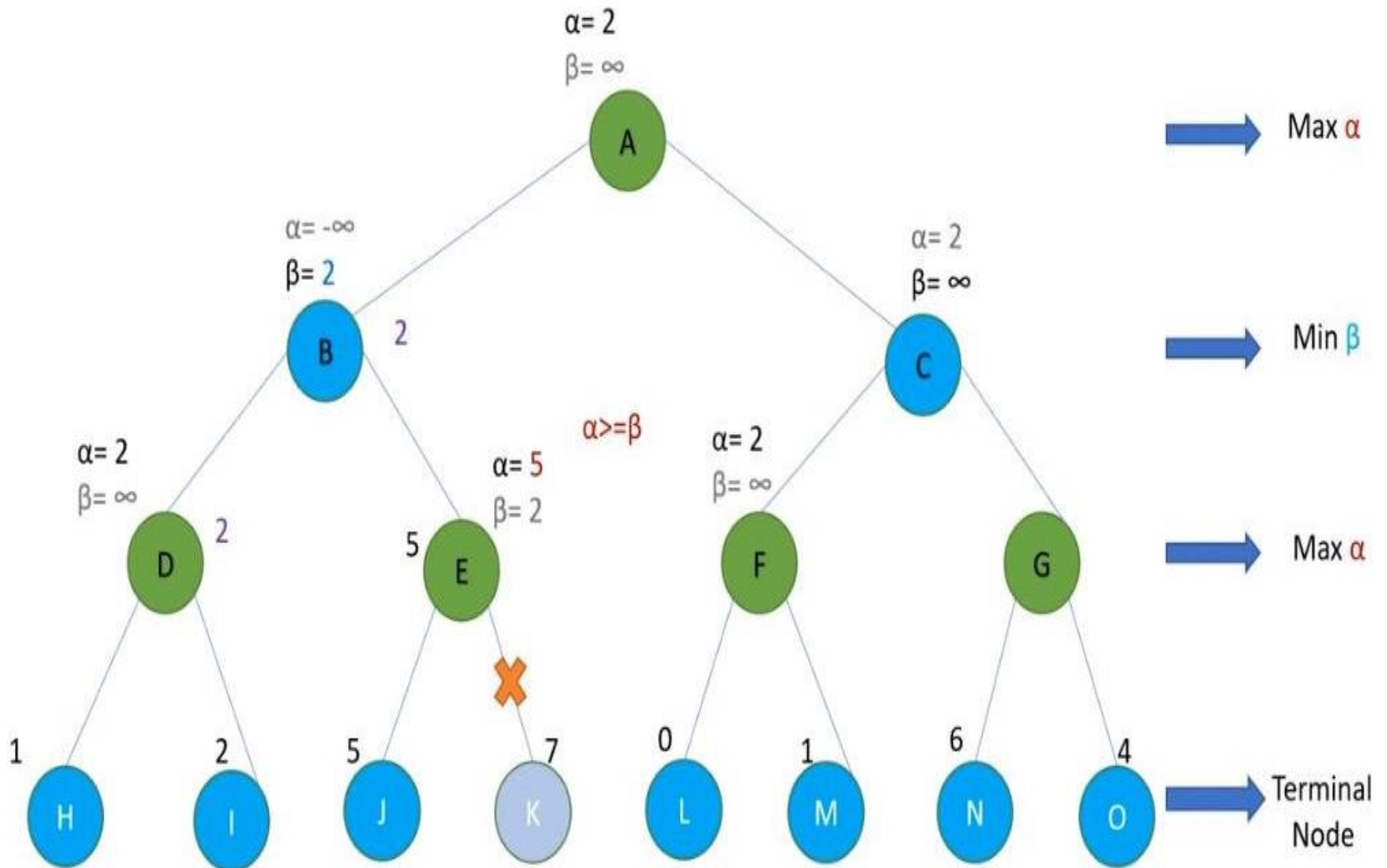
Alpha-Beta pruning: Example



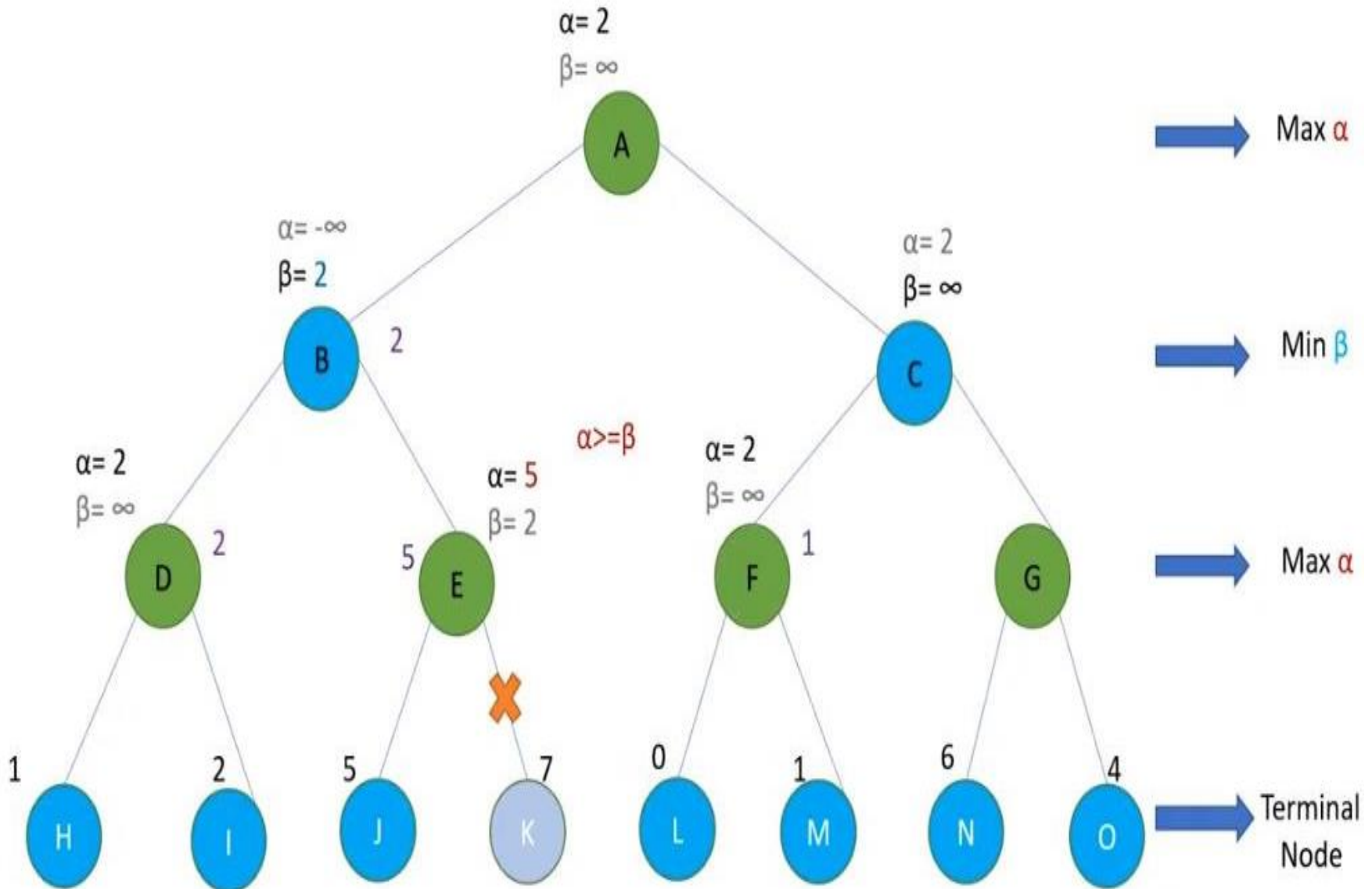
Alpha-Beta pruning: Example



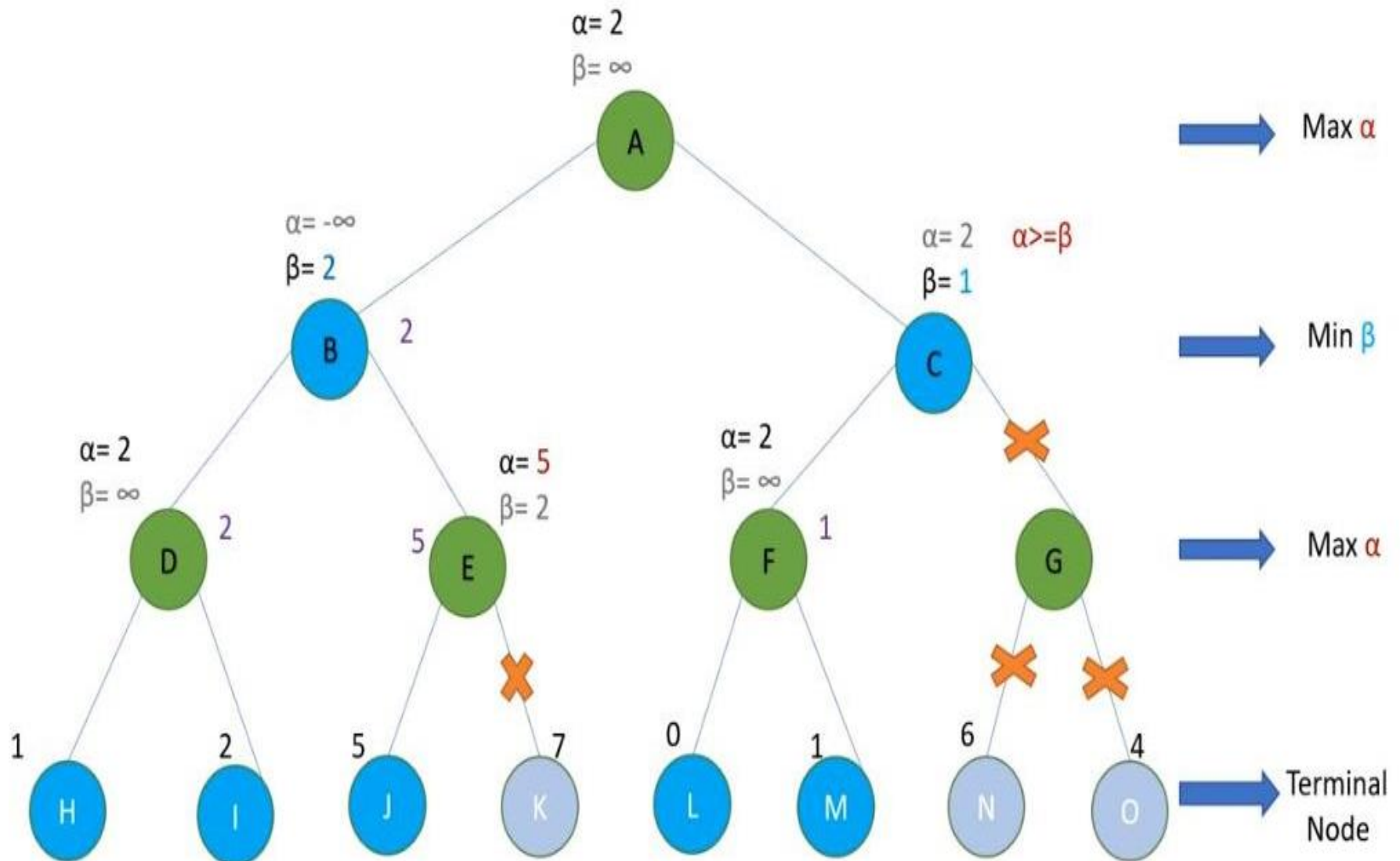
Alpha-Beta pruning: Example



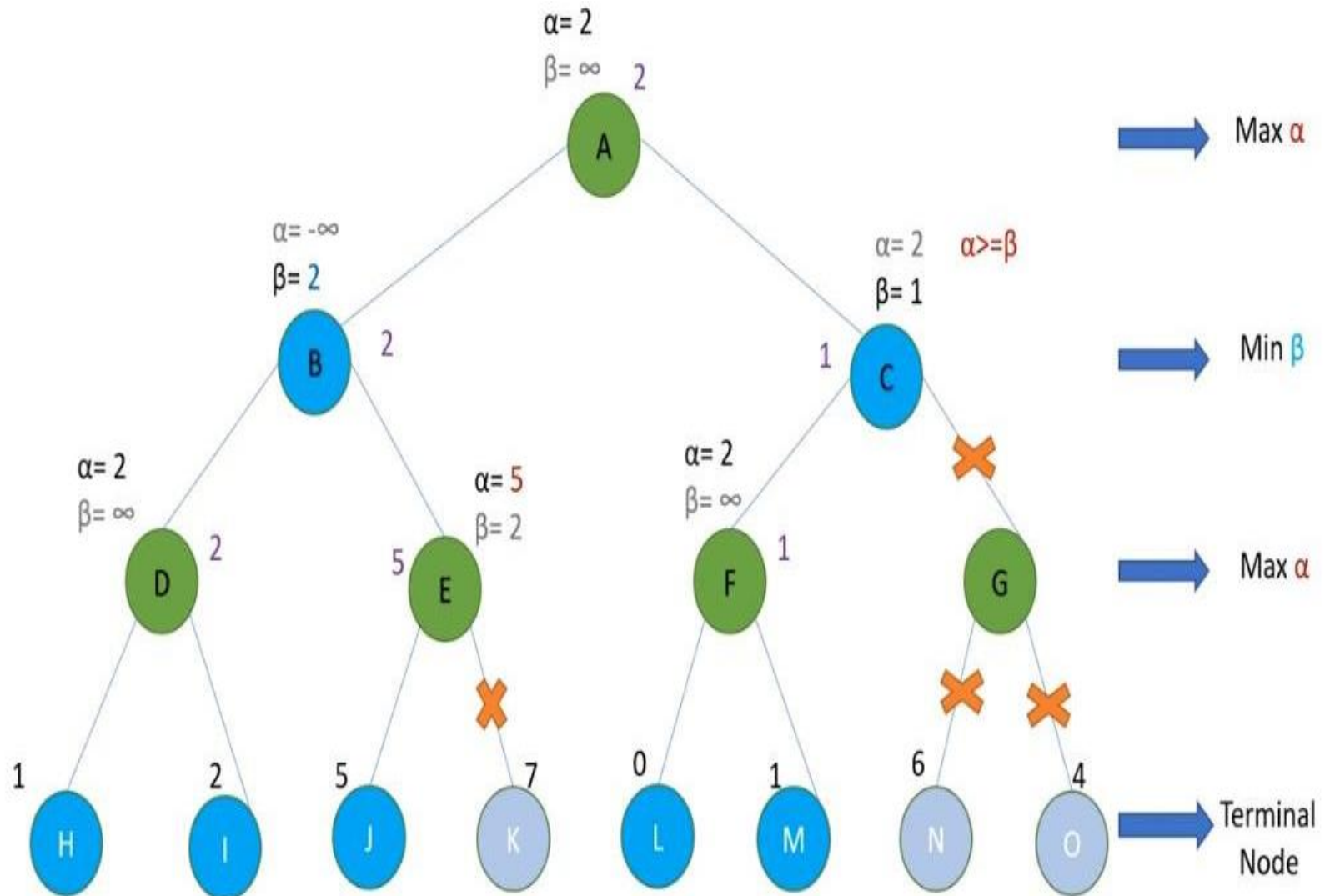
Alpha-Beta pruning: Example



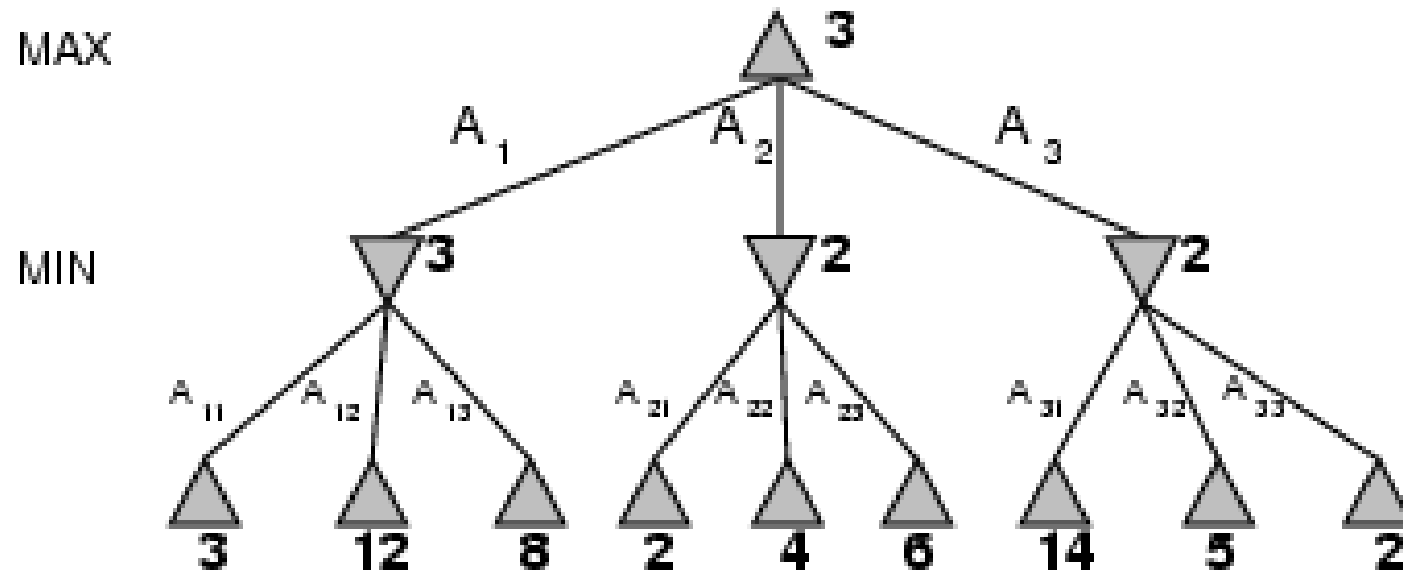
Alpha-Beta pruning: Example



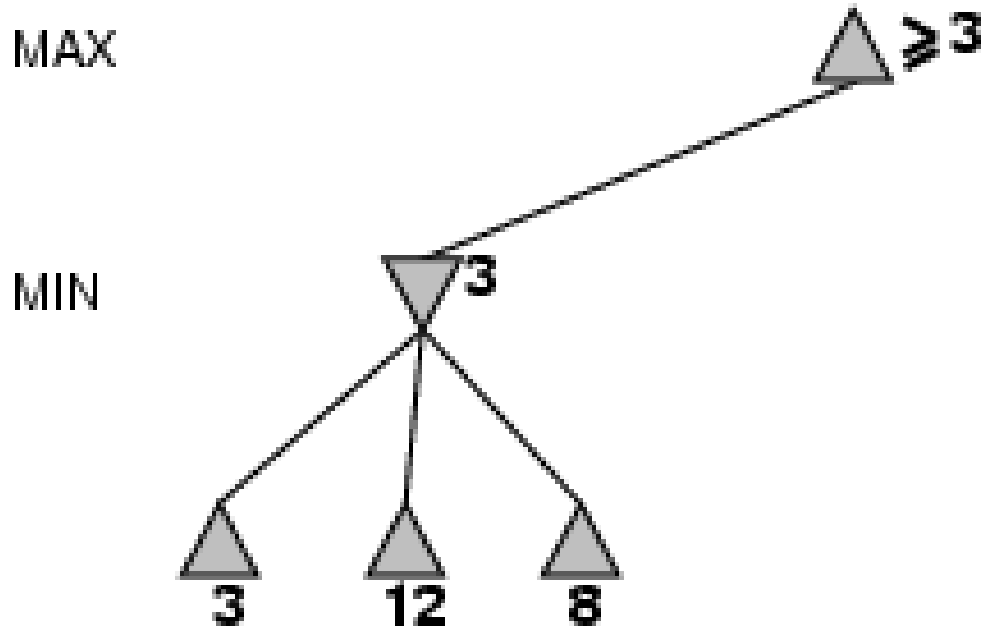
Alpha-Beta pruning: Example



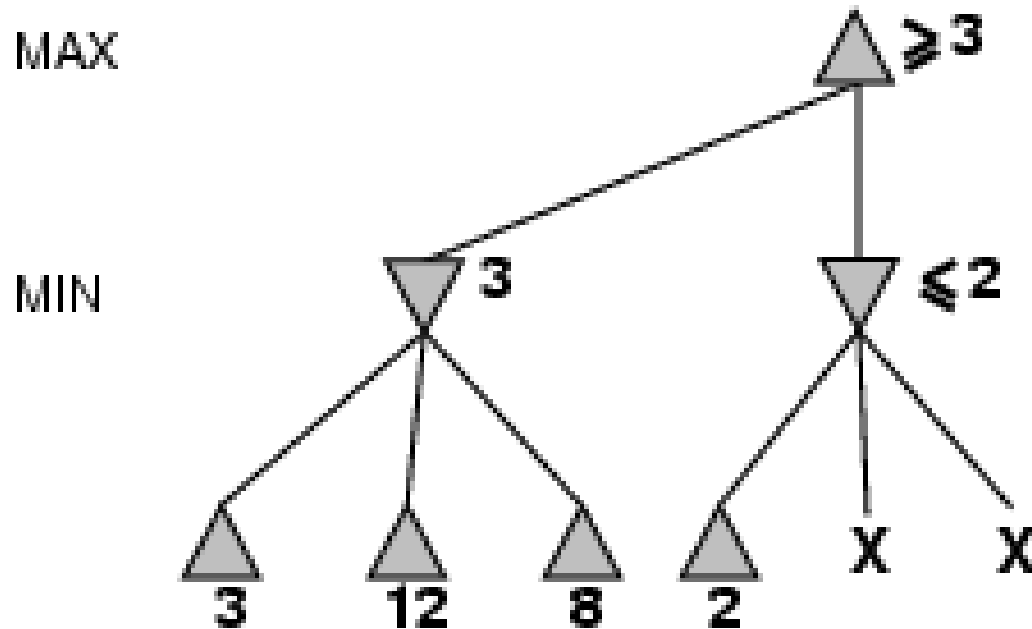
Alpha-Beta pruning: Example



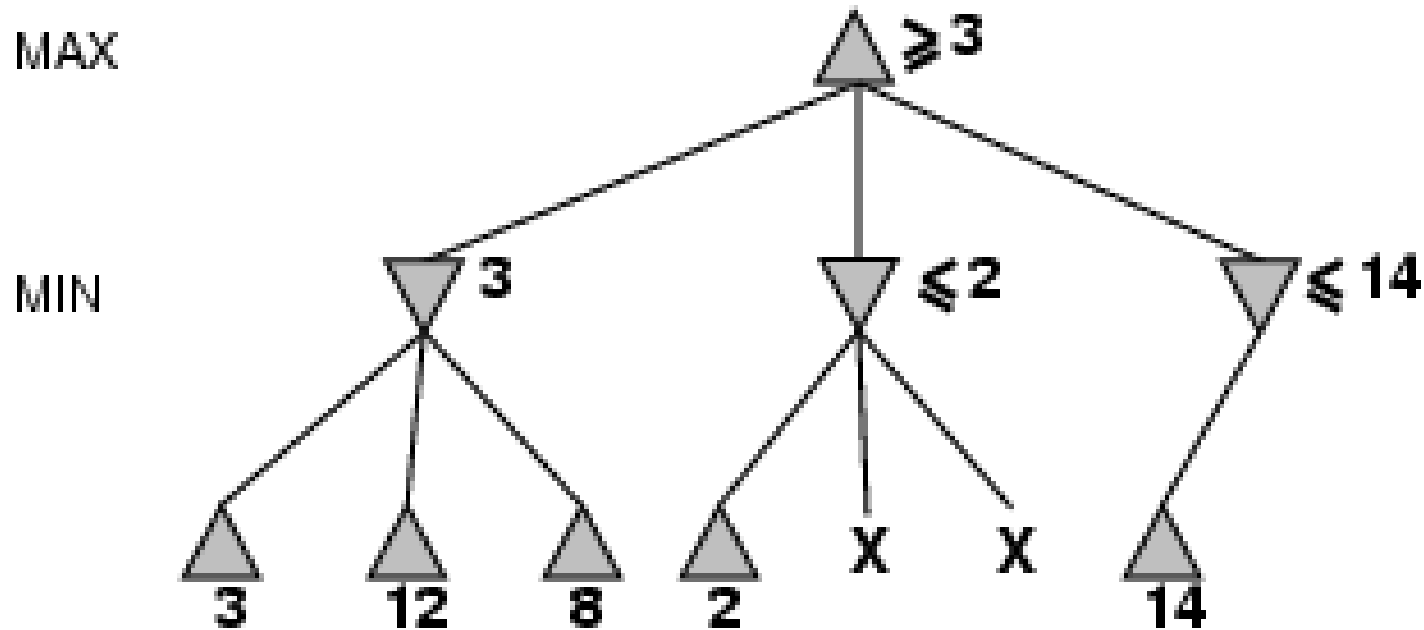
Alpha-Beta pruning: Example



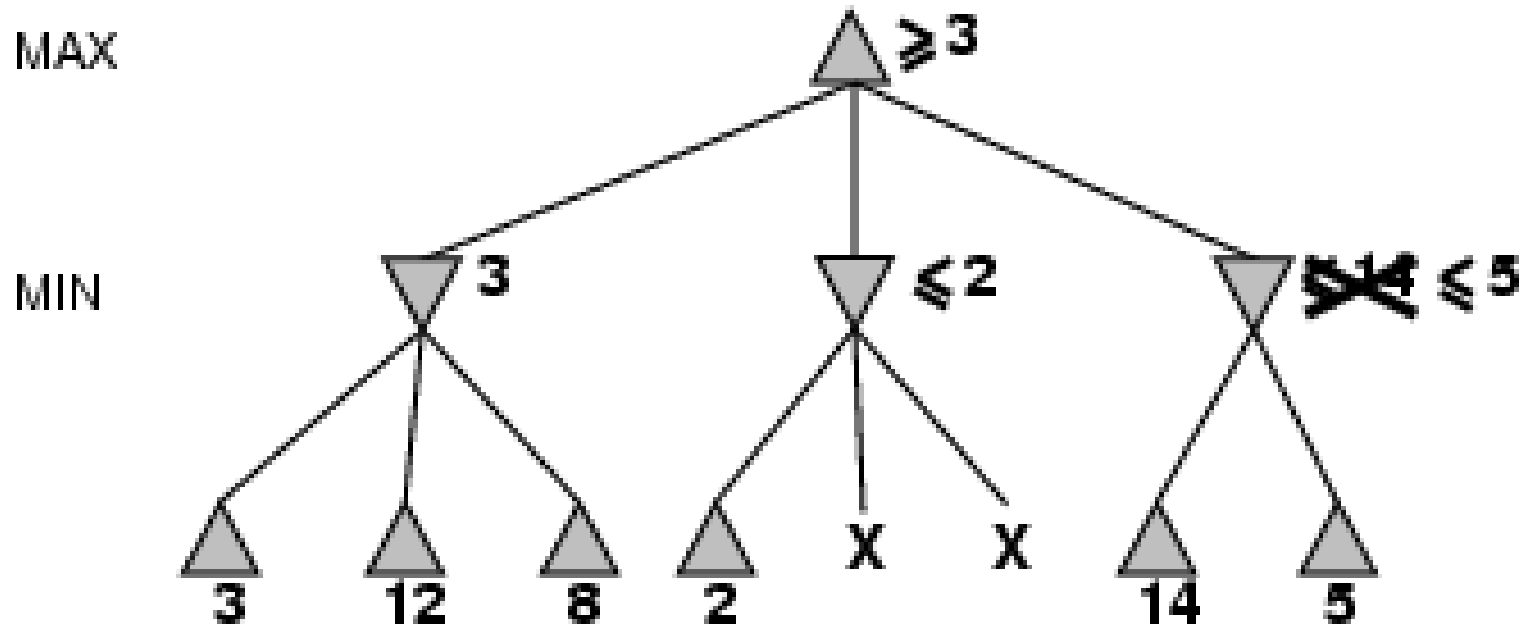
Alpha-Beta pruning: Example



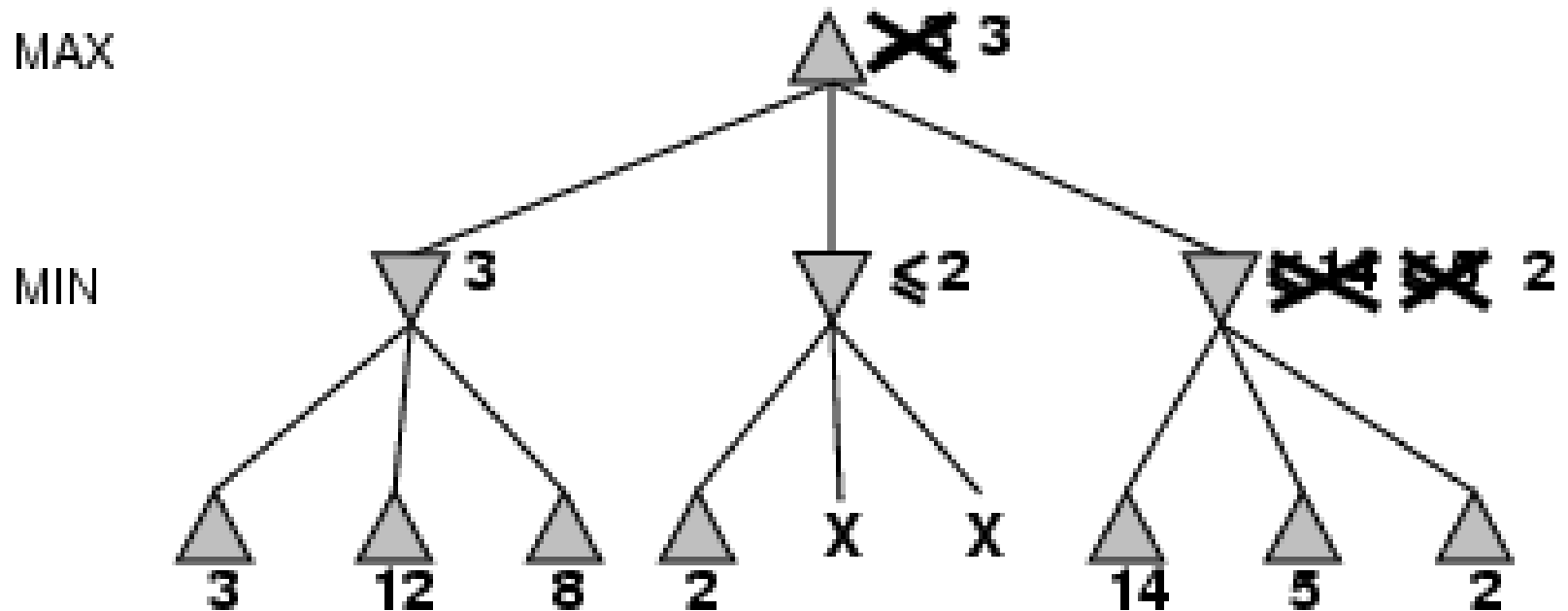
Alpha-Beta pruning: Example



Alpha-Beta pruning: Example



Alpha-Beta pruning: Example



Properties of α - β

- Pruning **does not** affect the final result
- Good move ordering improves the effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{d/2})$

Why is it called α - β ?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
- If v is worse than α , *max* will avoid it
 - prune that branch
- Define β similarly for *min*

MAX

MIN

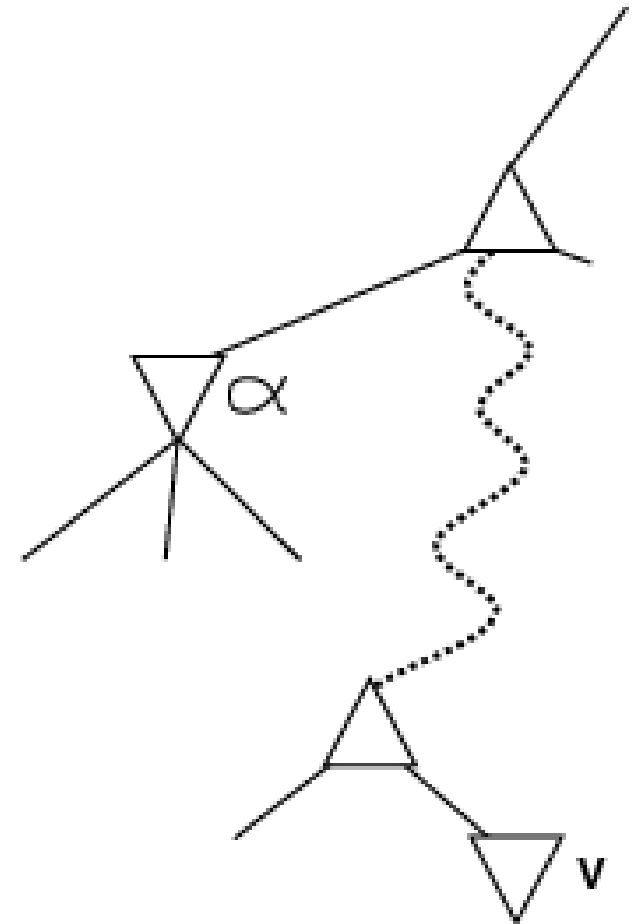
...

...

...

MAX

MIN



The α - β algorithm

```
function ALPHA-BETA-SEARCH(state) returns an action  
  inputs: state, current state in game  
   $v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$   
  return the action in SUCCESSORS(state) with value  $v$ 
```

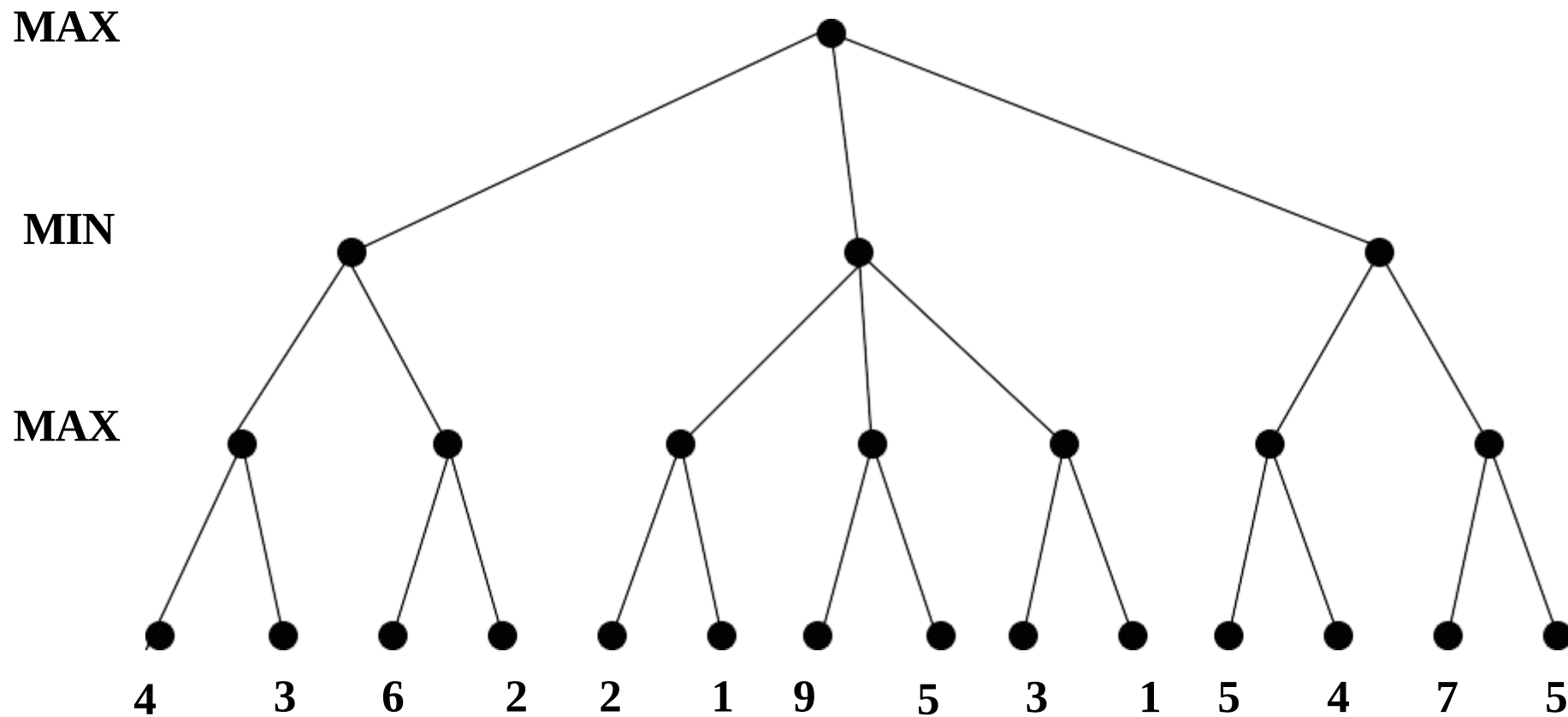
```
function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value  
  inputs: state, current state in game  
            $\alpha$ , the value of the best alternative for MAX along the path to state  
            $\beta$ , the value of the best alternative for MIN along the path to state  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for  $a, s$  in SUCCESSORS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$   
    if  $v \geq \beta$  then return  $v$   
     $\alpha \leftarrow \text{MAX}(\alpha, v)$   
  return  $v$ 
```

The α - β algorithm

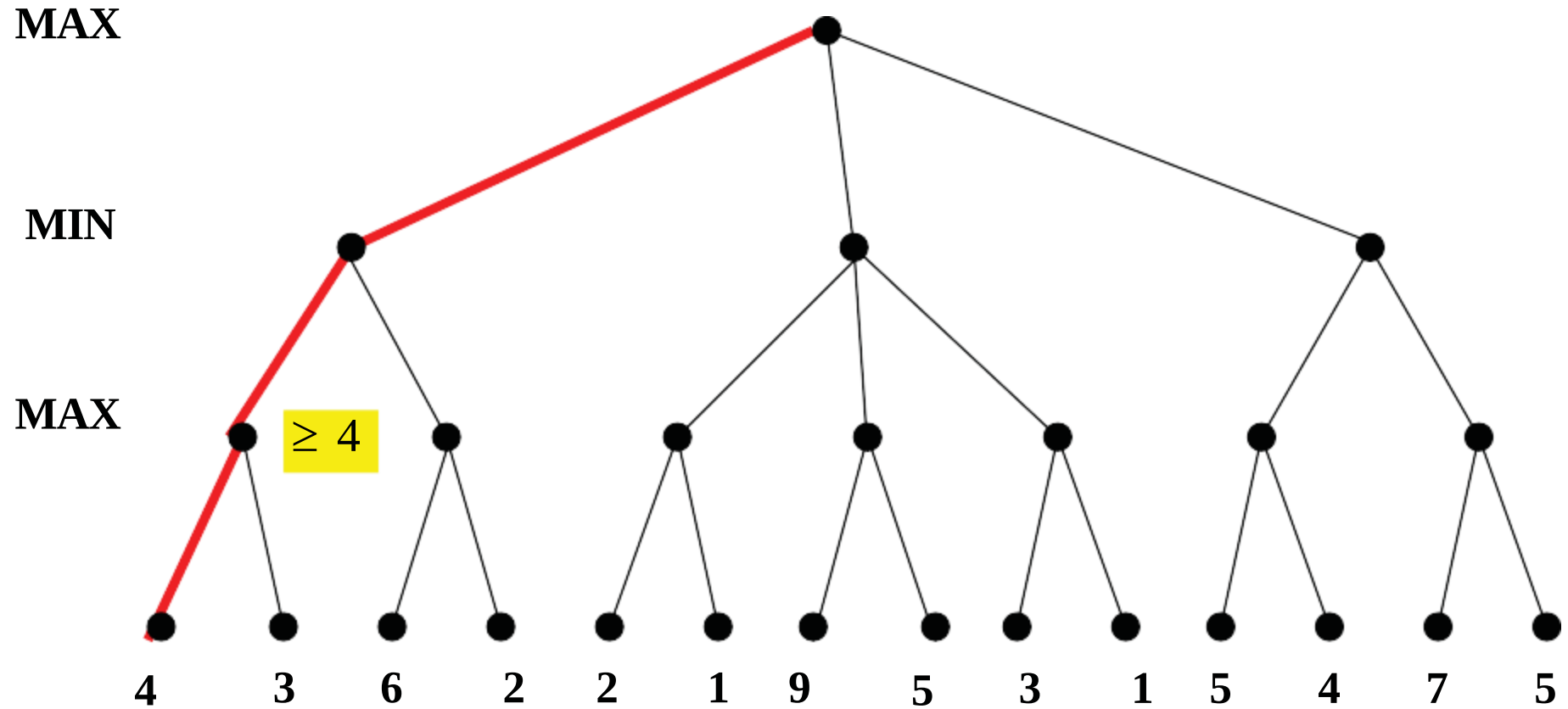
```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 
```

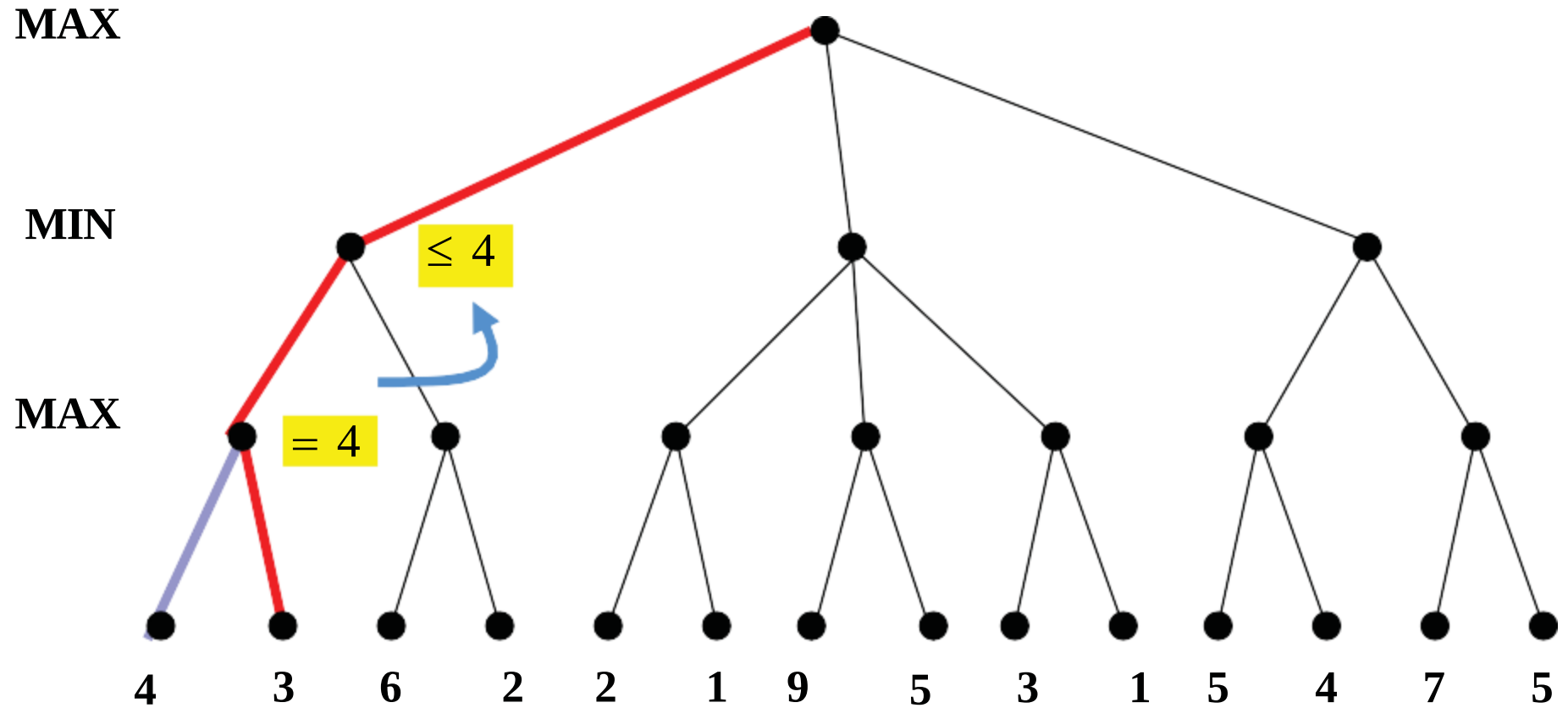
Alpha beta pruning. Example



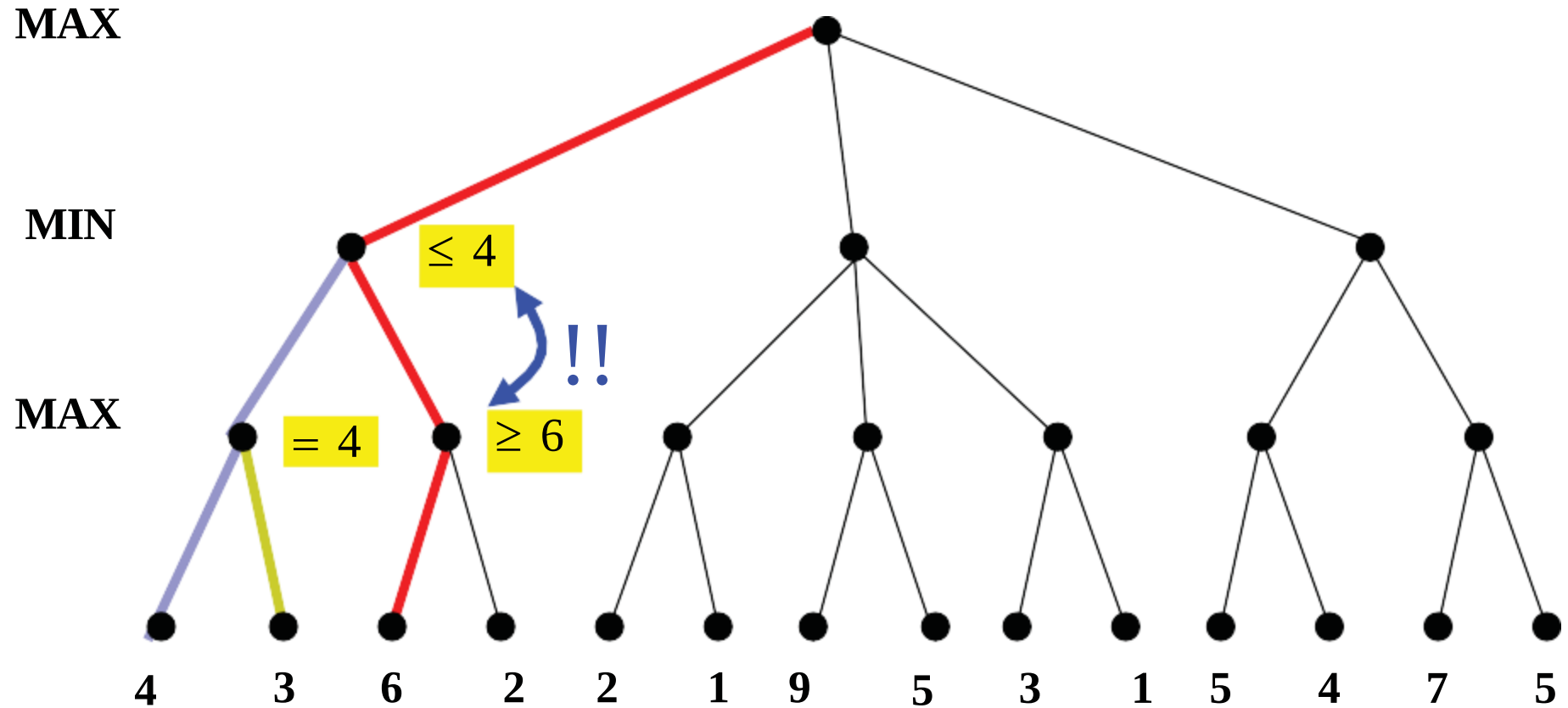
Alpha beta pruning. Example



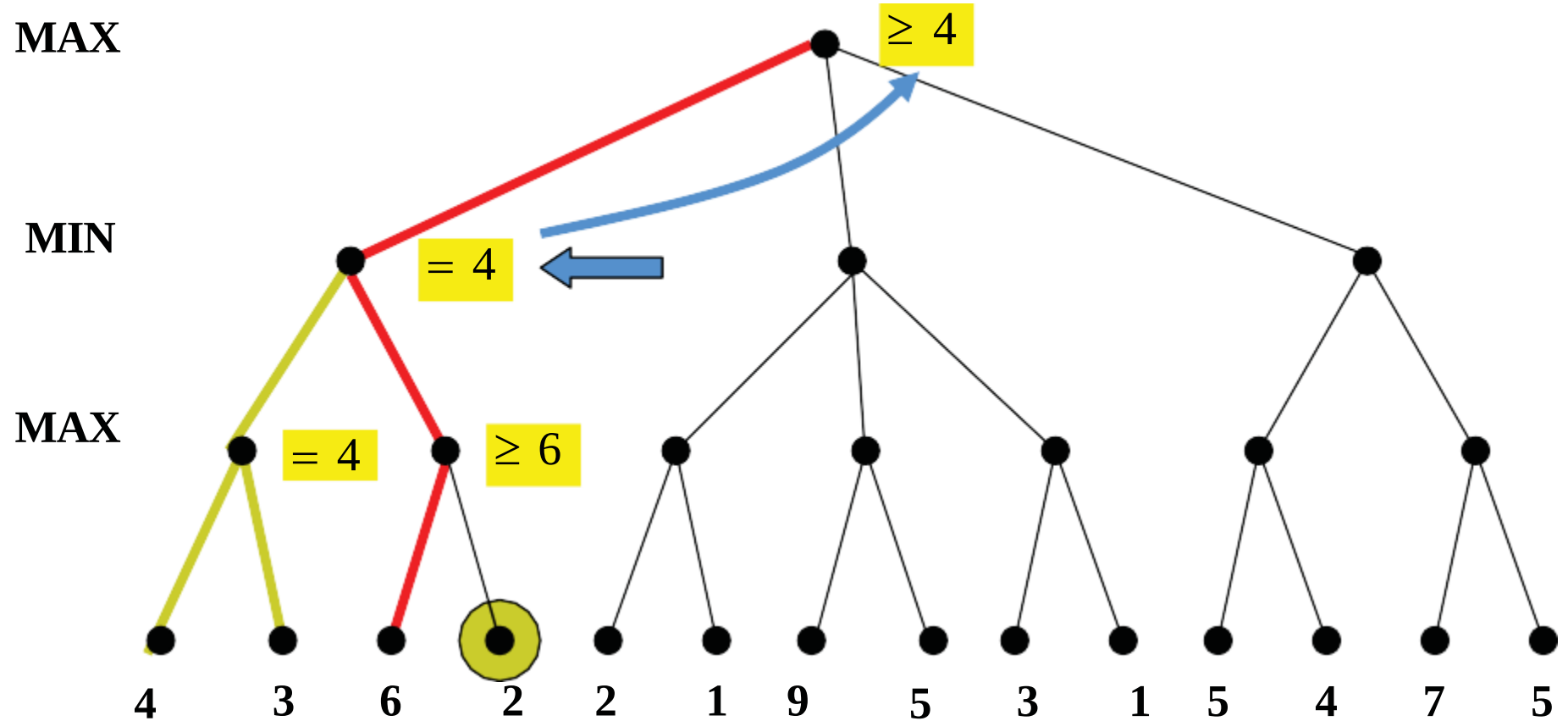
Alpha beta pruning. Example



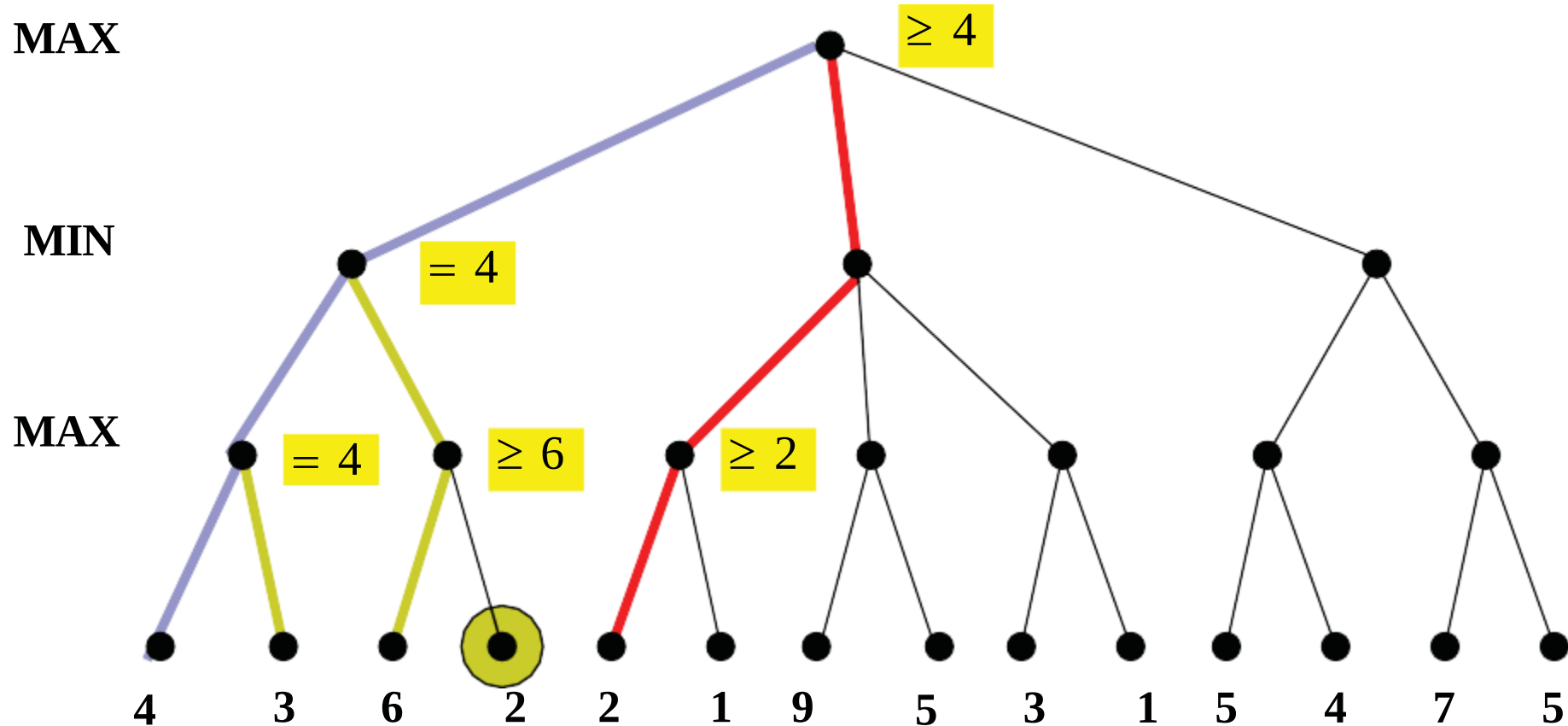
Alpha beta pruning. Example



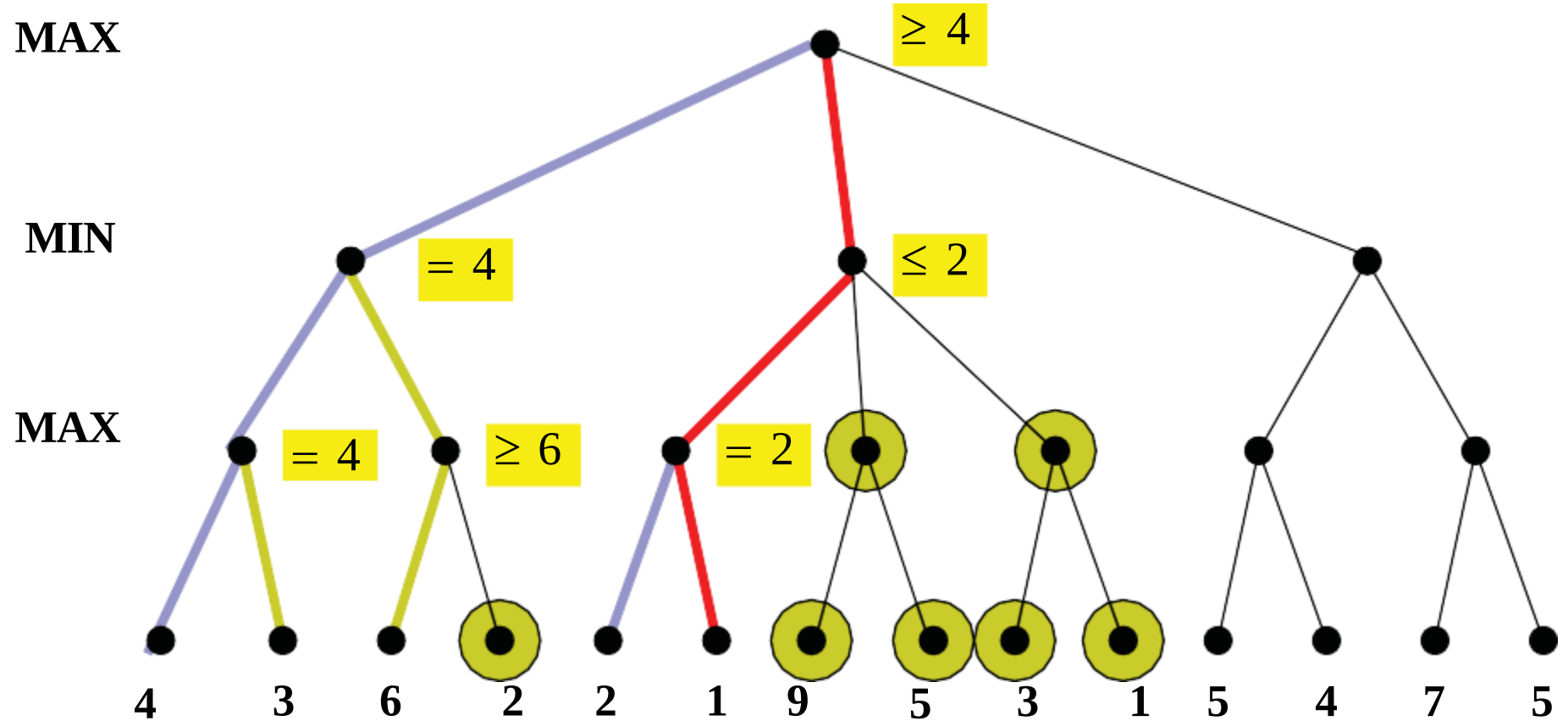
Alpha beta pruning. Example



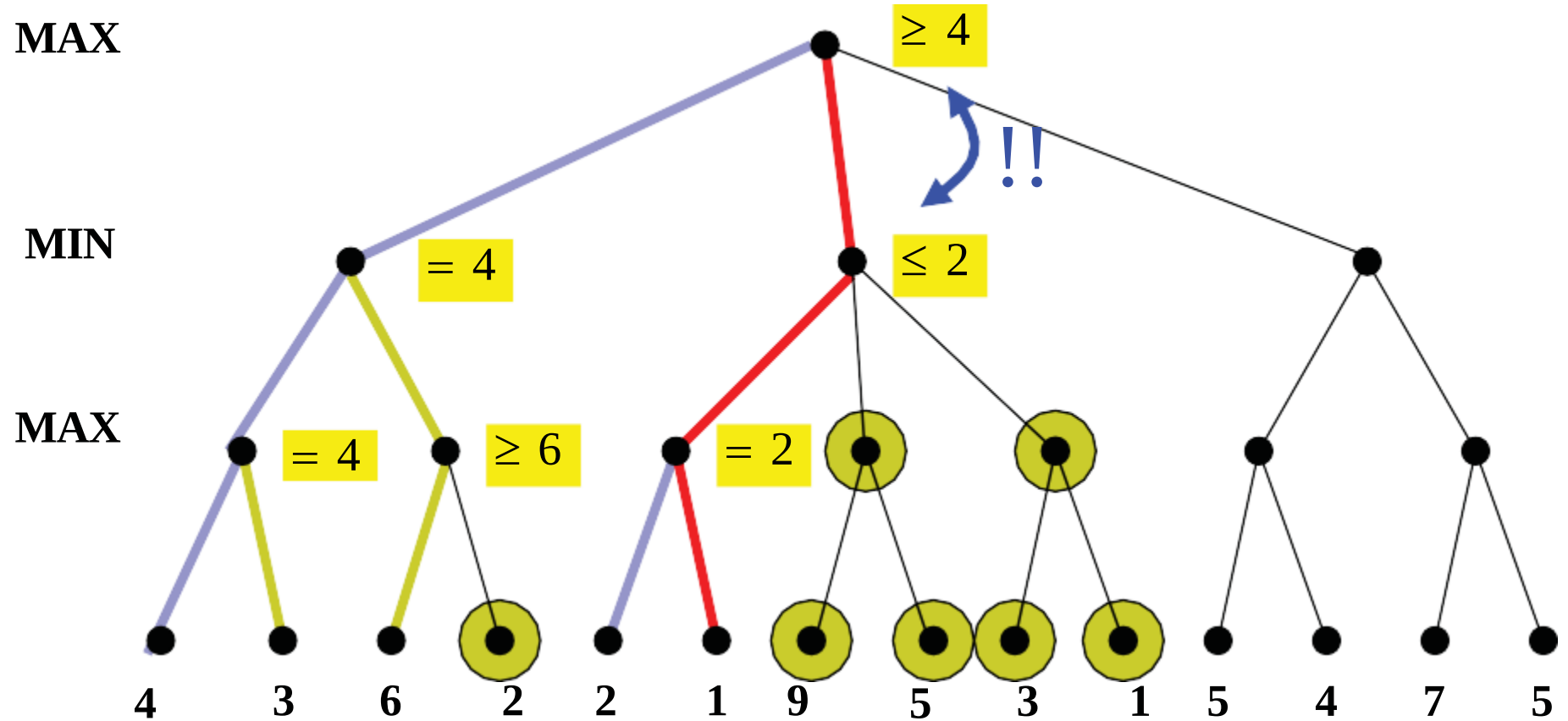
Alpha beta pruning. Example



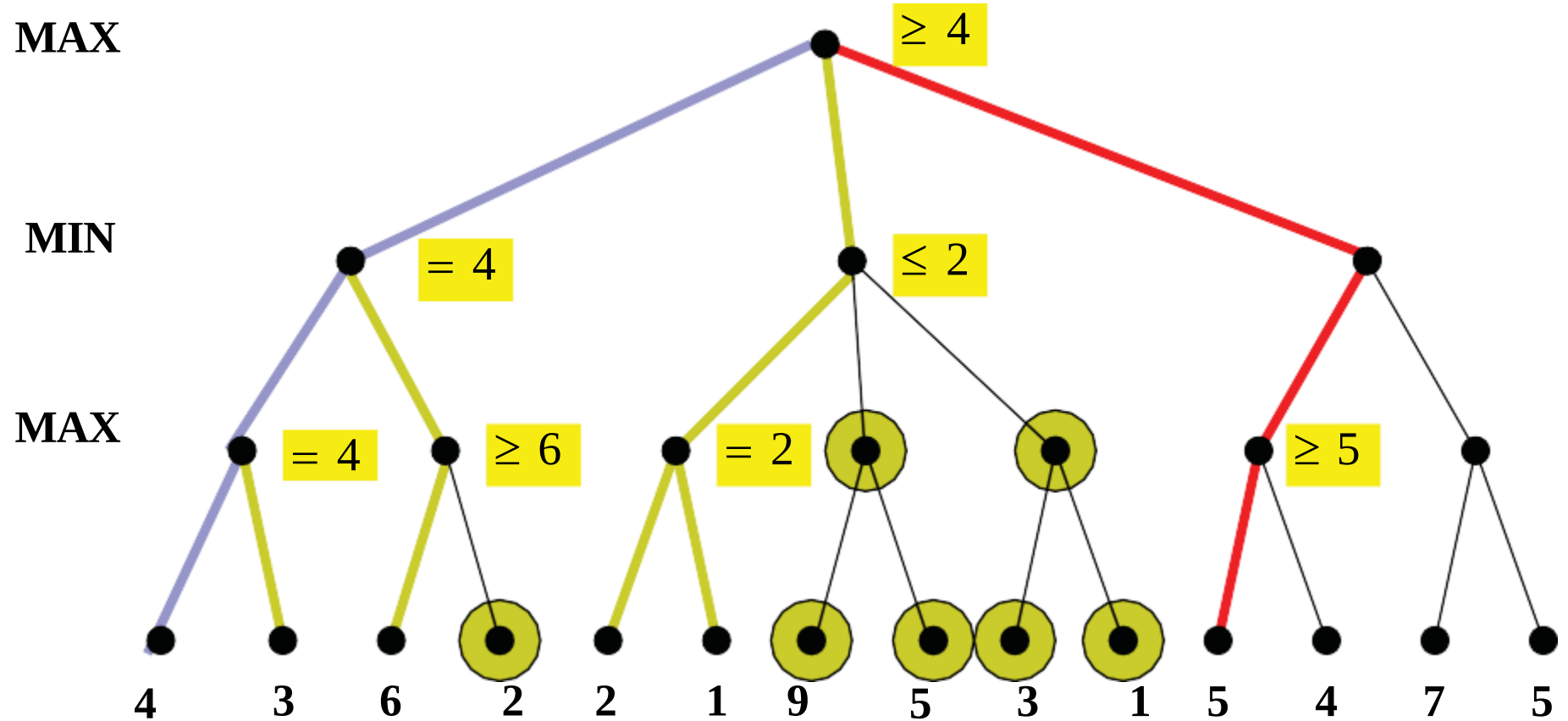
Alpha beta pruning. Example



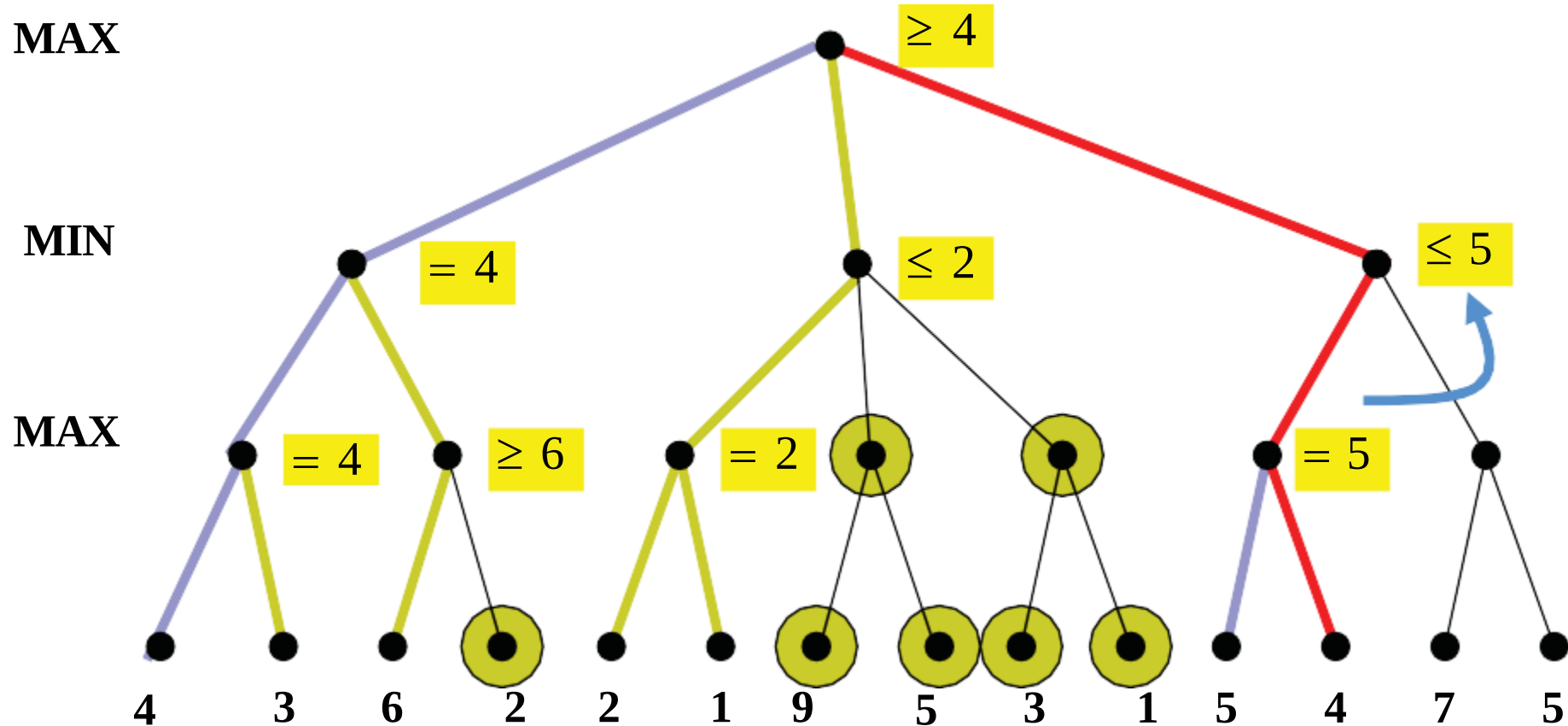
Alpha beta pruning. Example



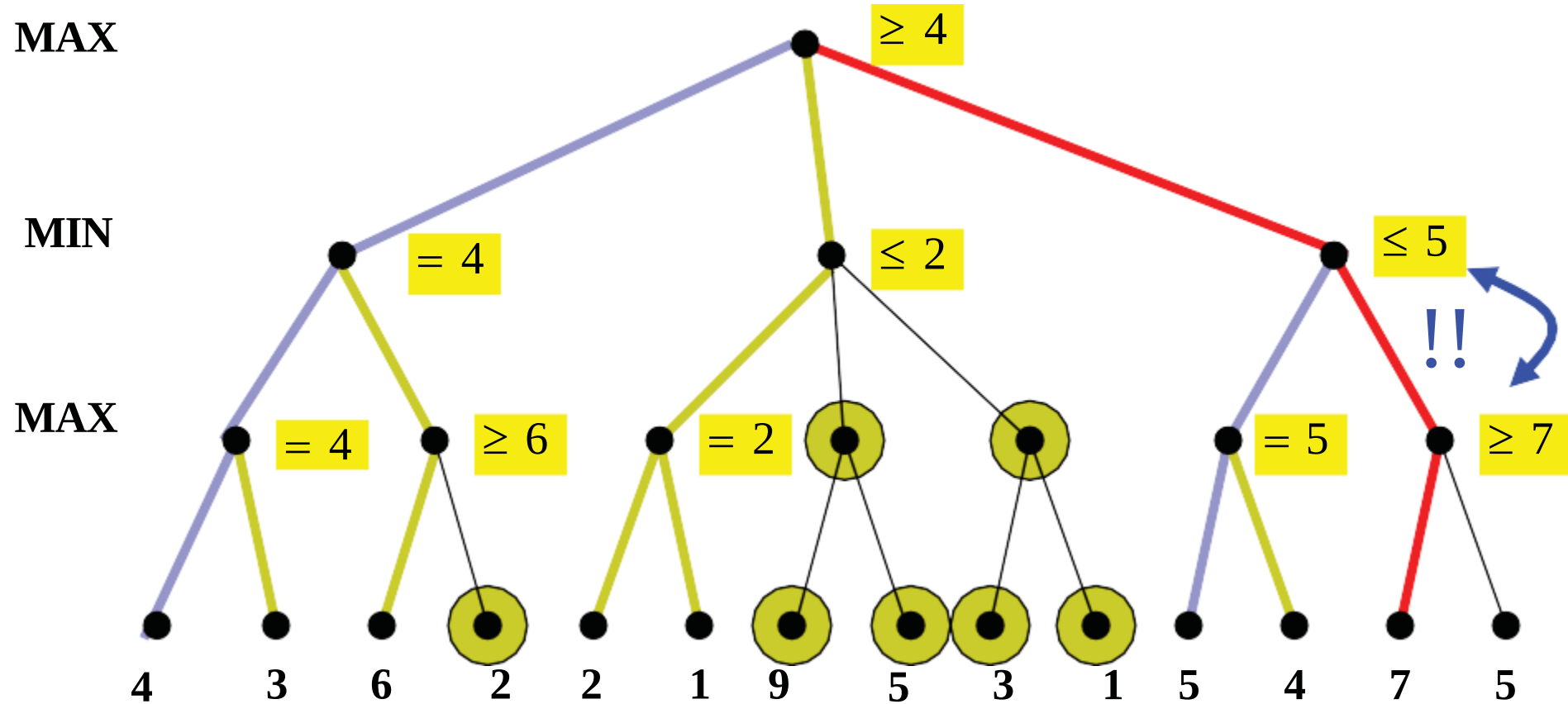
Alpha beta pruning. Example



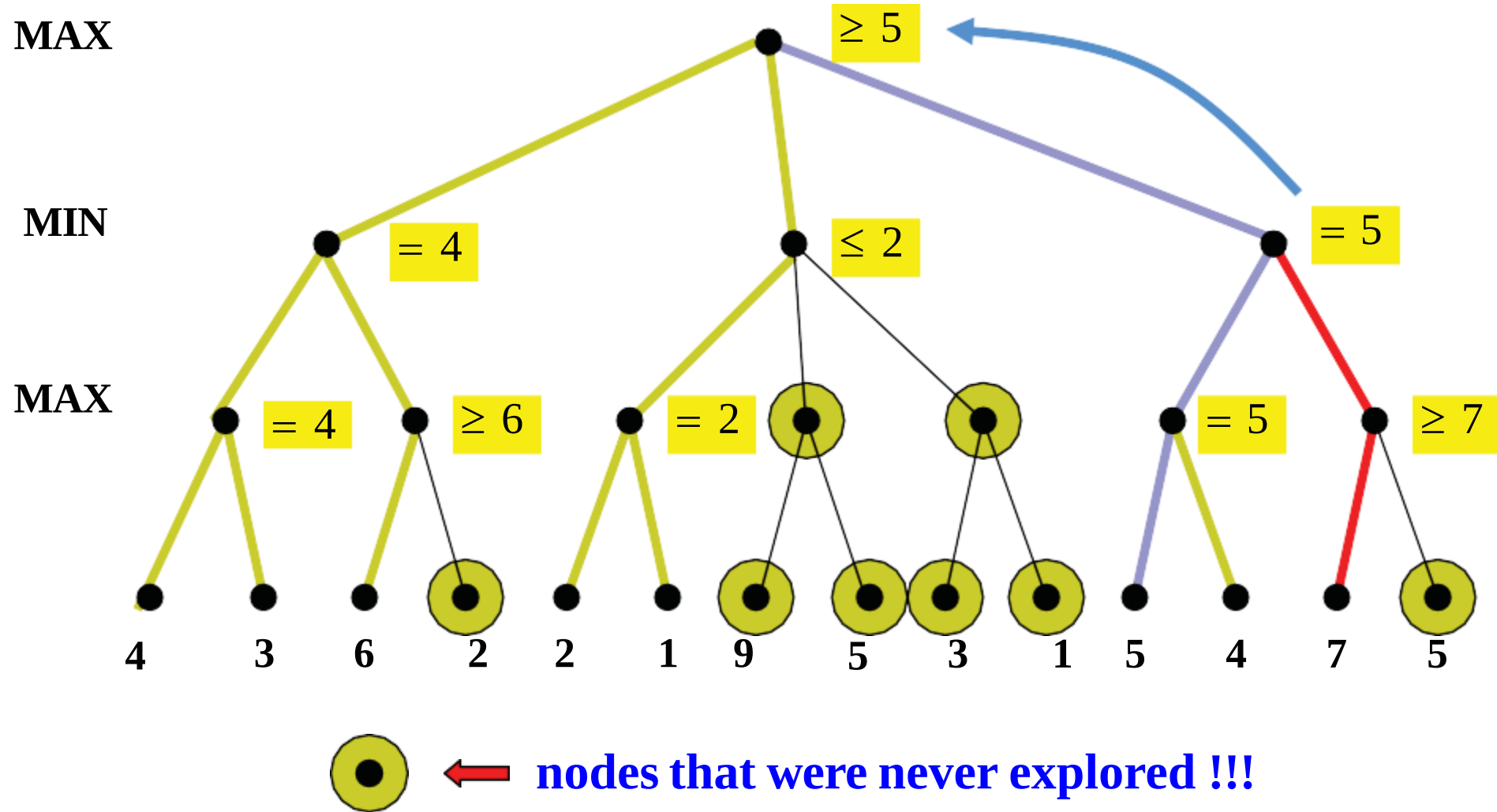
Alpha beta pruning. Example



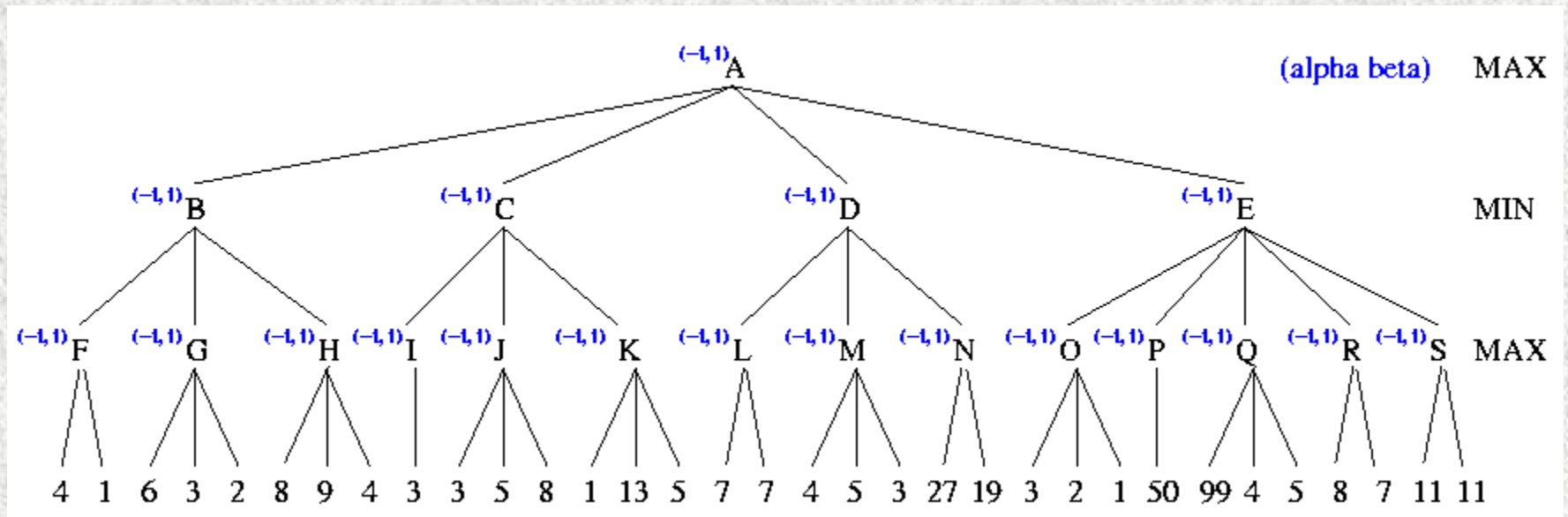
Alpha beta pruning. Example



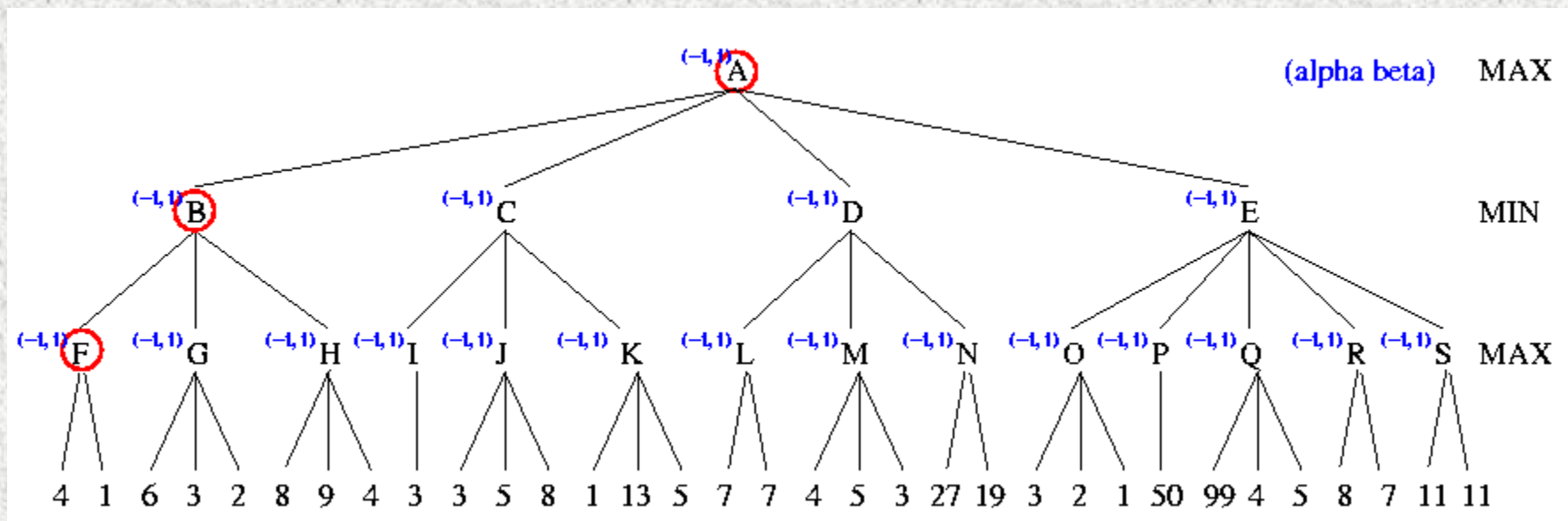
Alpha beta pruning. Example



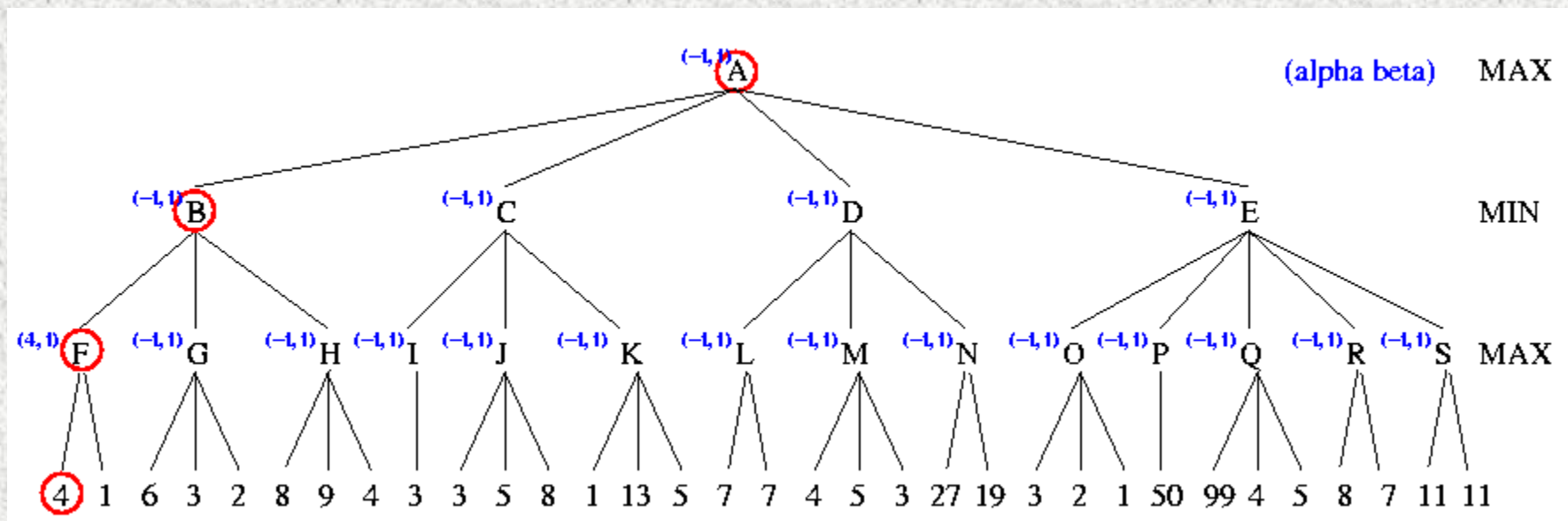
Example



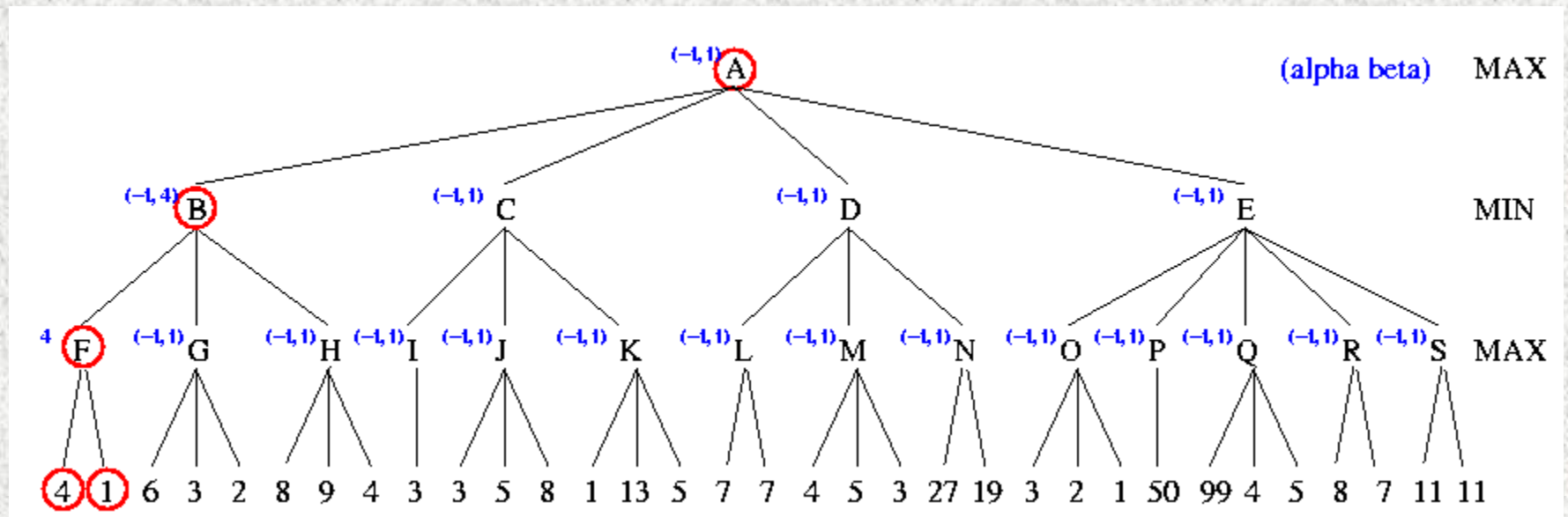
Example



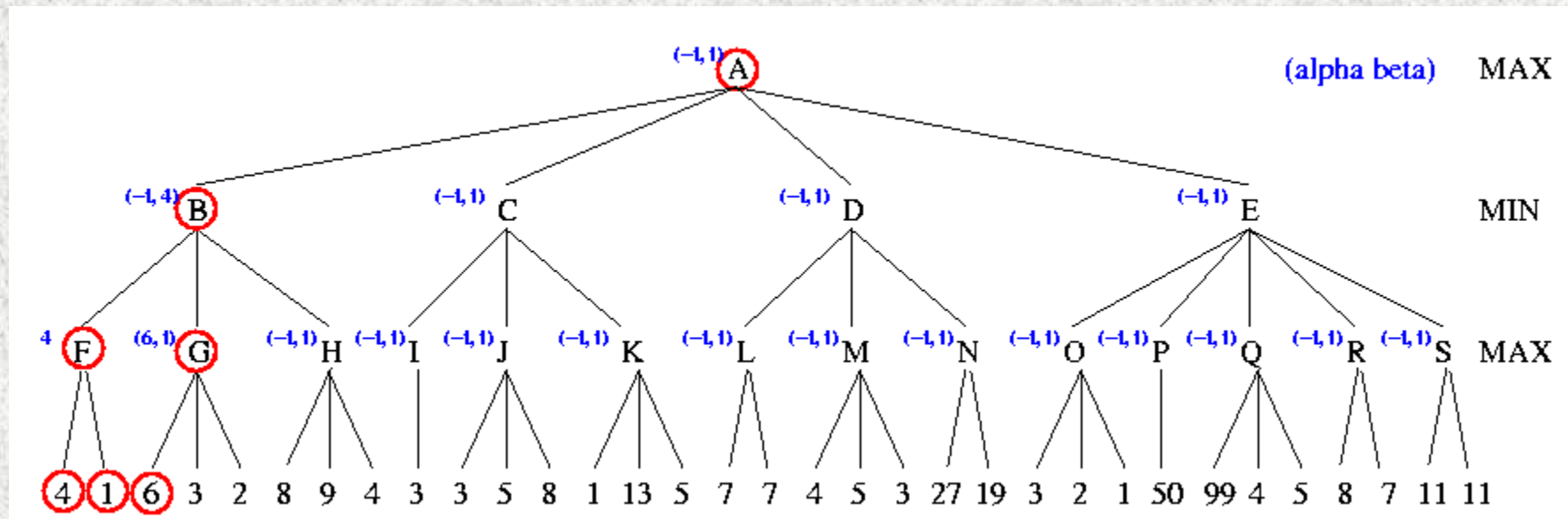
Example



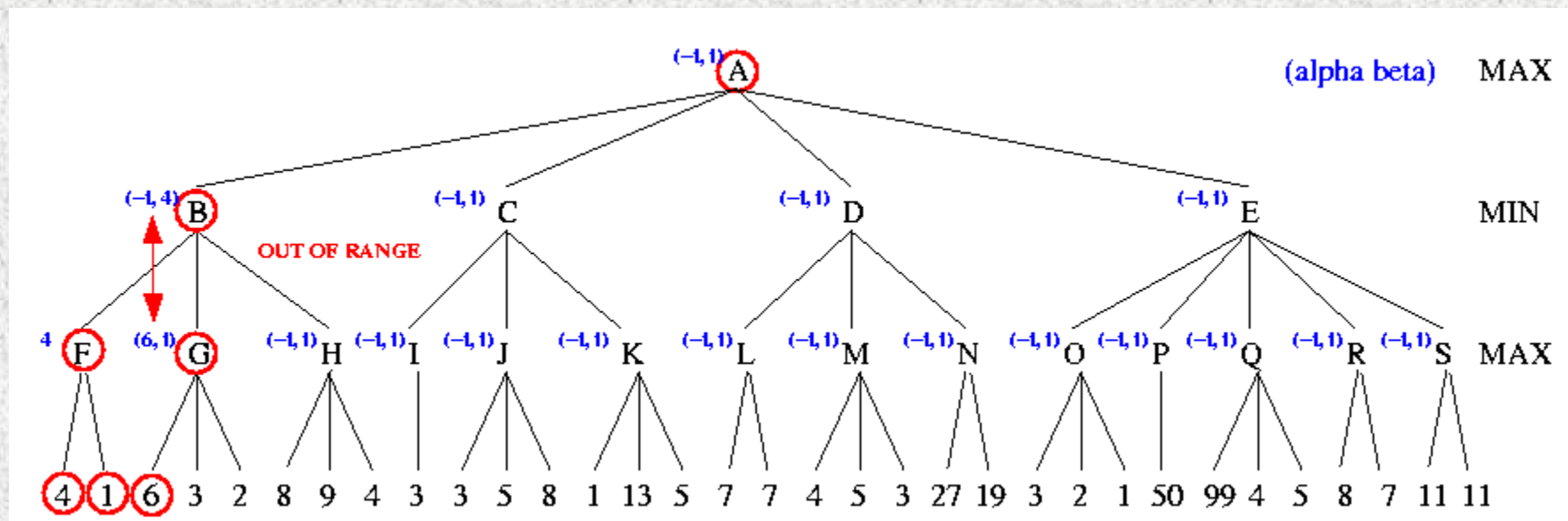
Example



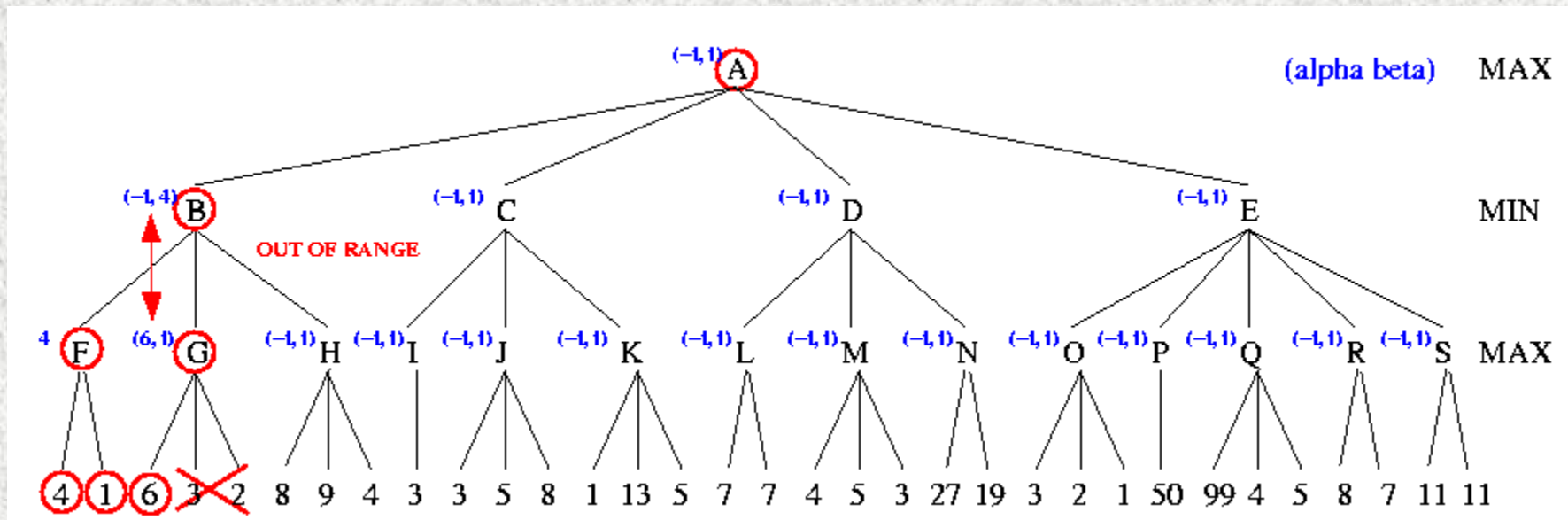
Example



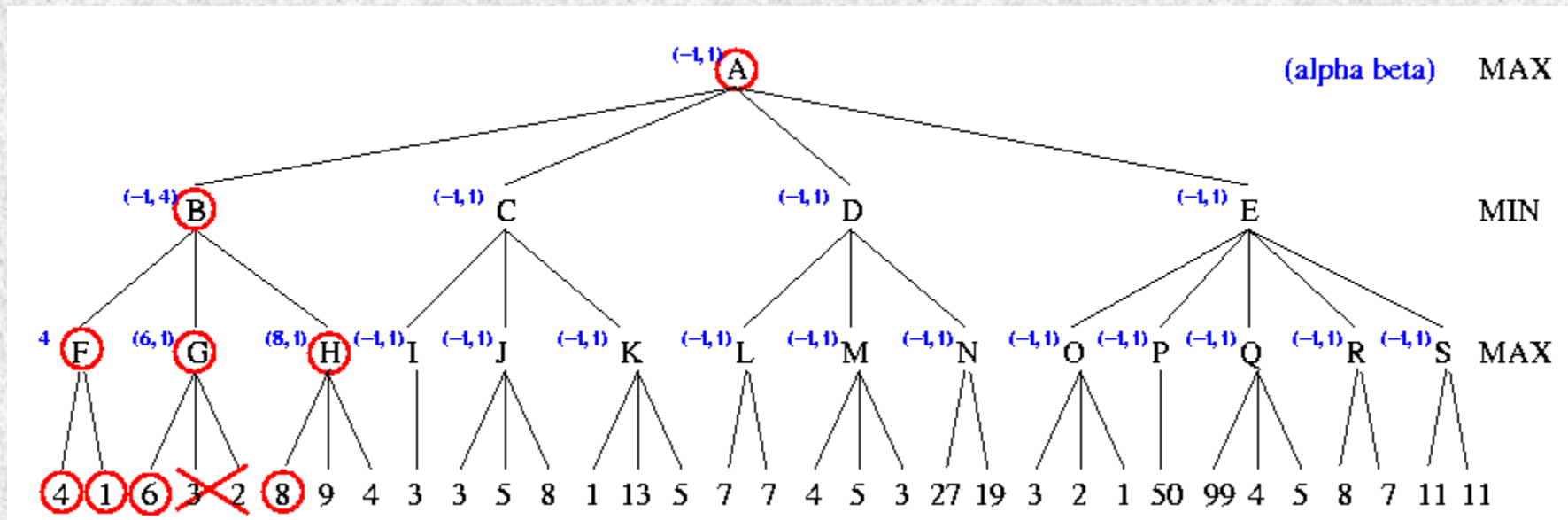
Example



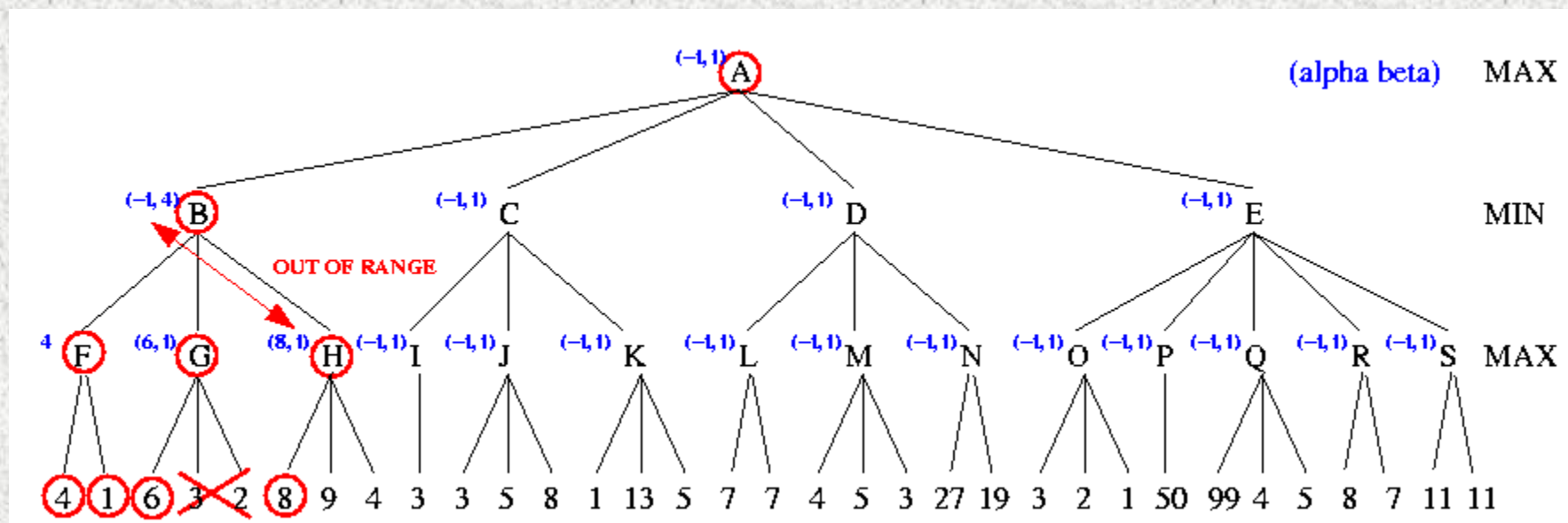
Example



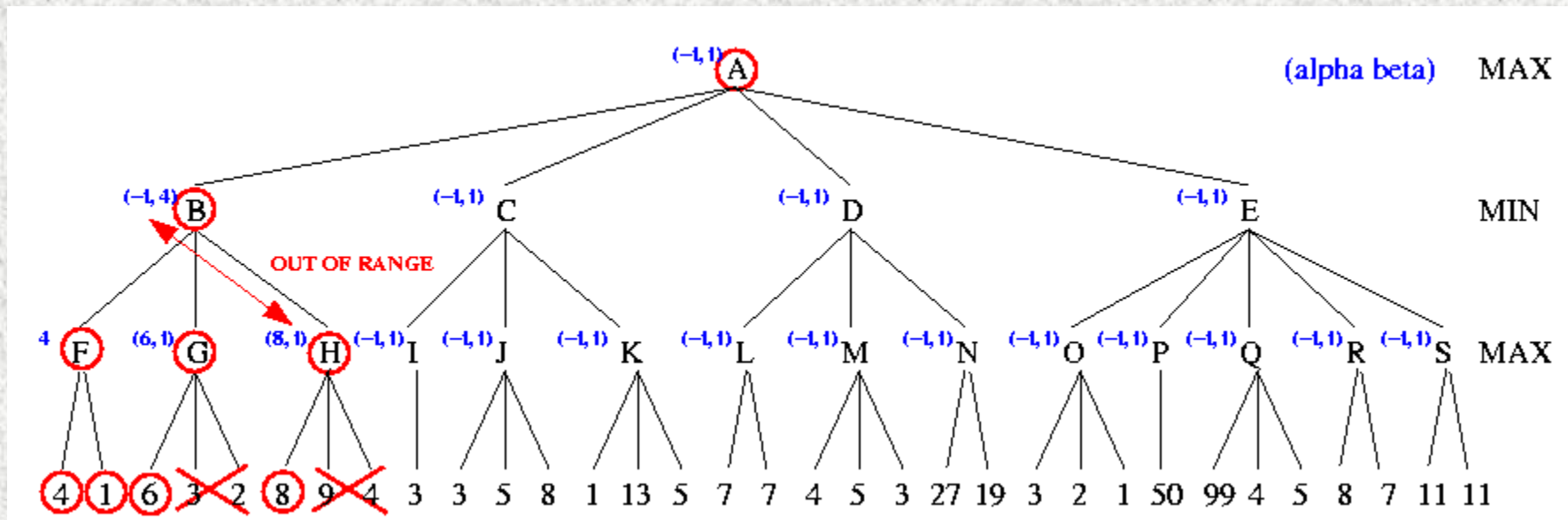
Example



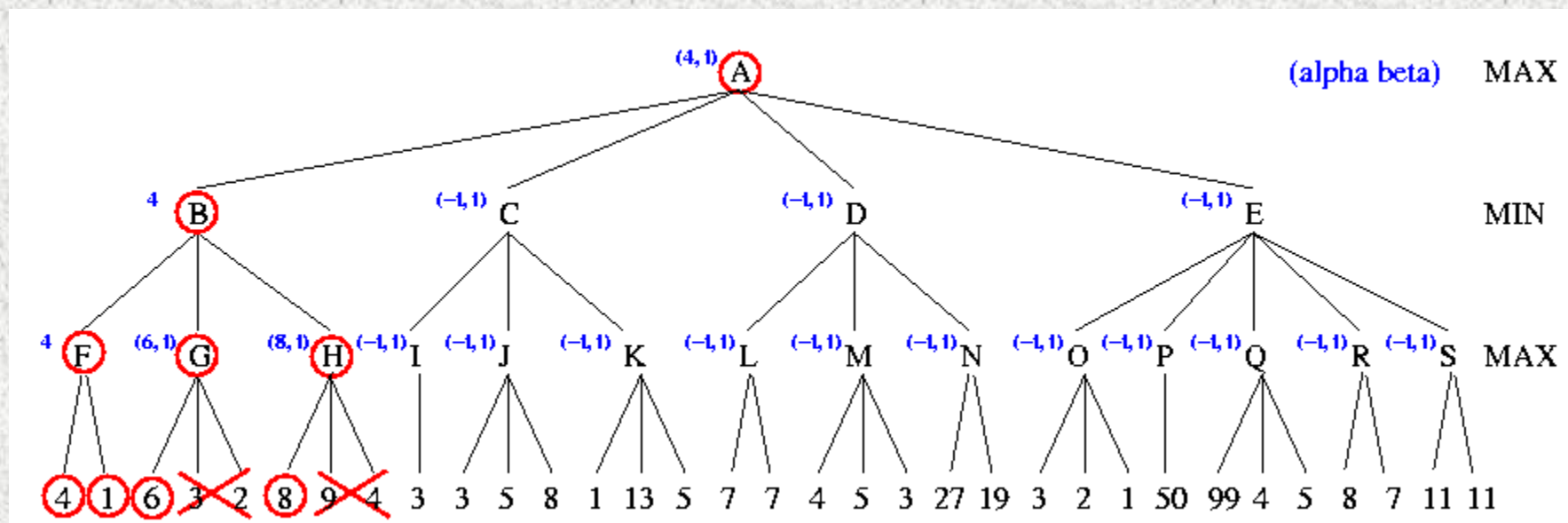
Example



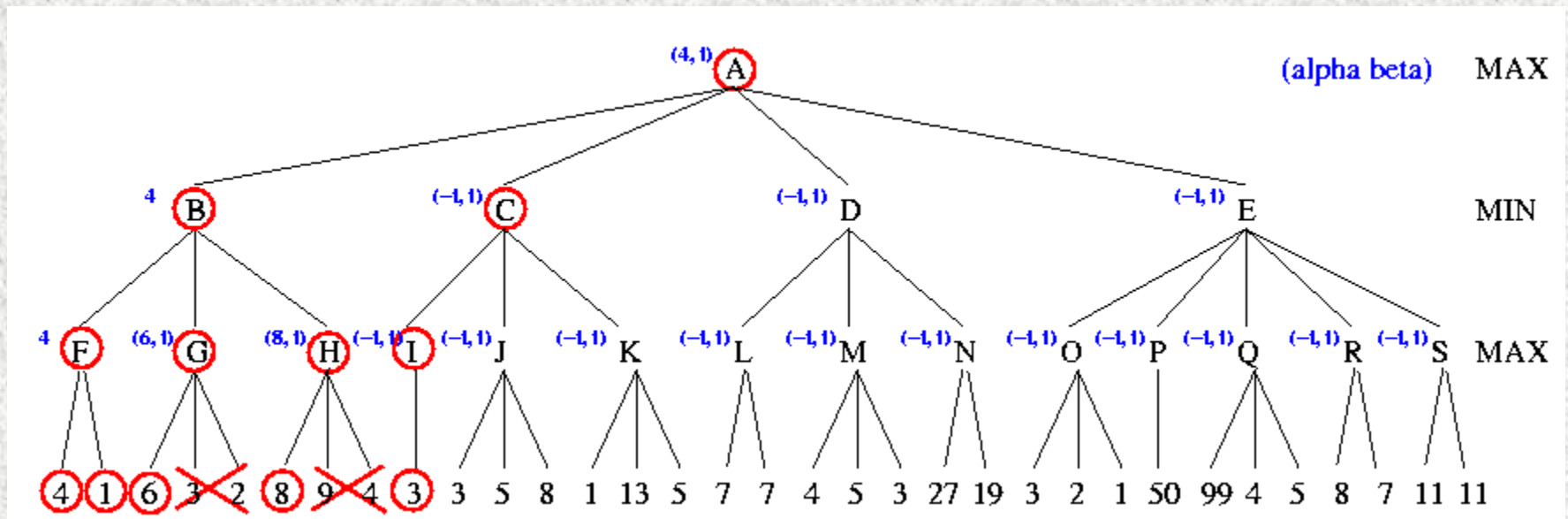
Example



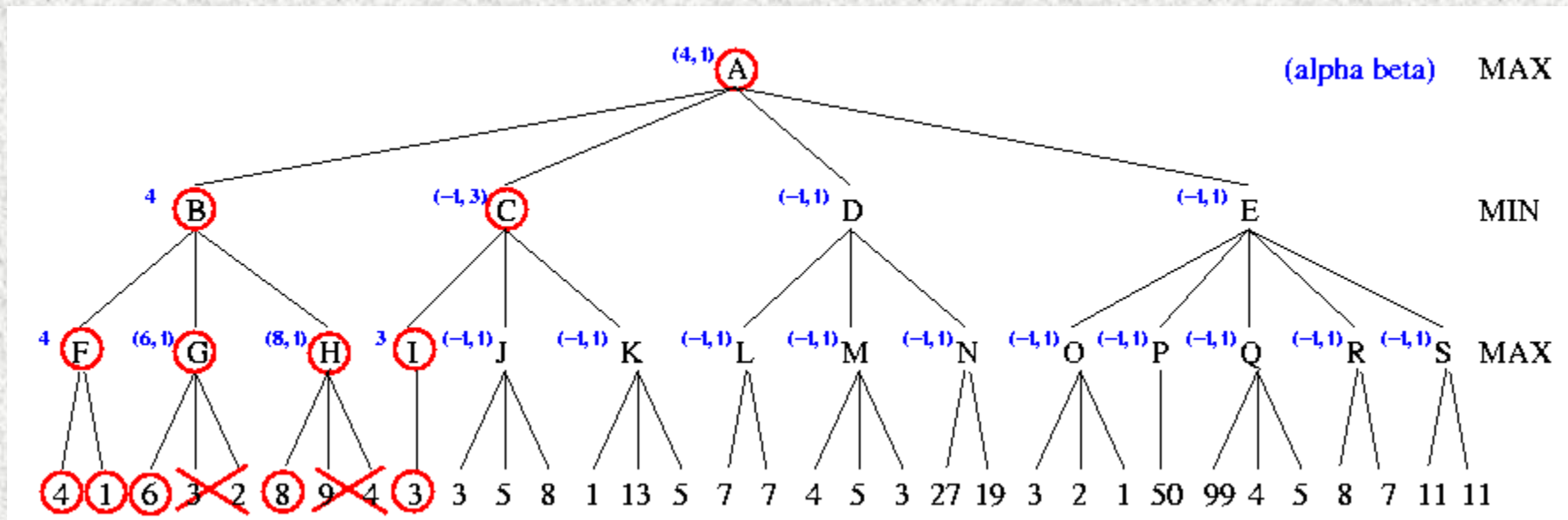
Example



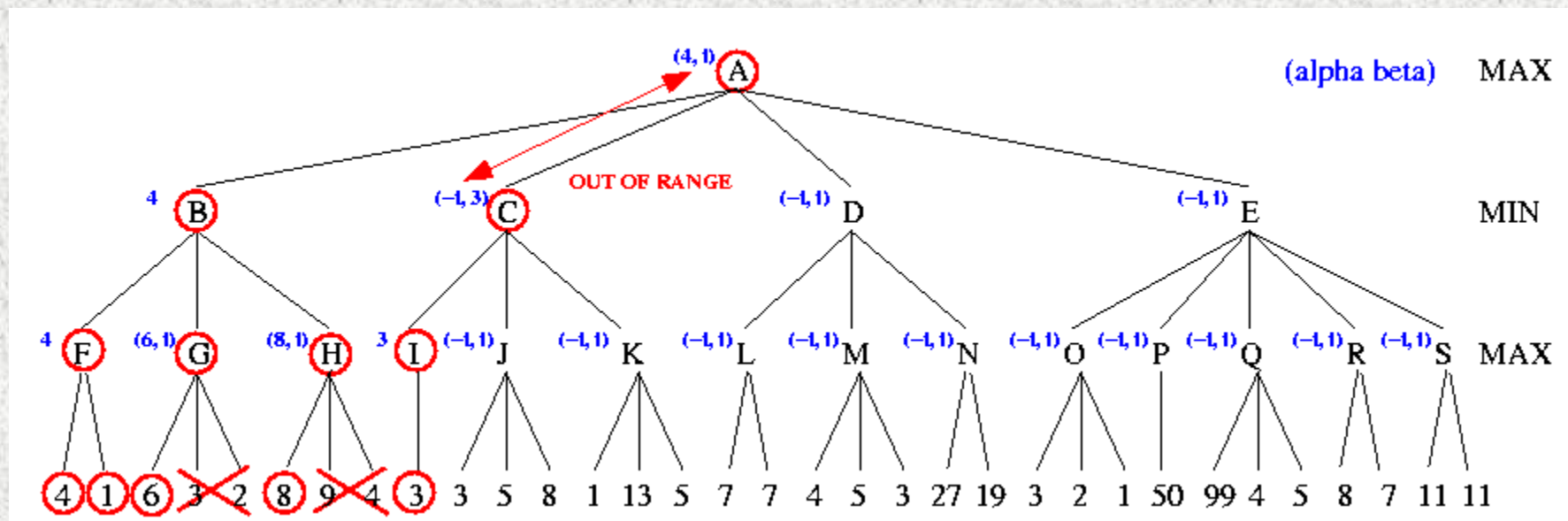
Example



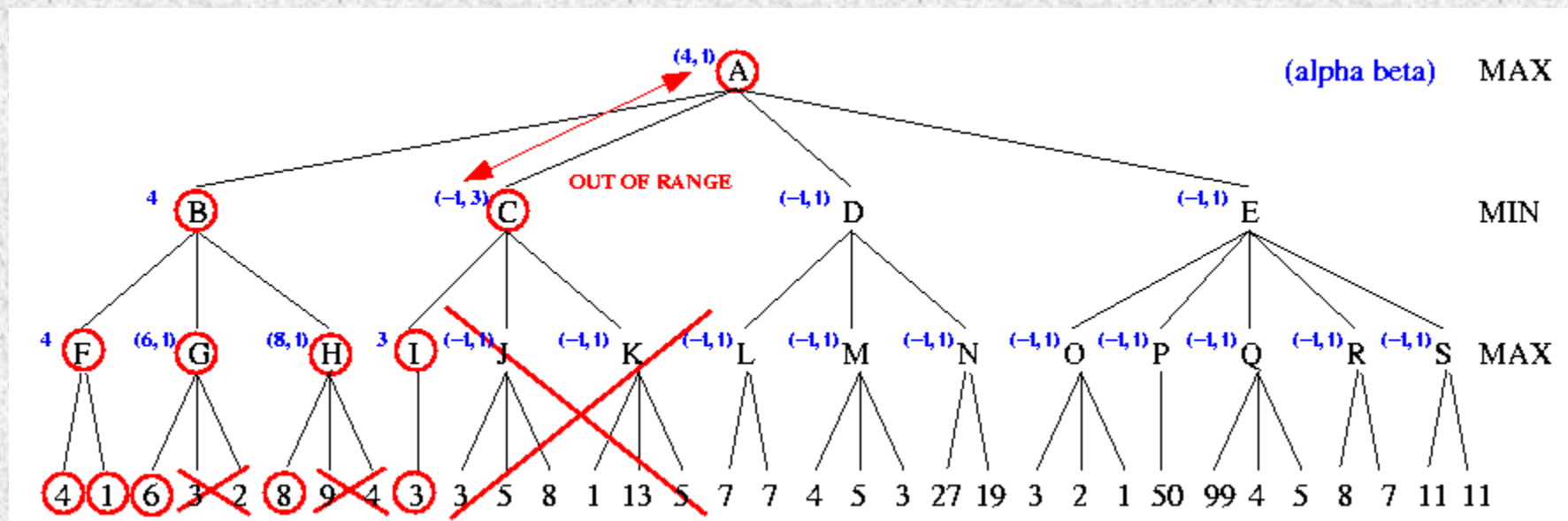
Example



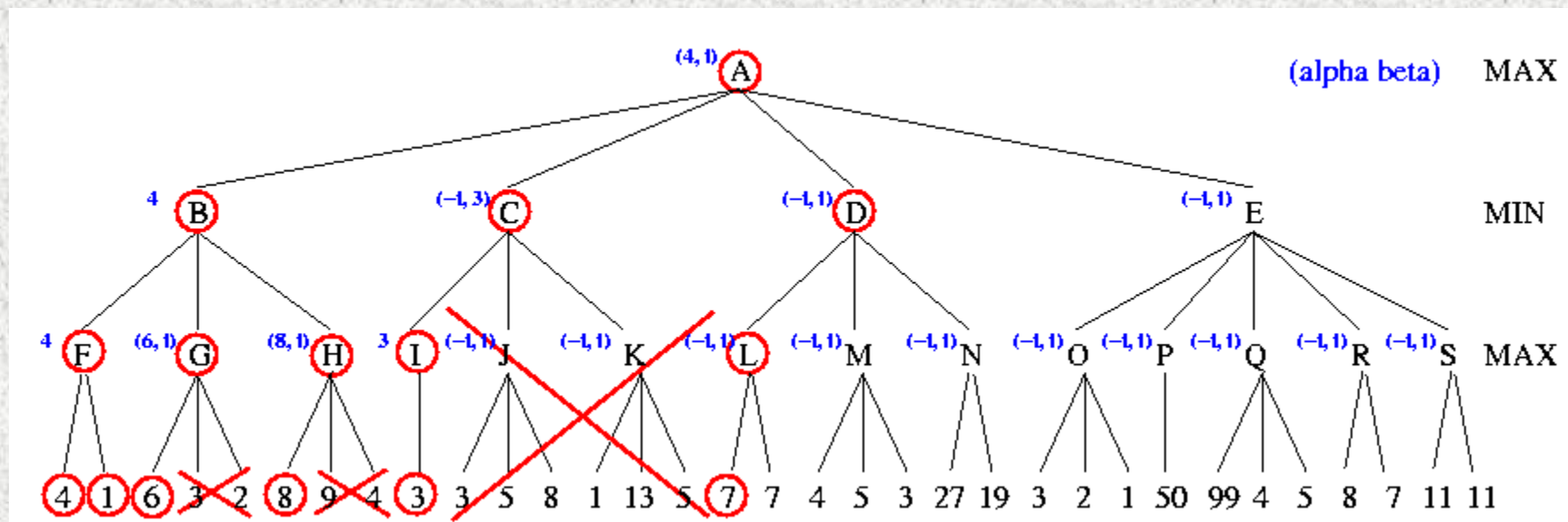
Example



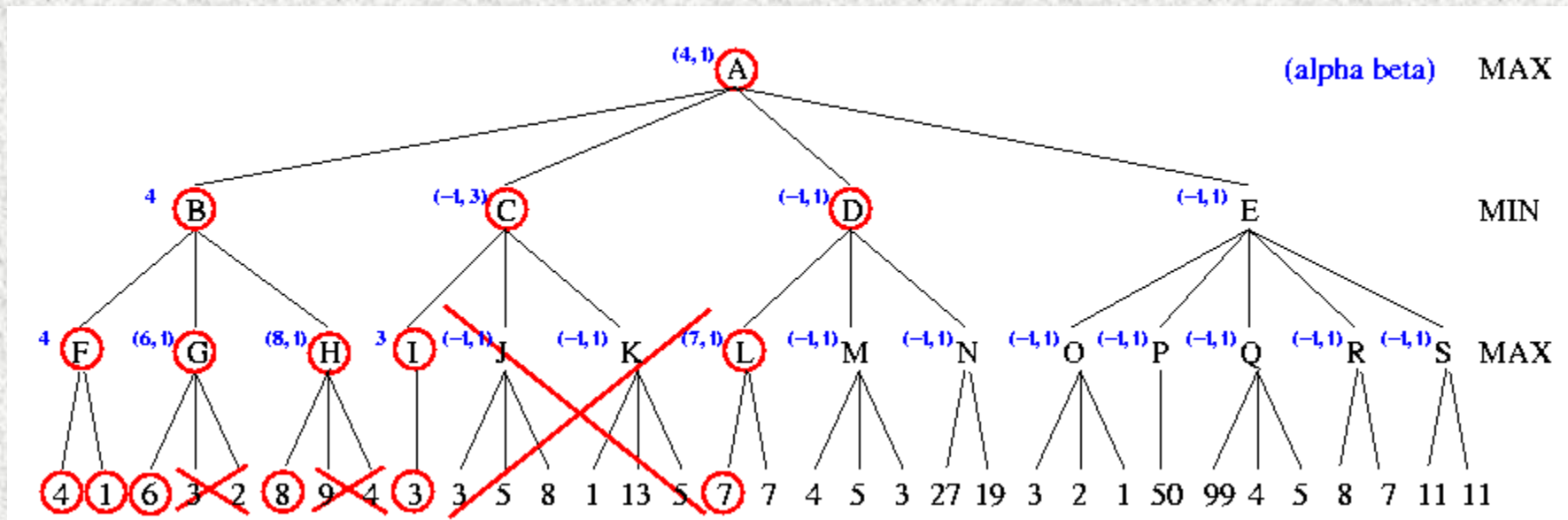
Example



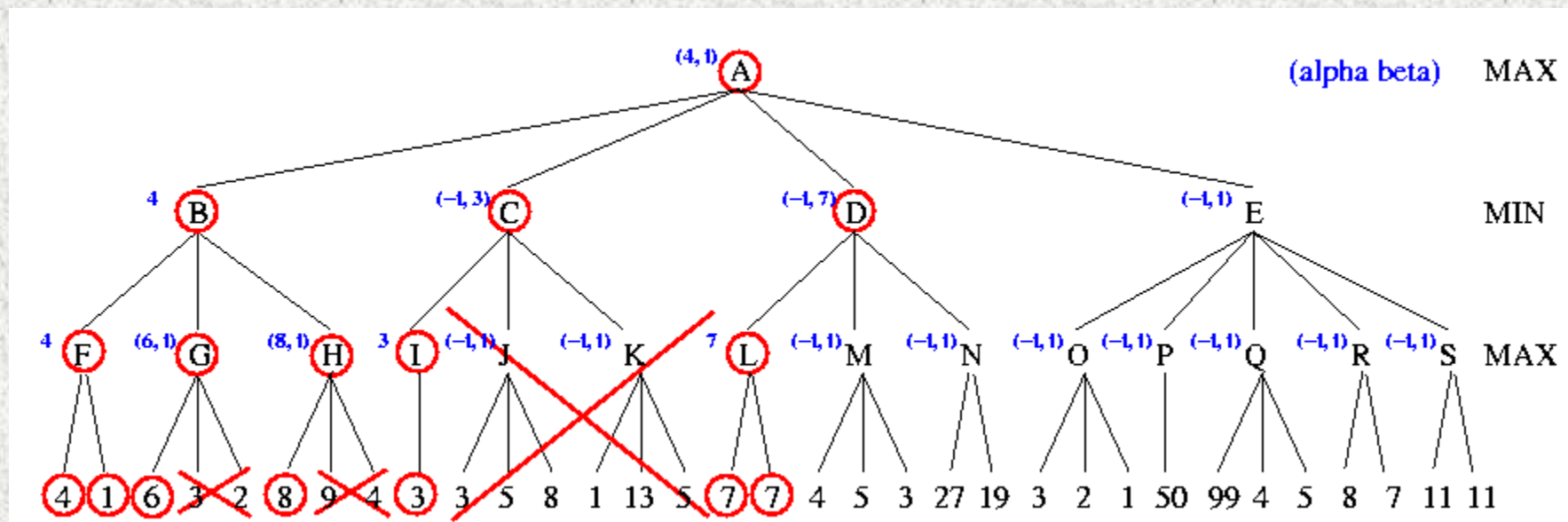
Example



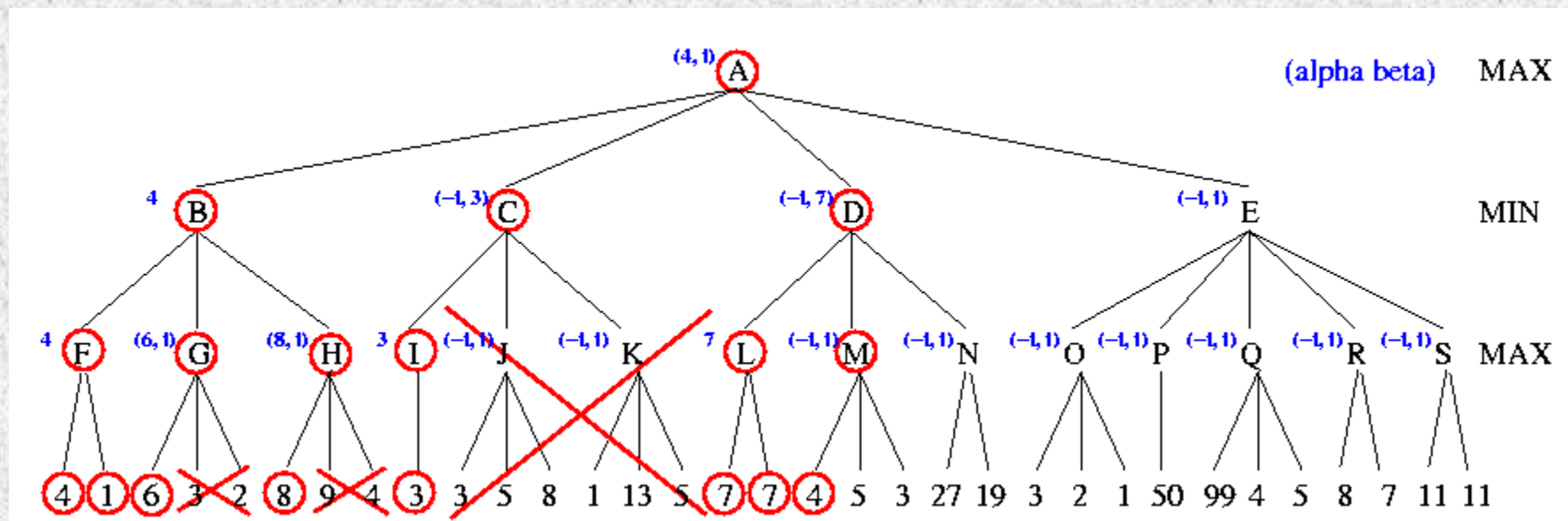
Example



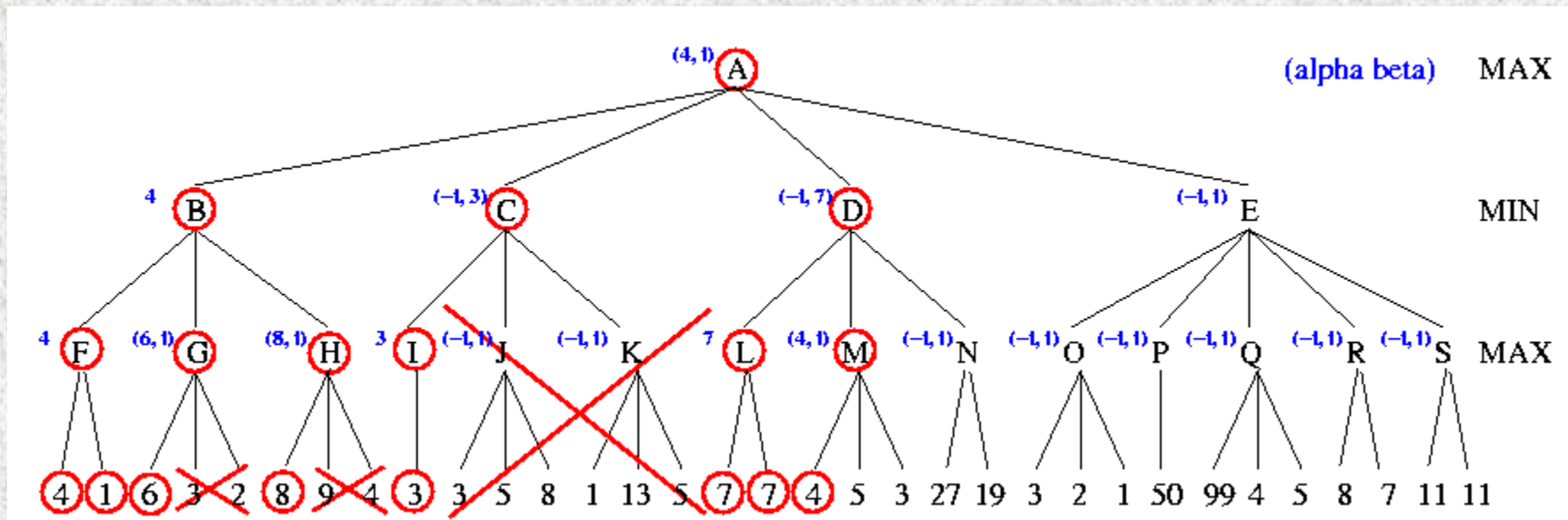
Example



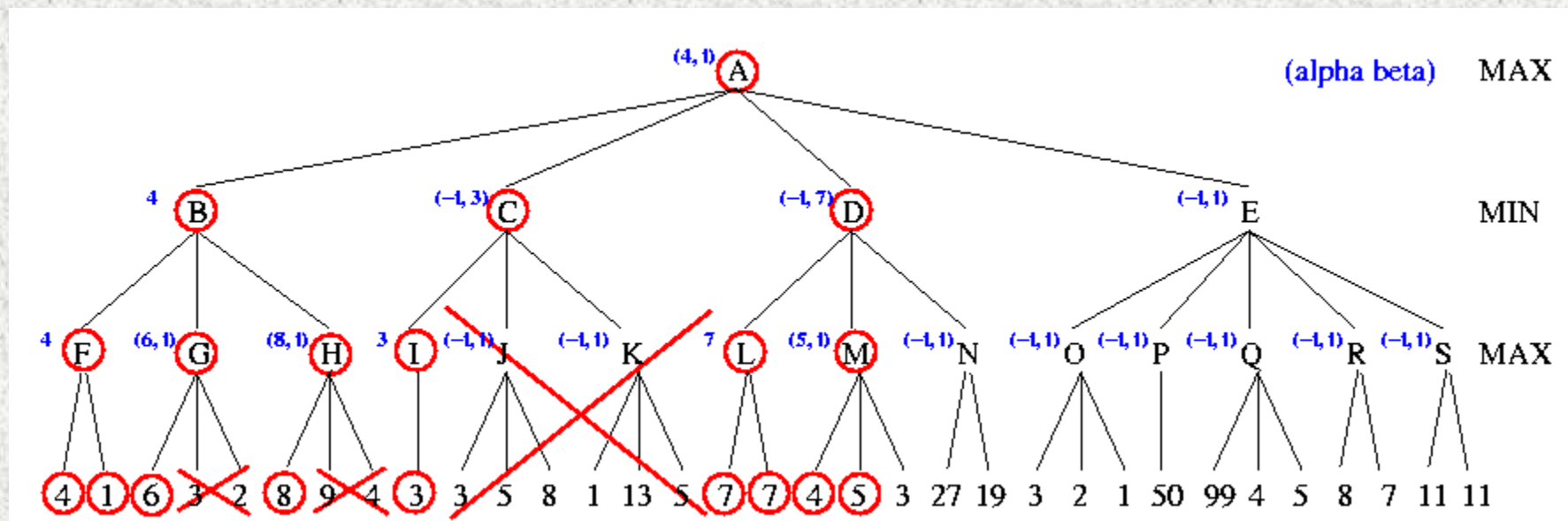
Example



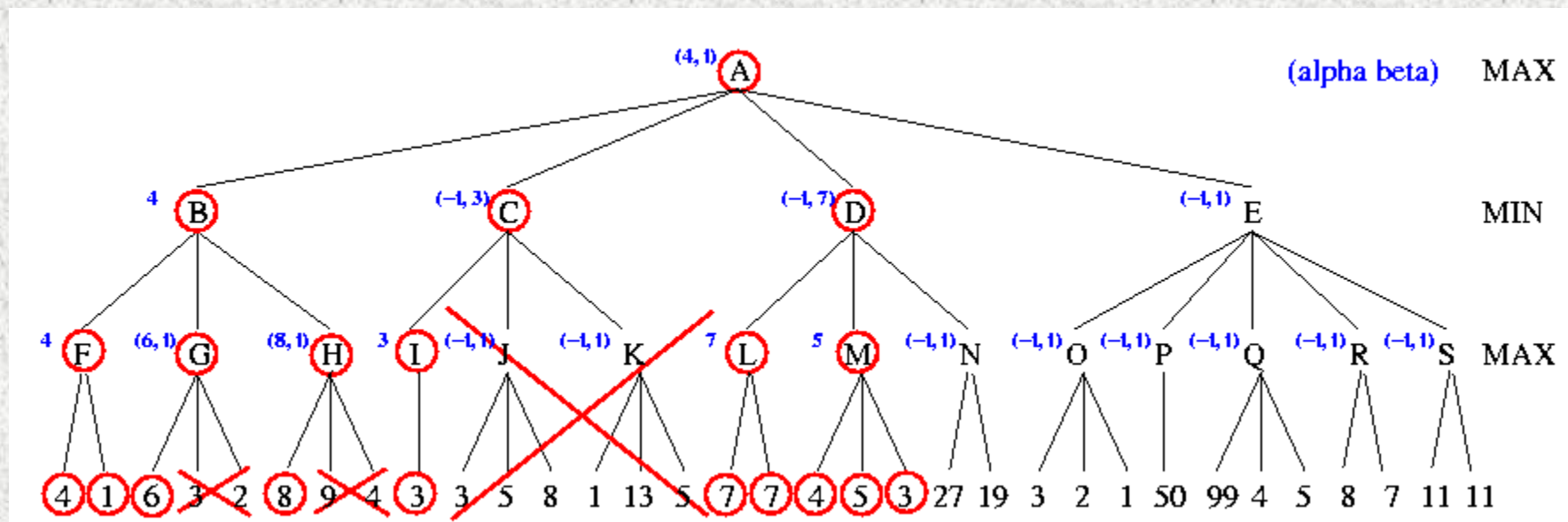
Example



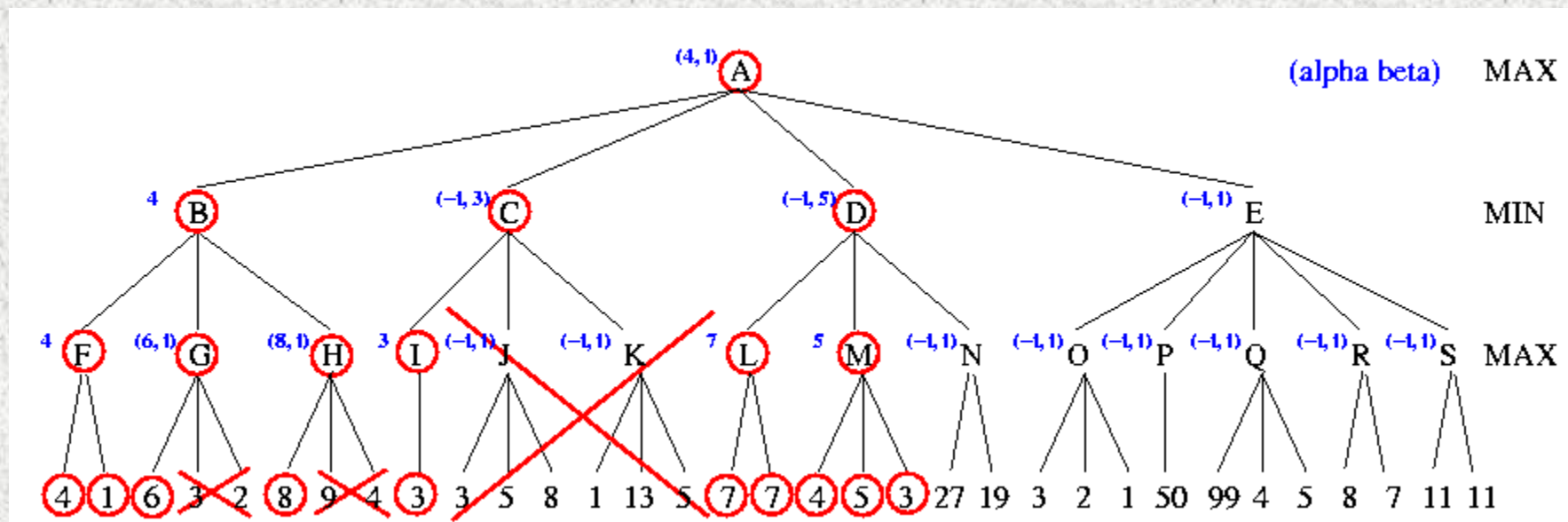
Example



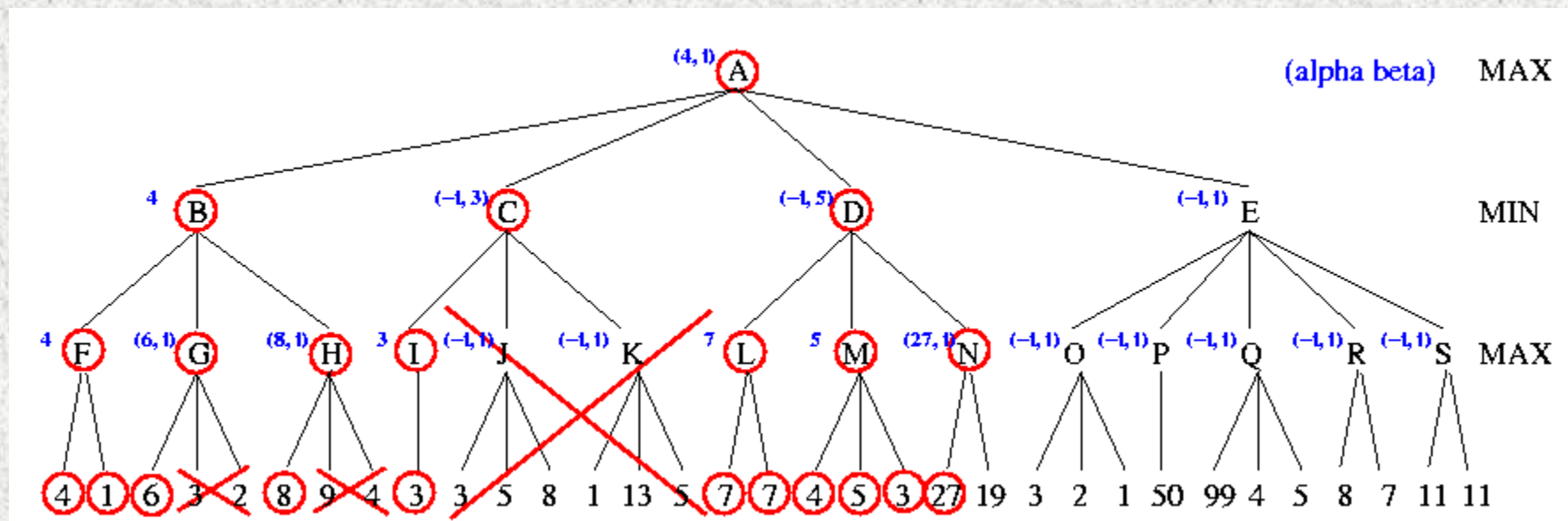
Example



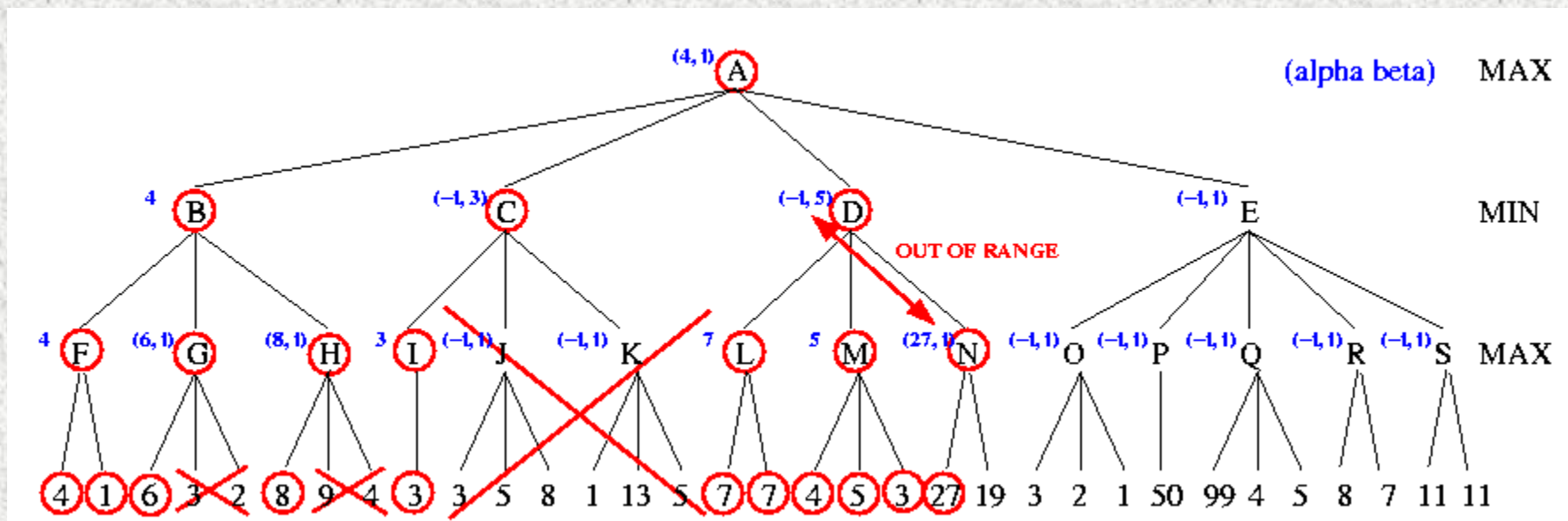
Example



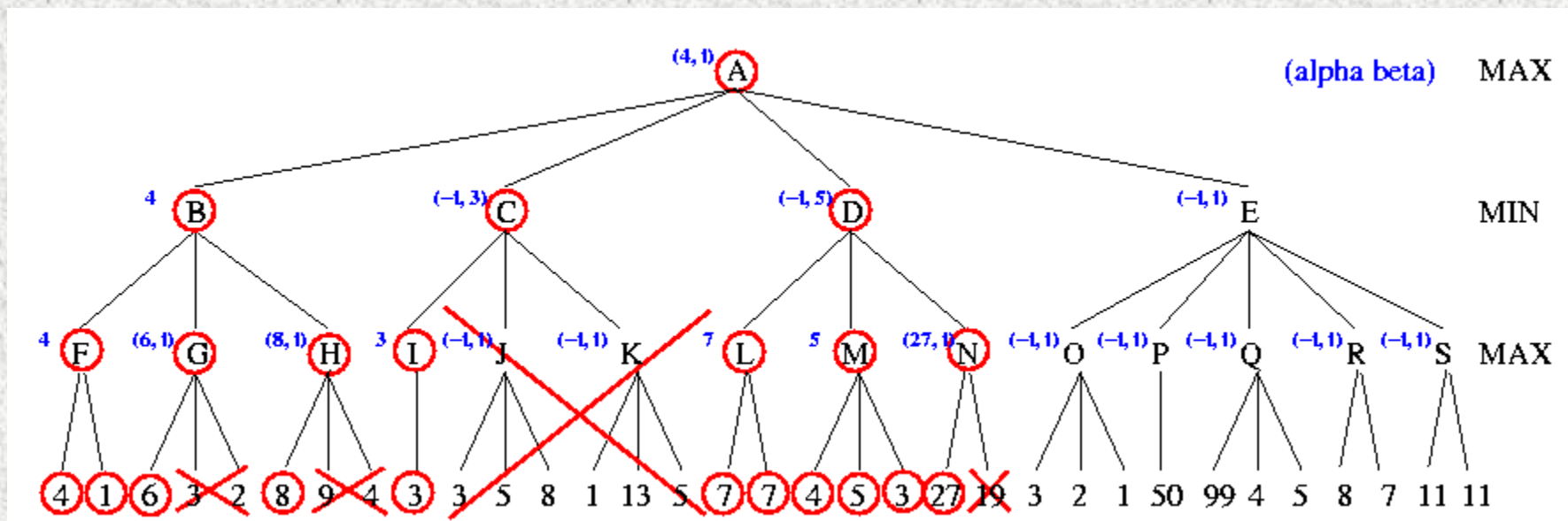
Example



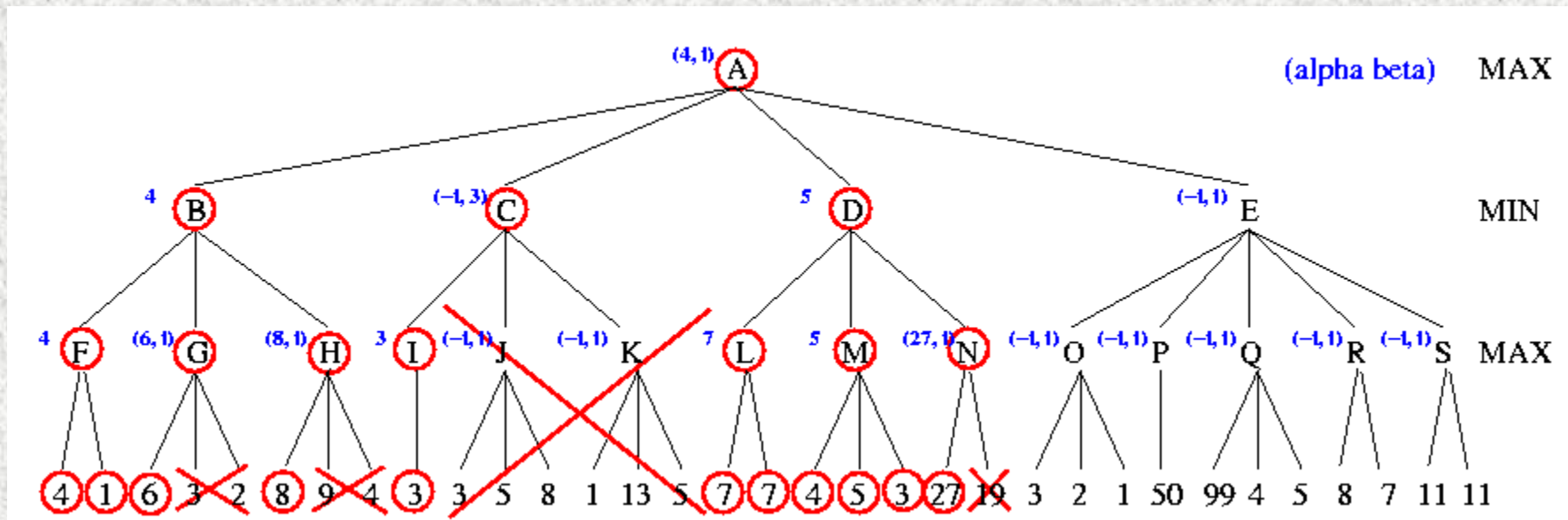
Example



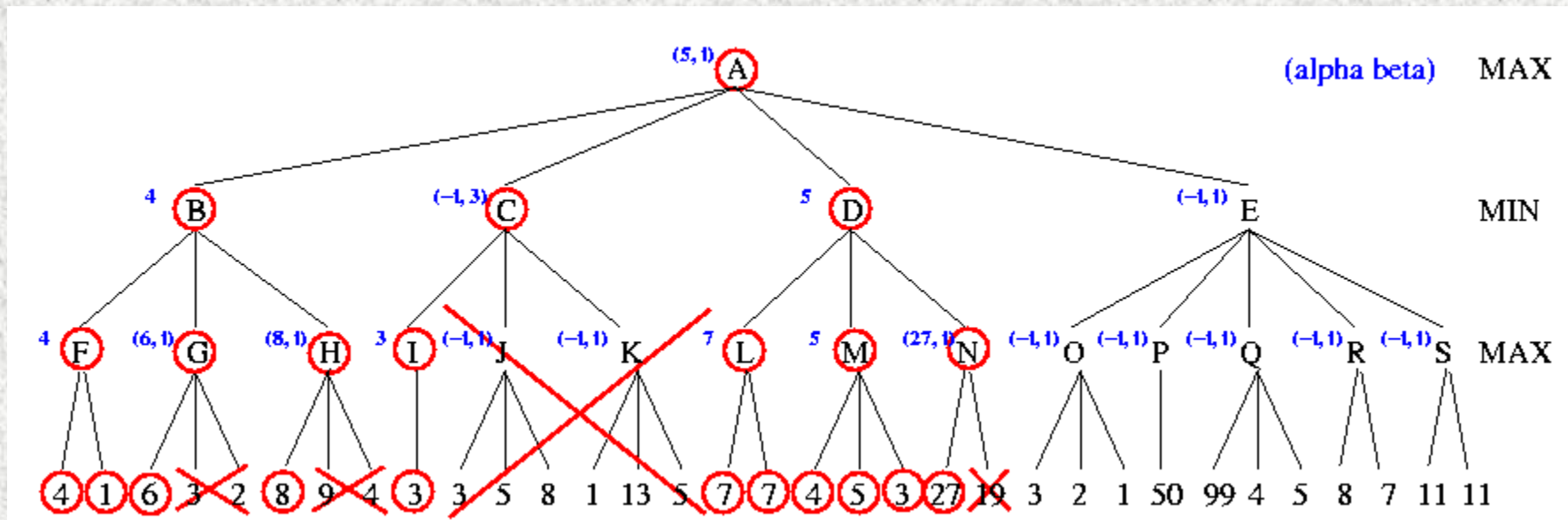
Example



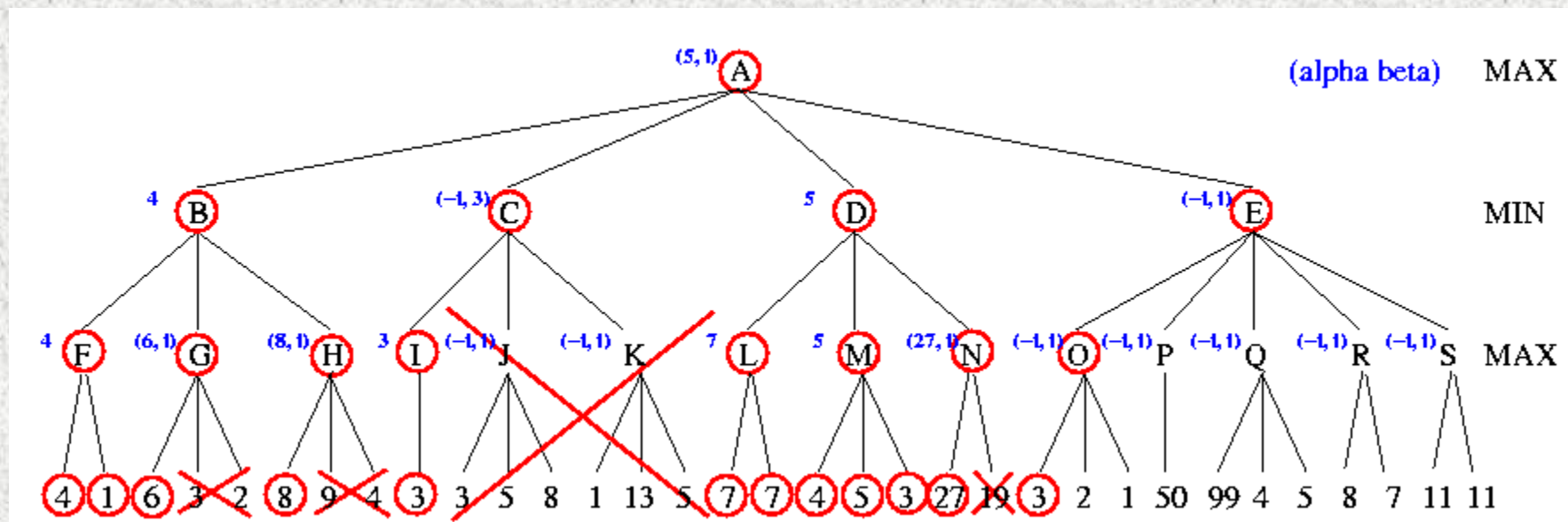
Example



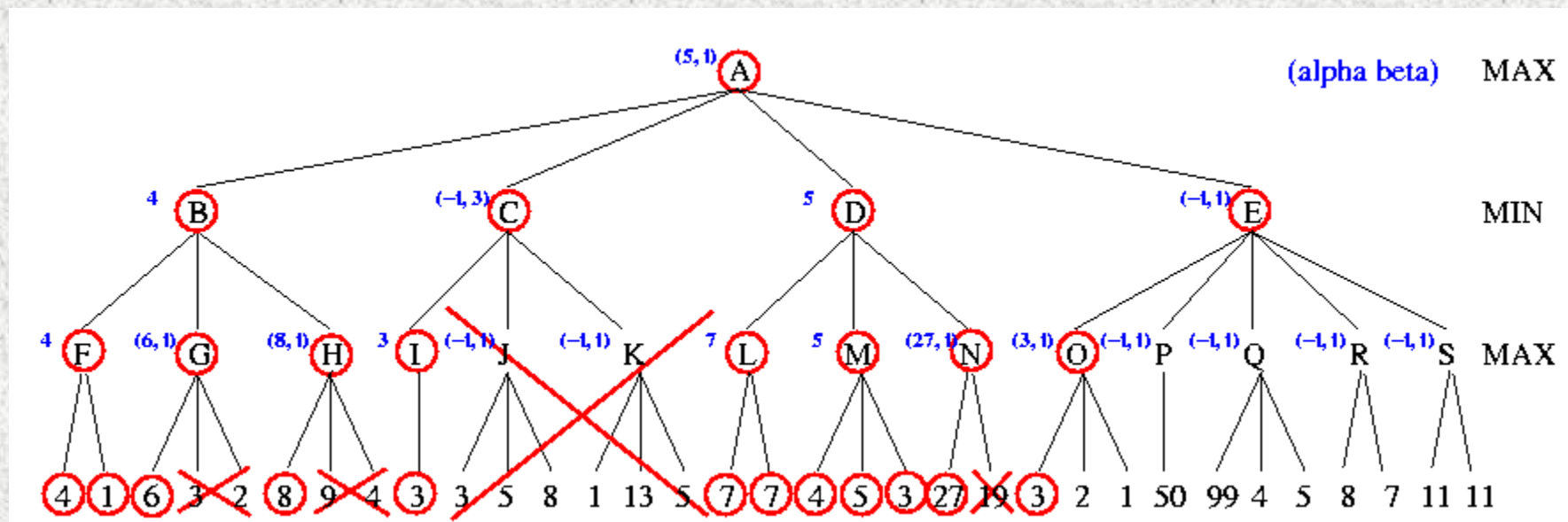
Example



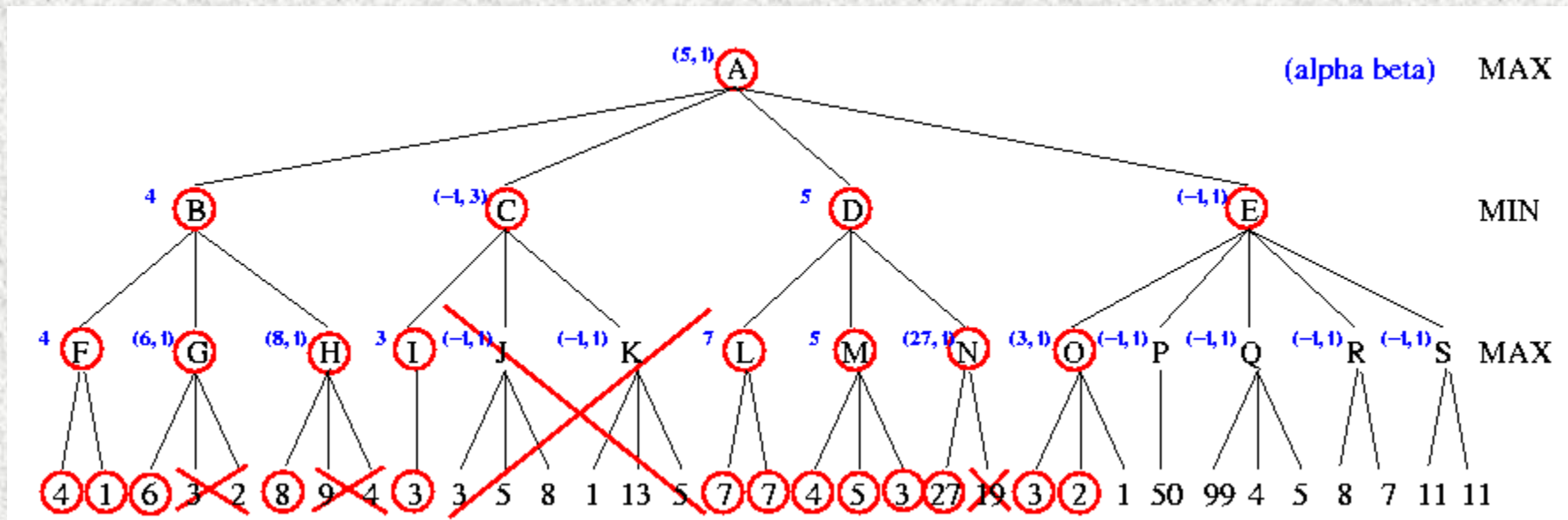
Example



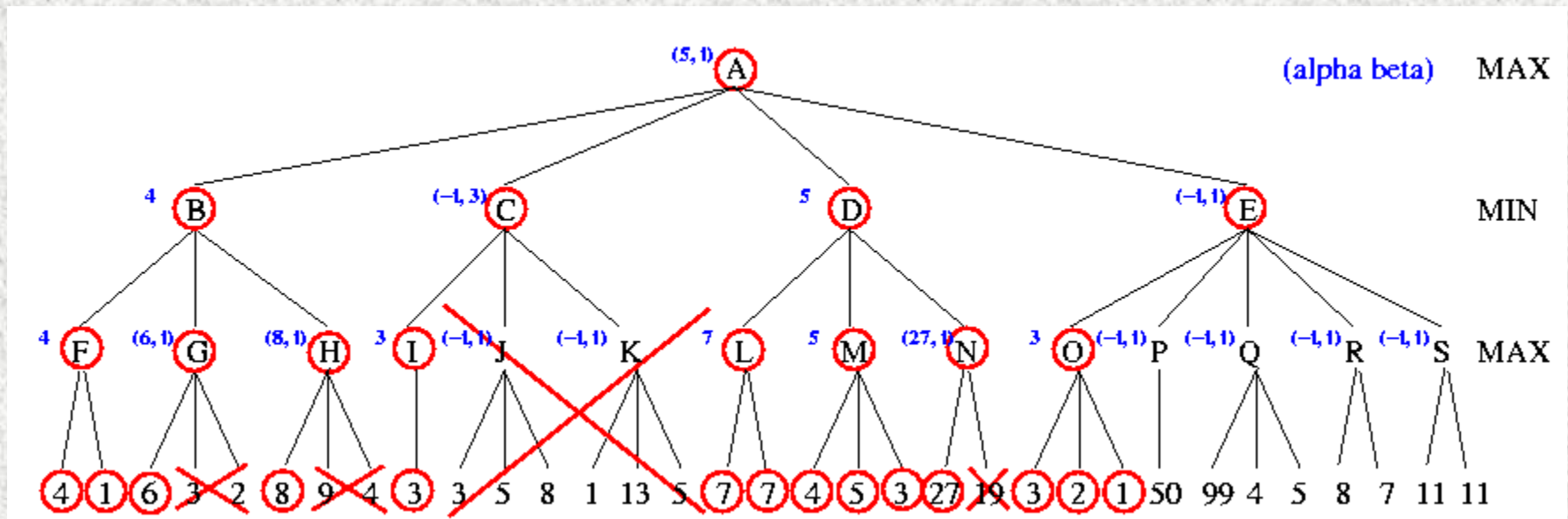
Example



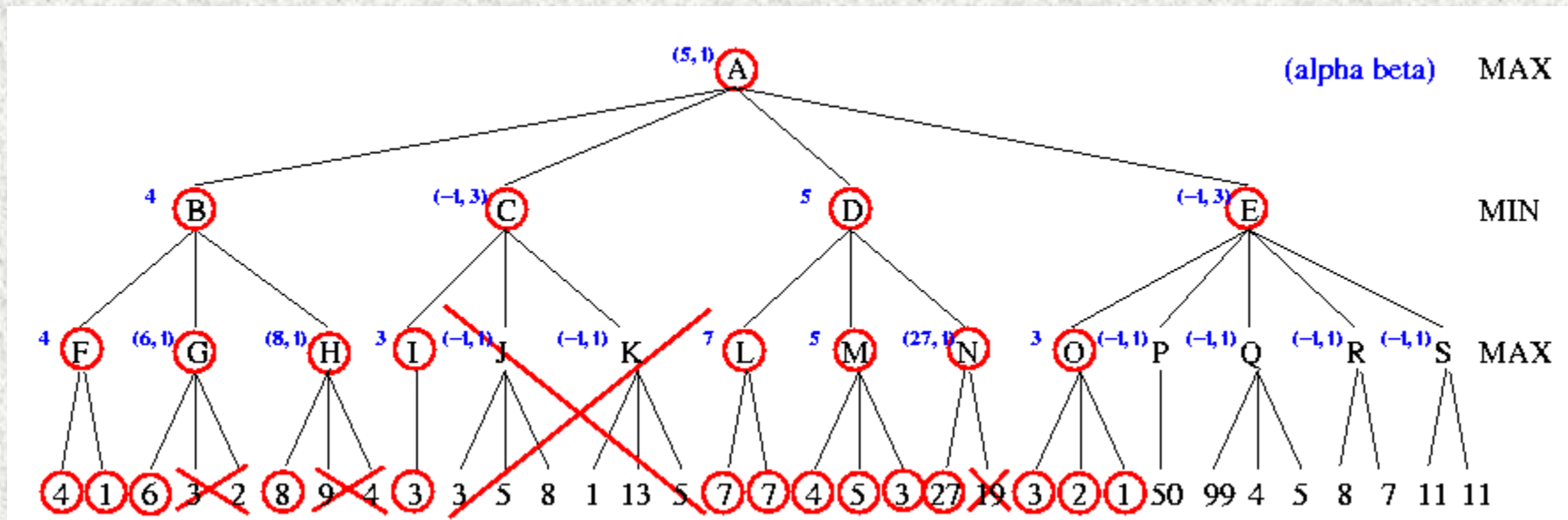
Example



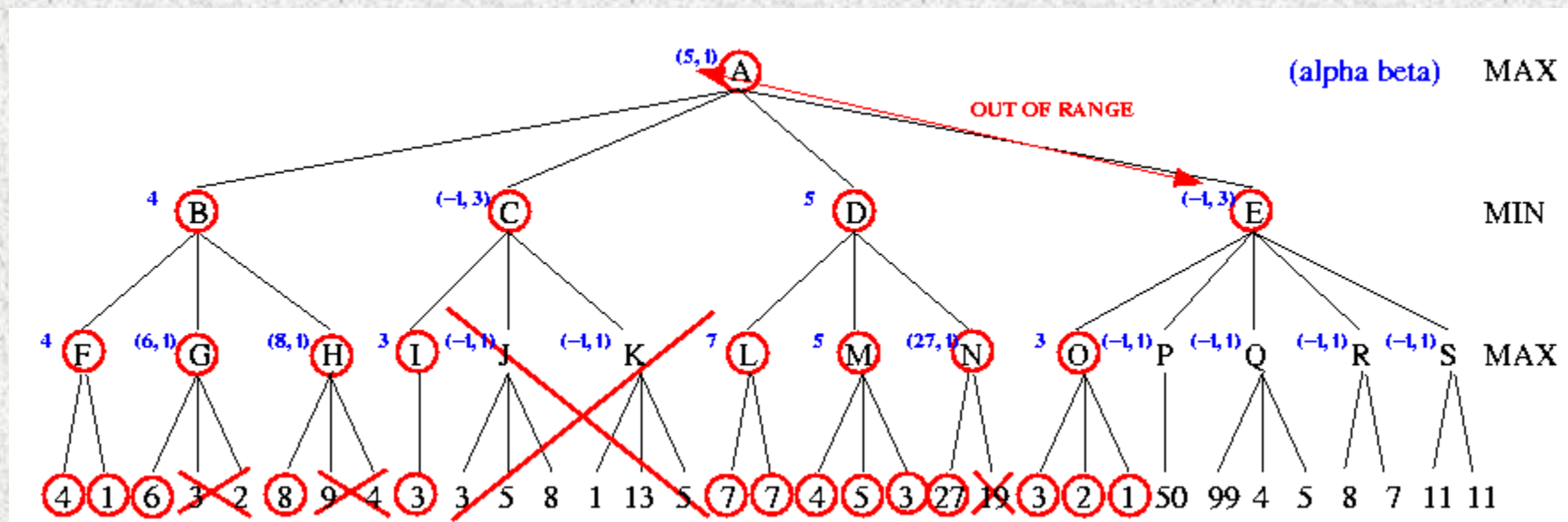
Example



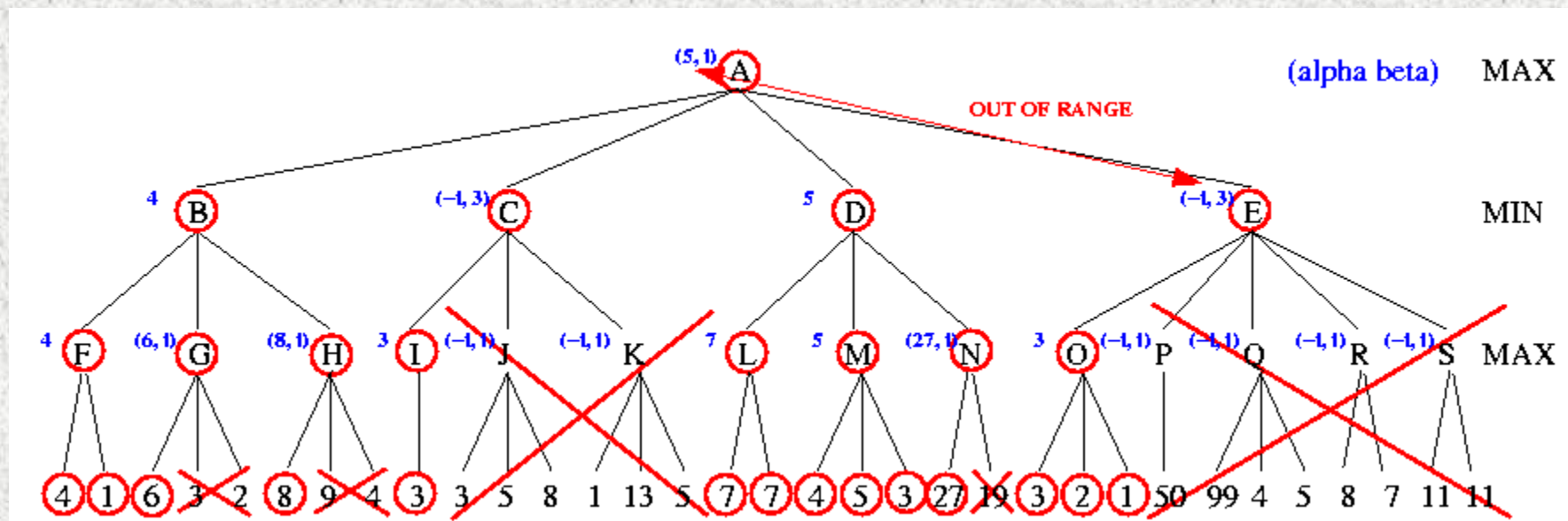
Example



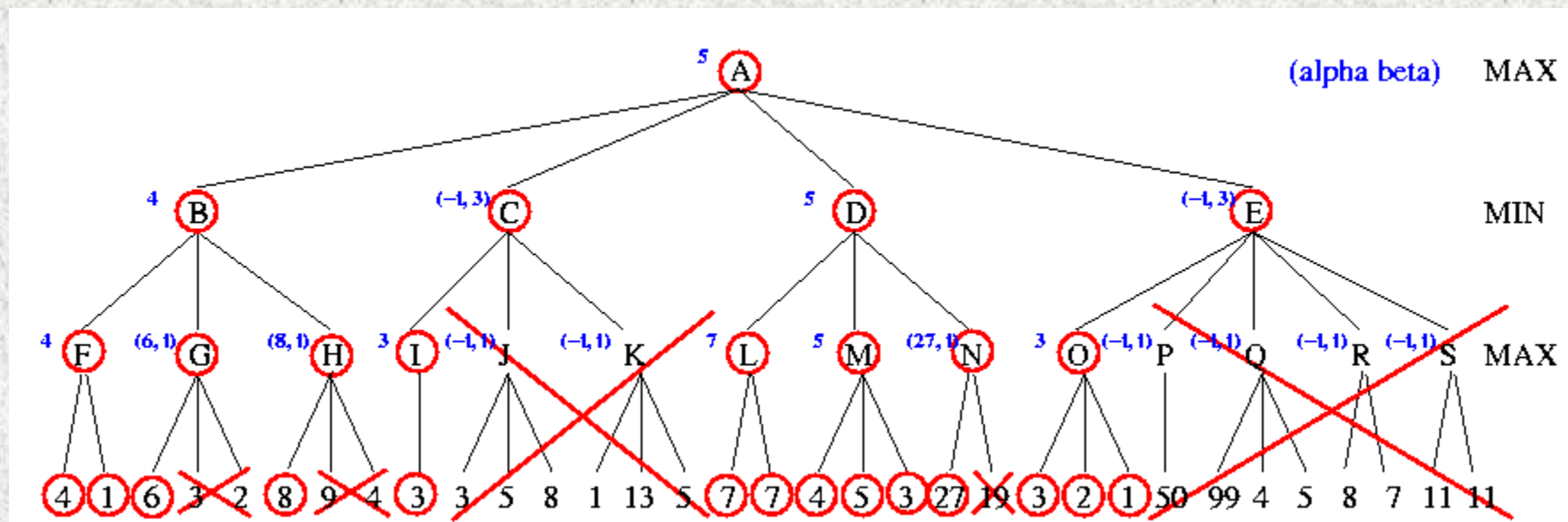
Example



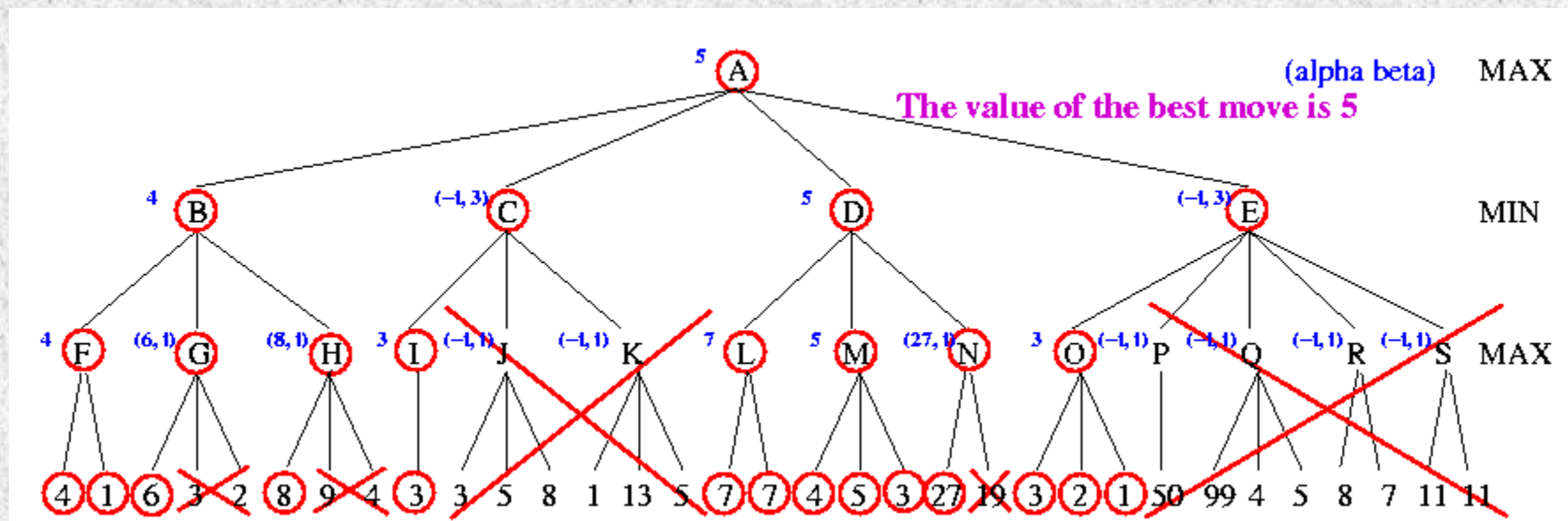
Example



Example



Example



Example

